

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Harry Pratama Yunus

NIM. 2310817210010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Harry Pratama Yunus

NIM : 2310817210010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL.....	7
MODUL 1 : ANDROID BASIC WITH KOTLIN.....	9
SOAL 1	9
A. Source Code.....	11
B. Output Program.....	19
C. Pembahasan.....	23
MODUL 2 : ANDROID LAYOUT	26
SOAL 1	26
A. Source Code.....	28
B. Output Program.....	36
C. Pembahasan	44
SOAL 2	47
A. Pembahasan	47
MODUL 3 : BUILD A SCROLLABLE LIST.....	48
SOAL 1	48
A. Source Code	50
B. Output Program	76
C. Pembahasan	80
SOAL 2	96
A. Pembahasan	96

MODUL 4 : VIEWMODEL AND DEBUGGING.....	97
SOAL 1	97
A. Source Code	98
B. Output Program	130
C. Pembahasan	138
SOAL 2	162
A. Pembahasan	162
MODUL 5 : CONNECT TO THE INTERNET.....	163
SOAL 1	163
A. Source Code	163
B. Output Program	187
C. Pembahasan	189
Tautan Git.....	208

DAFTAR GAMBAR

<i>Gambar 1. Tampilan Awal Aplikasi Modul 1</i>	<i>9</i>
<i>Gambar 2. Tampilan dadu setelah roll Modul 1</i>	<i>10</i>
<i>Gambar 3. Tampilan Roll Dadu Double Modul 1</i>	<i>10</i>
<i>Gambar 4. Screenshot Hasil menjalankan XML Modul 1</i>	<i>19</i>
<i>Gambar 5. Screenshot Hasil menjalankan XML Modul 1</i>	<i>20</i>
<i>Gambar 6. Screenshot Hasil menjalankan COMPOSE Modul 1</i>	<i>21</i>
<i>Gambar 7. Screenshot Hasil menjalankan COMPOSE Modul 1</i>	<i>22</i>
<i>Gambar 8. Tampilan awal aplikasi Modul 2</i>	<i>26</i>
<i>Gambar 9. Tampilan Pilihan Presentasi Tip Modul 2</i>	<i>27</i>
<i>Gambar 10. Tampilan Aplikasi Setelah Dijalankan Modul 2</i>	<i>27</i>
<i>Gambar 11. Screenshot Hasil Jawaban Soal 1 XML bagian a Modul 2</i>	<i>36</i>
<i>Gambar 12. Screenshot Hasil Jawaban Soal 1 XML bagian b Modul 2</i>	<i>37</i>
<i>Gambar 13. Screenshot Hasil Jawaban Soal 1 XML bagian c Modul 2</i>	<i>38</i>
<i>Gambar 14. Screenshot Hasil Jawaban Soal 1 XML bagian d Modul 2</i>	<i>39</i>
<i>Gambar 15. Screenshot Hasil Jawaban Soal 1 compose bagian a Modul 2</i>	<i>40</i>
<i>Gambar 16. Screenshot Hasil Jawaban Soal 1 compose bagian b Modul 2</i>	<i>41</i>
<i>Gambar 17. Screenshot Hasil Jawaban Soal 1 compose bagian c Modul 2</i>	<i>42</i>
<i>Gambar 18. Screenshot Hasil Jawaban Soal 1 compose bagian d Modul 2</i>	<i>43</i>
<i>Gambar 19. Contoh UI List Modul 3</i>	<i>49</i>
<i>Gambar 20. Contoh UI List Model 3</i>	<i>49</i>
<i>Gambar 21. Screenshot Hasil Jawaban Soal 1 XML bagian a Modul 3</i>	<i>76</i>
<i>Gambar 22. Screenshot Hasil Jawaban Soal 1 XML bagian b Modul 3</i>	<i>77</i>
<i>Gambar 23. Screenshot Hasil Jawaban Soal 1 compose bagian a Modul 3</i>	<i>78</i>
<i>Gambar 24. Screenshot Hasil Jawaban Soal 1 compose bagian b Modul 3</i>	<i>79</i>
<i>Gambar 25. Contoh Penggunaan Debugger Modul 4</i>	<i>97</i>
<i>Gambar 26. Screenshot Hasil Jawaban Soal 1 XML bagian a Modul 4</i>	<i>130</i>
<i>Gambar 27. Screenshot Hasil Jawaban Soal 1 XML bagian b Modul 4</i>	<i>131</i>
<i>Gambar 28. Screenshot Hasil Jawaban Soal 1 compose bagian a Modul 4</i>	<i>132</i>
<i>Gambar 29. Screenshot Hasil Jawaban Soal 1 compose bagian b Modul 4</i>	<i>133</i>
<i>Gambar 30. Screenshot Hasil Jawaban Soal 1 compose Log saat data item masuk ke dalam list Modul 4</i>	<i>134</i>
<i>Gambar 31. Screenshot Hasil Jawaban Soal 1 compose Log saat tombol Detail dan Log data dari list yang dipilih ketika berpindah ke halaman Detail Modul 4</i>	<i>134</i>
<i>Gambar 32. Screenshot Hasil Jawaban Soal 1 compose Log saat tombol Explicit Intent ditekan Modul 4</i>	<i>134</i>
<i>Gambar 33. Screenshot Hasil Jawaban Soal 1 compose debugger Modul 4</i>	<i>134</i>
<i>Gambar 34. Screenshot Hasil Jawaban Soal 1 compose debugger step into Modul 4</i>	<i>135</i>

<i>Gambar 35. Screenshot Hasil Jawaban Soal 1 compose debugger step out Modul 4</i>	135
<i>Gambar 36. Screenshot Hasil Jawaban Soal 1 compose debugger step over Modul 4</i>	136
<i>Gambar 37. Screenshot Hasil Jawaban Soal 1 xml Log saat data item masuk ke dalam list Modul 4</i>	136
<i>Gambar 38. Screenshot Hasil Jawaban Soal 1 xml Log saat tombol Detail dan Log data dari list yang dipilih ketika berpindah ke halaman Detail Modul 4</i>	136
<i>Gambar 39. Screenshot Hasil Jawaban Soal 1 xml Log saat tombol Explicit Intent ditekan Modul 4</i>	136
<i>Gambar 40. Screenshot Hasil Jawaban Soal 1 xml debugger Modul 4</i>	137
<i>Gambar 41. Screenshot Hasil Jawaban Soal 1 xml step into Modul 4</i>	137
<i>Gambar 42. Screenshot Hasil Jawaban Soal 1 xml step out Modul 4</i>	137
<i>Gambar 43. Screenshot Hasil Jawaban Soal 1 xml step over Modul 4</i>	138
<i>Gambar 44. Screenshot output light mode Modul 5</i>	187
<i>Gambar 45. Screenshot output dark mode Modul 5</i>	188

DAFTAR TABEL

Table 1. Source code Main.activity.kt XML Modul 1	11
Table 2. Source code activity_main.xml XML Modul 1	13
Table 3. Source code Main.activity.kt COMPOSE Modul 1	15
Table 4. source code soal 1 mainactivity.kt Model 2	28
Table 5. source code soal 1 acivity_main.xml Model 2	29
Table 6. source code soal 1 mainactivity.kt(compose) Modul 2	32
Table 7. source code soal 1 Games.kt (xml) Modul 3	50
Table 8. source code soal 1 DataSource.kt (xml) Model 3	50
Table 9. source code soal 1 GamesAdapter.kt (xml) Modul 3	51
Table 10. source code soal 1 GamesListFragment.kt (xml) Modul 3	52
Table 11. source code soal 1 DetailFragment.kt (xml) Modul 3	54
Table 12. source code soal 1 mainactivity.kt (xml) Modul 3	55
Table 13. source code soal 1 acivity_main.xml (xml) Modul 3	55
Table 14. source code soal 1 fragment_games_list.xml (xml) Modul 3	55
Table 15. source code soal 1 fragment_detail.xml (xml) Modul 3	56
Table 16. source code soal 1 item_games.xml (xml) Modul 3	57
Table 17. source code soal 1 strings.xml (xml) Modul 3	60
Table 18. source code soal 1 style.xml (xml) Modul 3	62
Table 19. source code soal 1 mainactivity.kt (compose) Modul 3	62
Table 20. source code soal 1 HomeScreen.kt (compose) Modul 3	64
Table 21. source code soal 1 DetailScreen.kt (compose) Modul 3	64
Table 22. source code soal 1 GameCard.kt (compose) Modul 3	67
Table 23. source code soal 1 Datasource.kt (compose) Modul 3	71
Table 24. source code soal 1 strings.xml (compose) Modul 3	72
Table 25. source code soal 1 Games.kt (compose) Modul 3	75
Table 26. source code soal 1 Games.kt (xml) Modul 4	98
Table 27. source code soal 1 DataSource.kt (xml) Modul 4	98
Table 28. source code soal 1 GamesAdapter.kt (xml) Modul 4	99
Table 29. source code soal 1 GamesListFragment.kt (xml) Modul 4	100
Table 30. source code soal 1 DetailFragment.kt (xml) Modul 4	103
Table 31. source code soal 1 mainactivity.kt (xml) Modul 4	104
Table 32. source code soal 1 acivity_main.xml (xml) Modul 4	104
Table 33. source code soal 1 fragment_games_list.xml (xml) Modul 4	104
Table 34. source code soal 1 fragment_detail.xml (xml) Modul 4	105
Table 35. source code soal 1 item_games.xml (xml) Modul 4	106
Table 36. source code soal 1 strings.xml (xml) Modul 4	109
Table 37. source code soal 1 style.xml (xml) Modul 4	111
Table 38. source code soal 1 GamesViewModel.kt (xml) Modul 4	111
Table 39. source code soal 1 GamesViewModelFactory.kt (xml) Modul 4	112
Table 40. source code soal 1 MyApplication.kt (xml) Modul 4	112

<i>Table 41. source code soal 1 mainactivity.kt (compose) Modul 4</i>	<i>113</i>
<i>Table 42. source code soal 1 HomeScreen.kt (compose) Modul 4</i>	<i>115</i>
<i>Table 43. source code soal 1 DetailScreen.kt (compose) Modul 4</i>	<i>116</i>
<i>Table 44. source code soal 1 GameCard.kt (compose) Modul 4</i>	<i>119</i>
<i>Table 45. source code soal 1 Datasource.kt (compose) Modul 4</i>	<i>123</i>
<i>Table 46. source code soal 1 strings.xml (compose) Modul 4</i>	<i>124</i>
<i>Table 47. source code soal 1 Games.kt (compose) Modul 4</i>	<i>127</i>
<i>Table 48. source code soal 1 GamesViewModel.kt (compose) Modul 4</i>	<i>127</i>
<i>Table 49. source code soal 1 GamesViewModelFactory.kt (compose) Modul 4</i>	<i>128</i>
<i>Table 50. source code soal 1 MyFavoriteGamesApplication.kt (compose) Modul 4</i>	<i>129</i>
<i>Table 51. Source code MovieDao.kt Modul 5</i>	<i>163</i>
<i>Table 52. Source code MovieDatabase.kt Modul 5</i>	<i>164</i>
<i>Table 53. Source code MovieEntity.kt Modul 5</i>	<i>165</i>
<i>Table 54. Source code PreferencesManager.kt Modul 5</i>	<i>165</i>
<i>Table 55. Source code ApiService.kt Modul 5</i>	<i>166</i>
<i>Table 56. Source code MovieDao.kt Modul 5</i>	<i>166</i>
<i>Table 57. Source code RetrofitInstance.kt Modul 5</i>	<i>167</i>
<i>Table 58. Source code MovieRepositoryImpl.kt Modul 5</i>	<i>167</i>
<i>Table 59. Source code Movie.kt Modul 5</i>	<i>169</i>
<i>Table 60. Source code MovieRepository.kt Modul 5</i>	<i>169</i>
<i>Table 61. Source code MovieUseCase.kt Modul 5</i>	<i>169</i>
<i>Table 62. Source code DetailScreen.kt Modul 5</i>	<i>170</i>
<i>Table 63. Source code MainScreen.kt Modul 5</i>	<i>174</i>
<i>Table 64. Source code MovieCard.kt Modul 5</i>	<i>176</i>
<i>Table 65. Source code TopBarApp.kt Modul 5</i>	<i>180</i>
<i>Table 66. Source code MainActivity.kt Modul 5</i>	<i>181</i>
<i>Table 67. Source code MovieViewModel.kt Modul 5</i>	<i>183</i>
<i>Table 68. Source code MovieViewModelFactory.kt Modul 5</i>	<i>184</i>
<i>Table 69. Source code ThemeViewModel.kt Modul 5</i>	<i>185</i>
<i>Table 70. Source code ThemeViewModelFactory.kt Modul 5</i>	<i>185</i>
<i>Table 71. Source code NetworkResult.kt Modul 5</i>	<i>186</i>

MODUL 1 : ANDROID BASIC WITH KOTLIN

SOAL 1

Soal Praktikum:

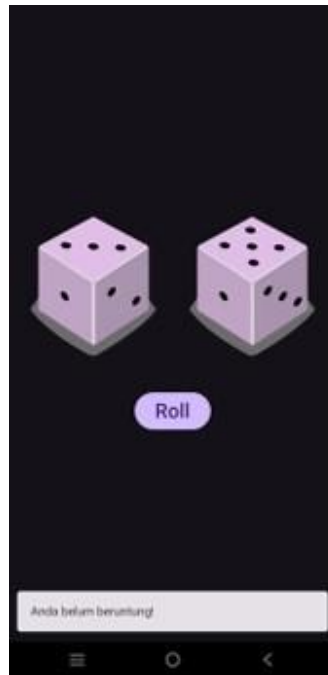
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



Gambar 1. Tampilan Awal Aplikasi Modul 1

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan dadu setelah roll Modul 1

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double Modul 1

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.

5. Upload aplikasi yang telah anda buat kedalam repository GitHub ke dalam **folder Module 1 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download

A. Source Code

XML :

MainActivity.kt

Table 1. Source code Main.activity.kt XML Modul 1

1	package com.pemrogamanmobile.mydadu
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import android.widget.Toast
7	import androidx.appcompat.app.AppCompatActivity
8	import kotlin.random.Random
9	
10	class MainActivity : AppCompatActivity() {
11	
12	private lateinit var dice1Image: ImageView
13	private lateinit var dice2Image: ImageView
14	private lateinit var rollButton: Button
15	
16	override fun onCreate(savedInstanceState: Bundle?) {
17	super.onCreate(savedInstanceState)
18	setContentView(R.layout.activity_main)

19	
20	dice1Image = findViewById(R.id.dice1Image)
21	dice2Image = findViewById(R.id.dice2Image)
22	rollButton = findViewById(R.id.rollButton)
23	
24	rollButton.setOnClickListener {
25	rollDice()
26	}
27	}
28	
29	private fun rollDice() {
30	val dadu1 = buatangkadadu()
31	val dadu2 = buatangkadadu()
32	
33	dice1Image.setImageResource(ambilgambardadu(dadu1))
34	dice2Image.setImageResource(ambilgambardadu(dadu2))
35	
36	val resultMessage = if (dadu1 == dadu2) {
37	"Selamat, anda dapat dadu double!"
38	} else {
39	"Anda belum beruntung!"
40	}
41	
42	Toast.makeText(this, resultMessage,
	Toast.LENGTH_SHORT).show()
43	}
44	
45	private fun buatangkadadu(): Int {

46	return Random.nextInt(1, 7)
47	}
48	
49	private fun ambilgambardadu(number: Int): Int {
50	return when (number) {
51	1 -> R.drawable.dice_1
52	2 -> R.drawable.dice_2
53	3 -> R.drawable.dice_3
54	4 -> R.drawable.dice_4
55	5 -> R.drawable.dice_5
56	6 -> R.drawable.dice_6
57	else -> R.drawable.dice_0
58	}
59	}
60	}

activity_main.xml

Table 2. Source code activity_main.xml XML Modul 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:id="@+id/mainLayout"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:padding="16dp"
7	android:gravity="center"
8	android:orientation="vertical">
9	
10	<LinearLayout

11	android:layout_width="315dp"
12	android:layout_height="150dp"
13	android:layout_marginBottom="50dp"
14	android:gravity="center"
15	android:orientation="horizontal">
16	
17	<ImageView
18	android:id="@+id/dice1Image"
19	android:layout_width="150dp"
20	android:layout_height="150dp"
21	android:contentDescription="Dadu 1"
22	android:src="@drawable/dice_0" />
23	
24	<ImageView
25	android:id="@+id/dice2Image"
26	android:layout_width="150dp"
27	android:layout_height="150dp"
28	android:layout_marginStart="16dp"
29	android:contentDescription="Dadu 2"
30	android:src="@drawable/dice_0" />
31	</LinearLayout>
32	
33	<Button
34	android:id="@+id/rollButton"
35	android:layout_width="wrap_content"
36	android:layout_height="wrap_content"
37	android:text="Roll"
38	android:backgroundTint="#d9cbde"

39	android:textColor="@color/black"
40	android:paddingHorizontal="24dp"
41	android:paddingVertical="12dp"/>
42	</LinearLayout>

COMPOSE :

MainActivity.kt

Table 3. Source code Main.activity.kt COMPOSE Modul 1

1	package com.tugaspraktikum1.mydadu
2	
3	import android.os.Bundle
4	import android.widget.Toast
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.compose.foundation.Image
8	import androidx.compose.foundation.layout.*
9	import androidx.compose.foundation.shape.RoundedCornerShape
10	import androidx.compose.material3.Button
11	import androidx.compose.material3.Text
12	import androidx.compose.runtime.*
13	import androidx.compose.ui.Alignment
14	import androidx.compose.ui.Modifier
15	import androidx.compose.ui.layout.ContentScale
16	import androidx.compose.ui.res.painterResource
17	import androidx.compose.ui.unit.dp
18	import com.tugaspraktikum1.mydadu.ui.theme.MyDaduTheme
19	import kotlin.random.Random
20	
21	class MainActivity : ComponentActivity() {

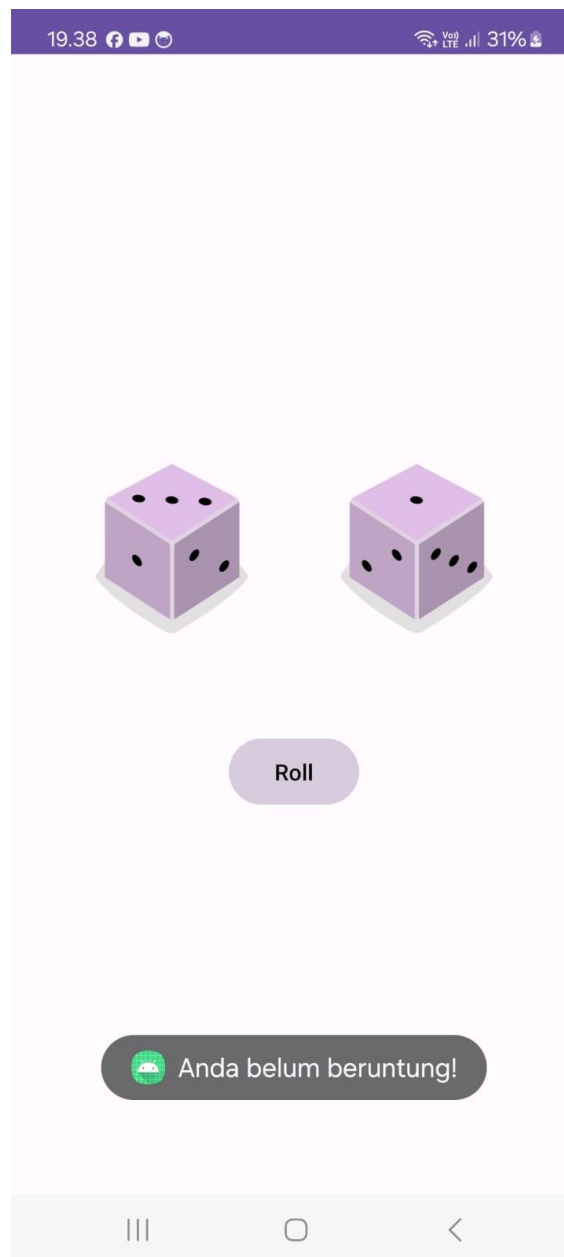
22	override fun onCreate(savedInstanceState: Bundle?) {
23	super.onCreate(savedInstanceState)
24	setContent {
25	MyDaduTheme {
26	DiceGame { message ->
27	Toast.makeText(this, message,
28	Toast.LENGTH_SHORT).show()
29	}
30	}
31	}
32	}
33	
34	@Composable
35	fun DiceGame(onResult: (String) -> Unit) {
36	
37	var dadu1 by remember { mutableIntStateOf(0) }
38	var dadu2 by remember { mutableIntStateOf(0) }
39	
40	fun rollDice() {
41	dadu1 = buatangkadadu()
42	dadu2 = buatangkadadu()
43	val resultMessage = if (dadu1 == dadu2) {
44	"Selamat, anda dapat dadu double!"
45	} else {
46	"Anda belum beruntung!"
47	}
48	onResult(resultMessage)
	}

49	
50	Column(
51	modifier = Modifier
52	.fillMaxSize()
53	.padding(16.dp),
54	horizontalAlignment = Alignment.CenterHorizontally,
55	verticalArrangement = Arrangement.Center
56	) {
57	Row(
58	horizontalArrangement = Arrangement.SpaceEvenly,
59	modifier = Modifier.fillMaxWidth()
60	) {
61	DiceView(sisidadu = dadu1)
62	DiceView(sisidadu = dadu2)
63	}
64	
65	Spacer(modifier = Modifier.height(16.dp))
66	
67	Button(
68	onClick = { rollDice() },
69	shape = RoundedCornerShape(8.dp),
70	modifier = Modifier.padding(8.dp)
71	) {
72	Text(text = "Roll")
73	}
74	}
75	}
76	

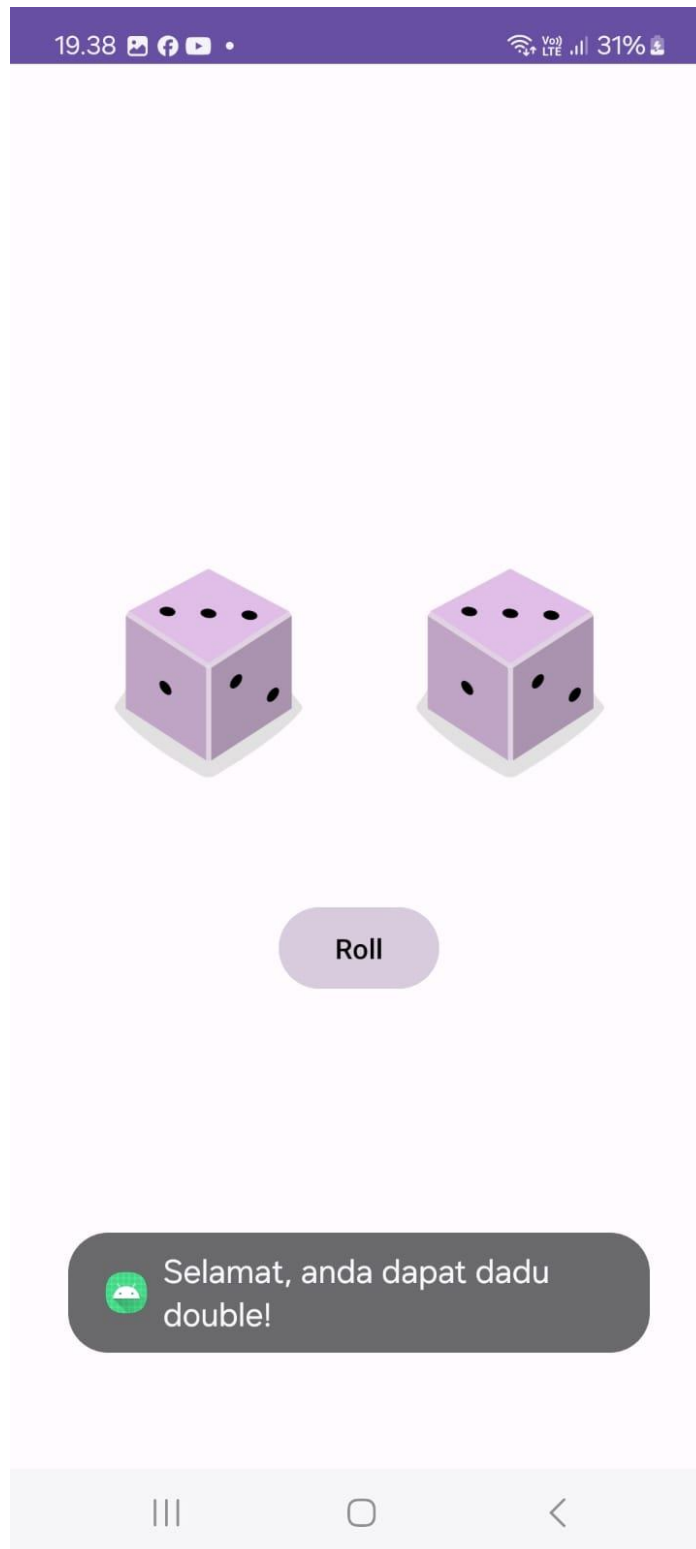
77	
78	@Composable
79	fun DiceView(sisidadu : Int) {
80	Column(horizontalAlignment = Alignment.CenterHorizontally) {
81	Image(
82	painter = painterResource(id =
83	ambilgambardadu(sisidadu)),
84	contentDescription = "tampilan dadu adalah yang ke \$sisidadu",
85	modifier = Modifier.size(100.dp),
86	contentScale = ContentScale.Fit
87	)
88	}
89	}
90	
91	fun buatangkadadu(): Int {
92	val angkadadu = Random.nextInt(1, 7)
93	return angkadadu
94	}
95	
96	fun ambilgambardadu(number: Int): Int {
97	return when (number) {
98	1 -> R.drawable.dice_1
99	2 -> R.drawable.dice_2
100	3 -> R.drawable.dice_3
101	4 -> R.drawable.dice_4
102	5 -> R.drawable.dice_5
103	6 -> R.drawable.dice_6
	else -> R.drawable.dice_0

104	}
105	}

B. Output Program



Gambar 4. Screenshot Hasil menjalankan XML Modul 1



Gambar 5. Screenshot Hasil menjalankan XML Modul 1



Gambar 6. Screenshot Hasil menjalankan COMPOSE Modul 1



Gambar 7. Screenshot Hasil menjalankan COMPOSE Modul 1

C. Pembahasan

XML :

MainActivity.kt:

Pada baris ke [1] terdapat “ package com.pemrogramanmobile.mydadu” yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file.

Pada baris ke [3] sampai [8], berfungsi untuk mengimport beberapa kelas yang akan digunakan, yaitu bundle, button, imageview, toast, appCompatActivity dan random.

Pada baris ke [10] terdapat class MainActivity : AppCompatActivity() { yang berfungsi untuk membuat class mainactivity yang merupakan activity utama aplikasi ini.

Pada baris ke [12] sampai [14] berfungsi untuk membuat variabel yang akan menyimpan gambar pada 2 dadu dan satu button.(lateinit digunakan untuk menunda inisialisasi sebelum button ditekan.

Pada baris ke [16] sampai [18] berfungsi untuk memanggil activity yang pertama kali dibuat, kemudian juga menampilkan layout yang diambil dari activity_main.xml.

Pada baris ke [20] sampai [22] berfungsi untuk menghubungkan variabel di kode ini ke file xml berdasarkan idnya.

Pada baris ke [24] sampai [26] berfungsi untuk menjalankan fungsi rolldice(), Ketika button diklik.

Pada baris ke [29] sampai [31] berfungsi untuk memanggil fungsi buatangkadadu() sebanyak dua kali dan dimasukkan ke variabel dadu1 dan dadu2.

Pada baris ke [33] sampai [34] berfungsi untuk menampilkan tampilan dadu berdasarkan angka yang didapat dari variabel dadu 2. Dan ambilgambardadu() digunakan untuk mengambil gambarnya dari folder drawable.

Pada baris ke [36] sampai [40] berfungsi untuk mengecek apakah angka dadu sama atau tidak, kemudian hasil teksnya di simpan di resultmessage.

Pada baris ke [42] terdapat “Toast.makeText(this, resultMessage, Toast.LENGTH_SHORT).show()”, yang berfungsi untuk menampilkan resultmessage menggunakan toast.

Pada baris ke [45] sampai [47] berfungsi untuk mengembalikan angka secaraacak dari rentang 1 sampai 6.

Pada baris ke [49] sampai [59] berfungsi untuk mengecek angka yang diambil sebelumnya dan mengembalikan gambar sesuai angka yang dicek tadi.

activity_main.xml:

Pada baris ke [1] terdapat “<?xml version="1.0" encoding="utf-8"?>”, yang berfungsi untuk menjadi pertanda bahwa file ini berformat xml dengan encoding UTF-8.

Pada baris ke [2] terdapat “<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android", yang berfungsi untuk membuat elemen tersusun secara linear bisa vertical atau horizontal. Kemudian xmlns:android digunakan agar atribut seperti android:layout_width dapat dikenali oleh android.

Pada baris ke [3] sampai [8] berfungsi untuk menjadi id mainlayout pada layout utama dan bisa digunakan di Kotlin. Kemudian match_parent digunakan untuk mengisi seluruh lebar dan tinggi layer. Padding ditambahkan supaya konten tidak mepet di tepi layer, memusatkan semua elemen ke tengah dan membuat linierlayout secara vertical.

Pada baris ke [10] sampai [15] berfungsi untuk menjadi tempat gambar dadu yang disusun secara horizontal di sub-layoutnya, lebar dari tempatnya ditentukan menjadi 315dp dan tingginya 150 dp, memberikan jarak kebawah supaya tombol tidak terlalu dekat dan gravity="center" digunakan untuk memusatkan gambarnya.

Pada baris ke [17] sampai [30] berfungsi untuk menampilkan dadu sesuai drawable sumbernya dan dikasih id dan contentdescription. Kemudian menambahkan marginstart untuk memberikan jarak untuk gambar dadu 1 ke gambar dadu 2.

Pada baris ke [33] sampai [41] berfungsi untuk menjadi tombol dan dikasih text berupa Roll, diberi warna tombol ungu muda, warna teks diubah menjadi hitam dan memberikan bentuk pada tombol menggunakan paddinghorizontal dan paddingvertical.

COMPOSE :

MainActivity.kt:

Pada baris ke [1] terdapat “ package com.tugaspraktikum1.mydadu” yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file.

Pada baris ke [3] sampai [19], berfungsi untuk mengimport beberapa kelas yang akan digunakan, yaitu bundle, button, image, toast, random layout, dll.

Pada baris ke [21] sampai [23] berfungsi untuk membuat class mainactivity yang merupakan activity utama aplikasi ini. Dan componentactivity digunakan karena menggunakan compose.

Pada baris ke [24] sampai [30] berfungsi untuk menyatakan tema yaitu mydaduthame dan menjadikan dcegame{} menjadi fungsi composable yang menjalankan activity dan menerima lambda untuk menampilkan hasil berupa toast.

Pada baris ke [34] sampai [35] berfungsi untuk penanda bahwa code akan menggunakan compose dan di fungsi ada onresult untuk menampilkan hasil yang akan digunakan oleh toast.

Pada baris ke [37] sampai [38] berfungsi untuk menyimpan angka kedalam dadul atau 2.

Pada baris ke [40] sampai [49] berfungsi untuk menjalankan fungsi rolldice(), Dimana dialamnya mengambil angka acak menggunakan buatangkadadu() dan dimasukkan ke dalam dadu 1 dan dadu 2, kemudian angka didalam dadu tersebut dicek apakah sama atau berbeda, hasil teksnya kemudian di masukkan ke resultmessage dan di return Kambale menggunakan onresult().

Pada baris ke [51] sampai [57] berfungsi untuk Menyusun elemen secara vertical, memenuhi seluruh layer, memberikan jarak tepi dan memusatkan elemen.

Pada baris ke [58] sampai [64] berfungsi untuk Menyusun dadu secara horizontal dan menampilkan dadu menggunakan diceview() berdasarkan angka di dalam dadul dan dadu 2.

Pada baris ke [66] terdapat “Spacer(modifier = Modifier.height(16.dp)), yang berfungsi untuk memberikan jarak vertical dari gambar dadu ke button.

Pada baris ke [68] sampai [74] berfungsi untuk membuat button dan Ketika ditekan akan menjalankan fungsi rooldice() dan diberi bentuk menggunakan shape dan dikasih teks berupa Roll.

Pada baris ke [78] sampai [88] berfungsi untuk membuat gambar menjadi berposisi kolom, kemudian mengambil gambar dari drawable dengan menggunakan dadu tadi dan dimodifikasi bentuk gambarnya, serta ditambah keterangan.

Pada baris ke [90] sampai [93] berfungsi untuk mengembalikan angka secara acak dari rentang 1 sampai 6.

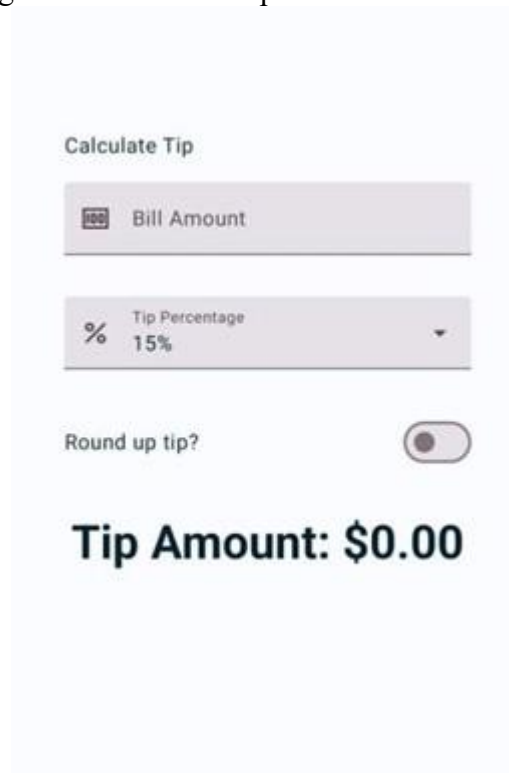
Pada baris ke [95] sampai [105] berfungsi untuk mengecek angka yang diterima sebelumnya dan mengembalikan gambar sesuai dengan angka yang dicek.

MODUL 2 : ANDROID LAYOUT

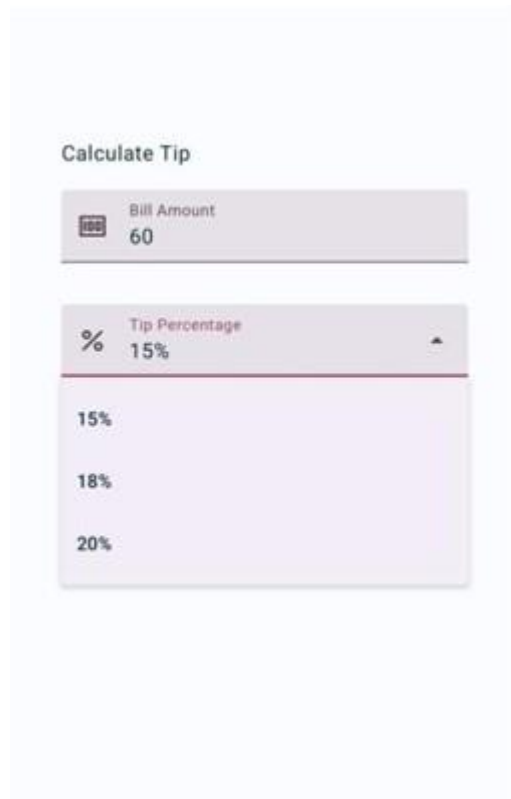
SOAL 1

Soal Praktikum:

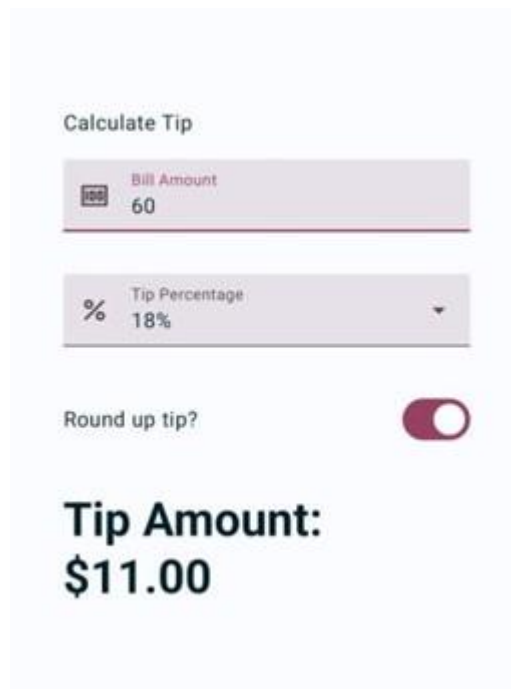
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
 - a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
 - b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
 - c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
 - d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 8. Tampilan awal aplikasi Modul 2



Gambar 9. Tampilan Pilihan Presentasi Tip Modul 2



Gambar 10. Tampilan Aplikasi Setelah Dijalankan Modul 2

A. Source Code

XML :

MainActivity.kt

Table 4. source code soal 1 mainactivity.kt Model 2

```
1 package com.pemrogamanmobile.mycalculatorortipapp
2
3 import android.os.Bundle
4 import android.text.Editable
5 import android.text.TextWatcher
6 import android.widget.*
7 import androidx.appcompat.app.AppCompatActivity
8 import kotlin.math.ceil
9
10 class MainActivity : AppCompatActivity() {
11
12     private lateinit var billAmountInput: EditText
13     private lateinit var tipPercentageDropdown:
14     AutoCompleteTextView
15     private lateinit var roundUpSwitch: Switch
16     private lateinit var tipResult: TextView
17
18     private val tipOptions = listOf("15%", "18%", "20%")
19
20     override fun onCreate(savedInstanceState: Bundle?) {
21         super.onCreate(savedInstanceState)
22         setContentView(R.layout.activity_main)
23
24         billAmountInput = findViewById(R.id.billAmountInput)
25         tipPercentageDropdown =
26         findViewById(R.id.tipPercentageDropdown)
27         roundUpSwitch = findViewById(R.id.roundUpSwitch)
28         tipResult = findViewById(R.id.tipResult)
29
30         val adapter = ArrayAdapter(this,
31         android.R.layout.simple_dropdown_item_1line, tipOptions)
32         tipPercentageDropdown.setAdapter(adapter)
33
34         tipPercentageDropdown.setText(tipOptions[0], false)
35
36         tipPercentageDropdown.setOnClickListener {
37             tipPercentageDropdown.showDropDown()
38         }
39
40         tipPercentageDropdown.setOnItemClickListener { _, _, _,
```

38	calculateTip()
39	}
40	
41	billAmountInput.addTextChangedListener(object : TextWatcher {
42	override fun afterTextChanged(s: Editable?) {
43	calculateTip()
44	}
45	
46	override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
47	override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
48	})
49	
50	roundUpSwitch.setOnCheckedChangeListener { _, _ ->
51	calculateTip()
52	}
53	
54	calculateTip()
55	}
56	
57	private fun calculateTip() {
58	val billAmount =
59	billAmountInput.text.toString().toDoubleOrNull() ?: 0.0
60	val tipPercentText =
61	tipPercentageDropdown.text.toString()
62	val tipPercent =
63	tipPercentText.removeSuffix("%").toDoubleOrNull() ?: 0.0
64	
65	var tip = billAmount * (tipPercent / 100)
66	if (roundUpSwitch.isChecked) {
67	tip = ceil(tip)
68	}
69	tipResult.text = "Tip Amount: \$%.2f".format(tip)
70	}

activity_main.xml

Table 5. source code soal 1 acivity_main.xml Model 2

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="24dp"
8	android:background="#F8FFFB">

```

9
10     <LinearLayout
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         android:gravity="center"
14         android:orientation="vertical">
15
16         <TextView
17             android:id="@+id/title"
18             android:layout_width="wrap_content"
19             android:layout_height="wrap_content"
20             android:layout_gravity="start"
21             android:layout_marginBottom="16dp"
22             android:text="Calculate Tip"
23             android:textColor="#090909"
24             android:textSize="20sp" />
25
26         <LinearLayout
27             android:layout_width="match_parent"
28             android:layout_height="70dp"
29             android:layout_marginBottom="16dp"
30             android:background="@drawable/box_background"
31             android:gravity="center_vertical"
32             android:orientation="horizontal"
33             android:padding="10dp">
34
35             <ImageView
36                 android:layout_width="24dp"
37                 android:layout_height="24dp"
38                 android:layout_gravity="center_vertical"
39                 android:src="@drawable/money" />
40
41
42     <com.google.android.material.textfield.TextInputLayout
43         android:layout_width="0dp"
44         android:layout_height="match_parent"
45         android:layout_marginStart="8dp"
46         android:layout_weight="1"
47         android:hint="Bill Amount"
48         app:boxBackgroundMode="none">
49
50
51     <com.google.android.material.textfield.TextInputEditText
52         android:id="@+id/billAmountInput"
53         android:layout_width="match_parent"
54         android:layout_height="wrap_content"
55
56     android:background="@android:color/transparent"
57     android:inputType="numberDecimal"

```

```

58         android:paddingVertical="8dp" />
59
60     </com.google.android.material.textfield.TextInputLayout>
61     </LinearLayout>
62
63     <LinearLayout
64         android:layout_width="match_parent"
65         android:layout_height="70dp"
66         android:layout_marginBottom="16dp"
67         android:background="@drawable/box_background"
68         android:gravity="center_vertical"
69         android:orientation="horizontal"
70         android:padding="10dp">
71
72         <ImageView
73             android:layout_width="24dp"
74             android:layout_height="24dp"
75             android:layout_gravity="center_vertical"
76             android:src="@drawable/percent" />
77
78
79     <com.google.android.material.textfield.TextInputLayout
80         android:layout_width="0dp"
81         android:layout_height="match_parent"
82         android:layout_marginStart="8dp"
83         android:layout_weight="1"
84         android:hint="Tip Percentage"
85         app:boxBackgroundMode="none">
86
87         <AutoCompleteTextView
88             android:id="@+id/tipPercentageDropdown"
89             android:layout_width="match_parent"
90             android:layout_height="match_parent"
91             android:gravity="center_vertical"
92
93         android:background="@android:color/transparent"
94             android:inputType="none"
95             android:dropDownWidth="match_parent"
96             android:paddingVertical="8dp" />
97
98     </com.google.android.material.textfield.TextInputLayout>
99     </LinearLayout>
100
101     <LinearLayout
102         android:layout_width="match_parent"
103         android:layout_height="50dp"
104         android:layout_marginBottom="24dp"
105         android:orientation="horizontal">
106

```

107	<TextView
108	android:layout_width="wrap_content"
109	android:layout_height="match_parent"
110	android:gravity="center_vertical"
111	android:text="Round up tip?"
112	android:textColor="#3C3A3A"
113	android:textSize="20dp" />
114	
115	<Switch
116	android:id="@+id/roundUpSwitch"
117	android:layout_width="match_parent"
118	android:layout_height="match_parent" />
119	</LinearLayout>
120	
121	<TextView
122	android:id="@+id/tipResult"
123	android:layout_width="wrap_content"
124	android:layout_height="wrap_content"
125	android:text="Tip Amount: \$0.00"
126	android:textColor="#000000"
127	android:textSize="34dp"
128	android:textStyle="bold" />
129	
130	</LinearLayout>
131	</ScrollView>

COMPOSE :

MainActivity.kt

Table 6. source code soal 1 mainactivity.kt(compose) Modul 2

1	package com.pemrogamanmobile.mycalculatorapp
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.Row
9	import androidx.compose.foundation.layout.fillMaxSize
10	import androidx.compose.foundation.layout.fillMaxWidth
11	import androidx.compose.foundation.layout.padding
12	import androidx.compose.foundation.text.KeyboardOptions
13	import androidx.compose.material.icons.Icons
14	import androidx.compose.material.icons.filled.Money
15	import androidx.compose.material.icons.filled.Percent
16	import androidx.compose.material3.DropdownMenu
17	import androidx.compose.material3.DropdownMenuItem
18	import androidx.compose.material3.ExperimentalMaterial3Api
19	import androidx.compose.material3.ExposedDropdownMenuBox


```

20 import androidx.compose.material3.ExposedDropDownMenuDefaults
21 import androidx.compose.material3.Icon
22 import androidx.compose.material3.Text
23 import androidx.compose.material3.TextField
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.getValue
26 import androidx.compose.runtime.mutableStateOf
27 import androidx.compose.runtime.remember
28 import androidx.compose.runtime.setValue
29 import androidx.compose.ui.Alignment
30 import androidx.compose.ui.Modifier
31 import androidx.compose.ui.text.font.FontWeight
32 import androidx.compose.ui.text.input.KeyboardType
33 import androidx.compose.ui.unit.dp
34 import androidx.compose.ui.unit.sp
35
36
37 class MainActivity : ComponentActivity() {
38     override fun onCreate(savedInstanceState: Bundle?) {
39         super.onCreate(savedInstanceState)
40         setContent {
41             Mycalculatortip(
42             )
43         }
44     }
45 }
46
47 @OptIn(ExperimentalMaterial3Api::class)
48 @Composable
49 fun Mycalculatortip() {
50     val options = listOf("15%", "18%", "20%")
51     var expanded by remember { mutableStateOf(false) }
52     var selectedOptionText by remember {
53         mutableStateOf(options[0]) }
54     var billAmount by remember { mutableStateOf("") }
55     var roundUp by remember { mutableStateOf(false) }
56
57     val tipPercentage =
58         selectedOptionText.removeSuffix("%").toDoubleOrNull() ?: 0.0
59     val bill = billAmount.toDoubleOrNull() ?: 0.0
60     var tip = bill * (tipPercentage / 100.0)
61     if (roundUp) {
62         tip = kotlin.math.ceil(tip)
63     }
64
65     androidx.compose.foundation.layout.Box(
66         modifier = Modifier
67             .fillMaxSize()
68             .padding(16.dp)

```

```

67     ) {
68         Column(
69             verticalArrangement = Arrangement.Center,
70             modifier = Modifier.fillMaxSize()
71         ) {
72             Text(
73                 text = "Calculate Tip",
74                 fontSize = 24.sp,
75                 modifier = Modifier
76                     .padding(bottom = 16.dp)
77             )
78             TextField(
79                 value = billAmount,
80                 onValueChange = { billAmount = it },
81                 label = { Text("Bill Amount") },
82                 leadingIcon = {
83                     Icon(Icons.Filled.Money,
contentDescription = "Money Icon")
84                 },
85                 keyboardOptions                                =
86                 KeyboardOptions.Default.copy(keyboardType      =
KeyboardType.Number),
87                 singleLine = true,
88                 modifier = Modifier
89                     .fillMaxWidth()
90                     .padding(bottom = 16.dp),
91                 colors                                          =
92                 ExposedDropdownMenuDefaults.textFieldColors()
93             )
94             ExposedDropdownMenuBox(
95                 expanded = expanded,
96                 onExpandedChange = { expanded = !expanded },
97                 modifier = Modifier
98                     .fillMaxWidth()
99                     .padding(bottom = 16.dp)
100             ) {
101                 TextField(
102                     readOnly = true,
103                     value = selectedOptionText,
104                     onValueChange = {},
105                     label = { Text("Tip Percentage") },
106                     leadingIcon = {
107                         Icon(Icons.Filled.Percent,
contentDescription = "Percent Icon")
108                     },
109                     trailingIcon = {
110
111

```

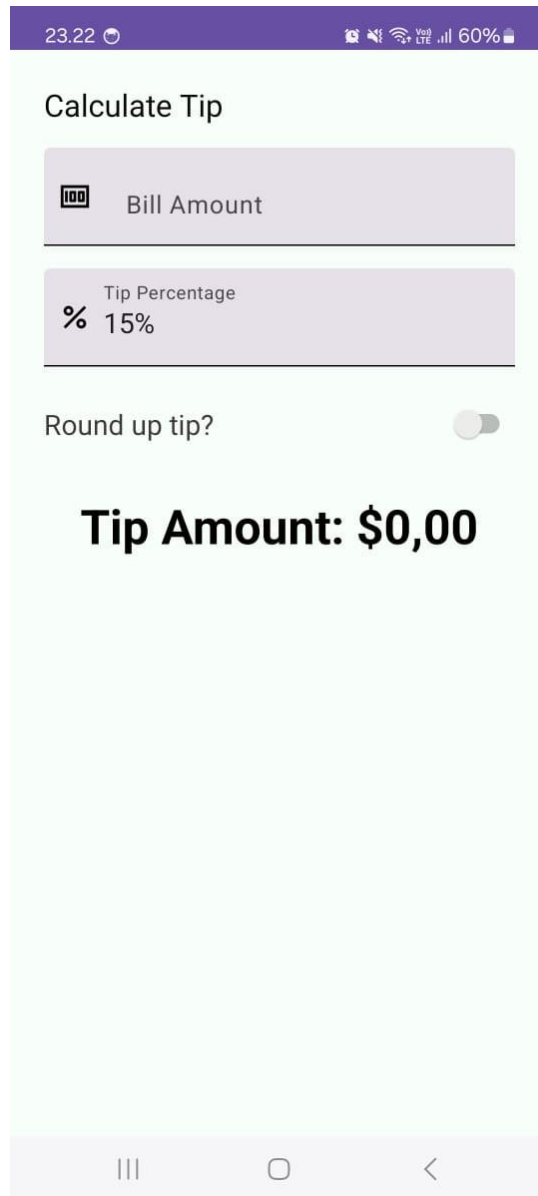
```

112
113 ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded)
114         },
115         modifier = Modifier
116             .menuAnchor()
117             .fillMaxWidth(),
118         colors                                     =
119 ExposedDropdownMenuDefaults.textFieldColors()
120     )
121     DropdownMenu(
122         expanded = expanded,
123         onDismissRequest = { expanded = false },
124         modifier = Modifier.fillMaxWidth()
125     ) {
126         options.forEach { selectionOption ->
127             DropdownMenuItem(
128                 text = { Text(selectionOption) },
129                 onClick = {
130                     selectionOption                =
131                         selectedOptionText
132                     expanded = false
133                 }
134             )
135         }
136     }
137
138     Row(
139         horizontalArrangement                       =
140 Arrangement.SpaceBetween,
141         modifier = Modifier
142             .fillMaxWidth()
143             .padding(vertical = 16.dp)
144     ) {
145         Text(text = "Round up tip?", fontSize = 20.sp)
146         androidx.compose.material3.Switch(
147             checked = roundUp,
148             onCheckedChange = { roundUp = it }
149         )
150     }
151
152     Text(
153         text = "Tip Amount: $${"%.2f".format(tip)}",
154         fontSize = 38.sp,
155         fontWeight = FontWeight.Bold,
156         modifier = Modifier
            .padding(top = 16.dp)

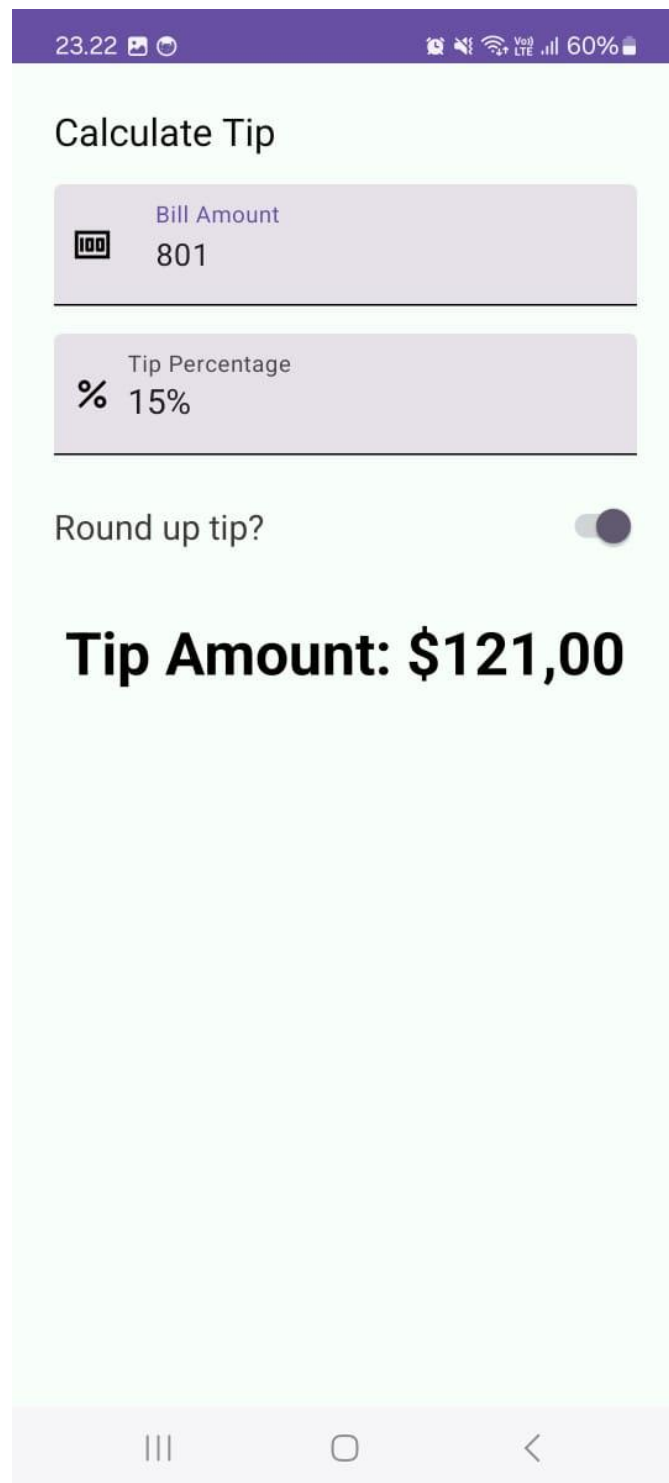
```

157	.align(Alignment.CenterHorizontally)
158	)
159	}
160	}

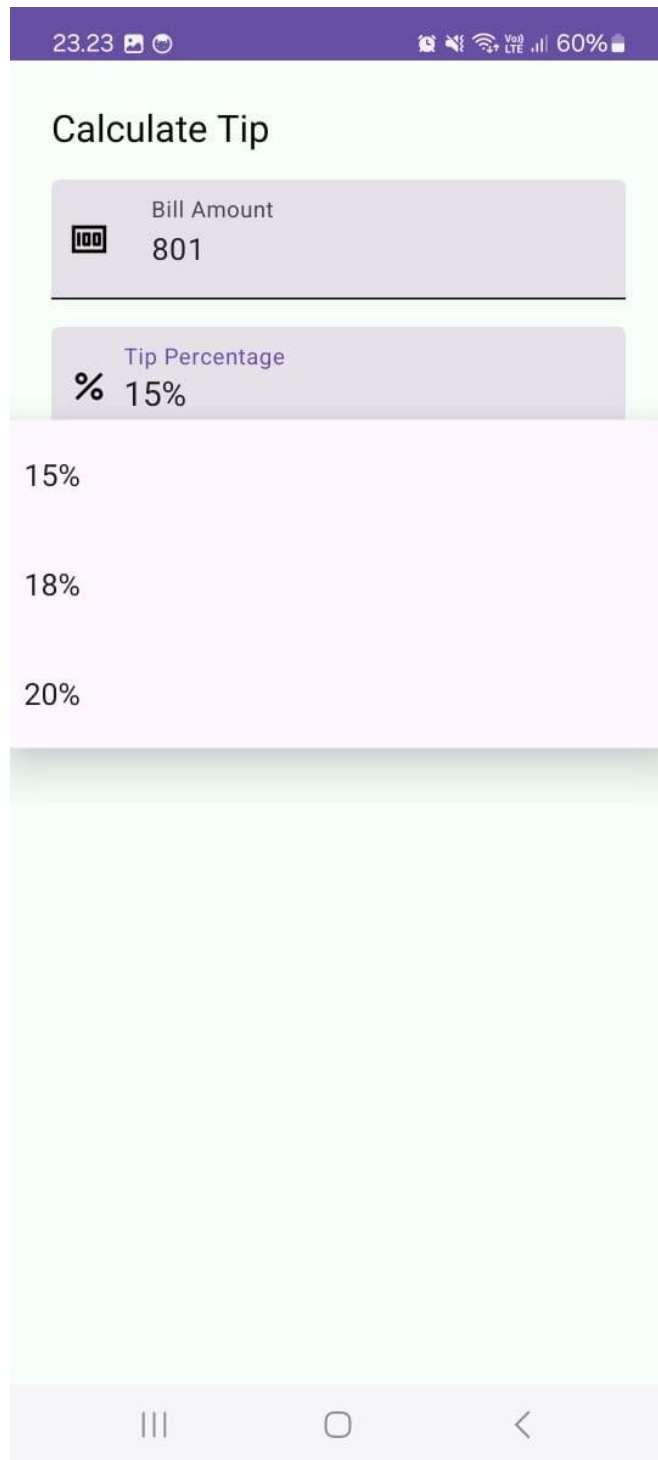
B. Output Program



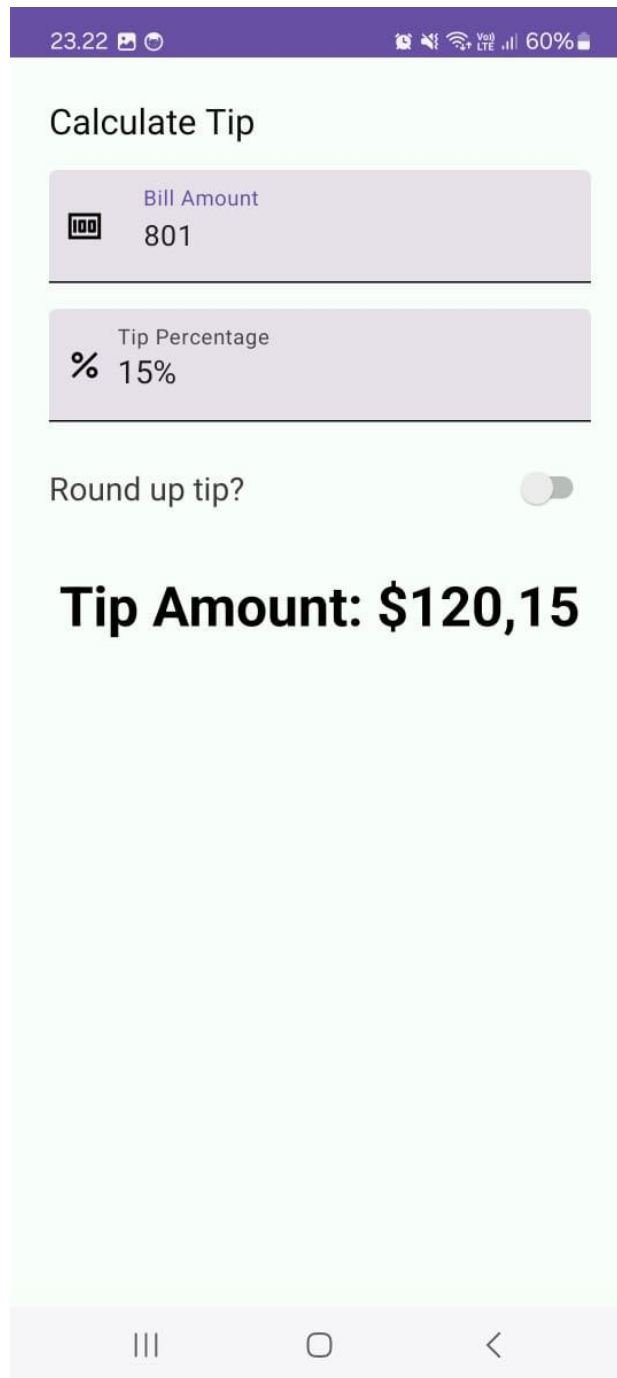
Gambar 11. Screenshot Hasil Jawaban Soal 1 XML bagian a Modul 2



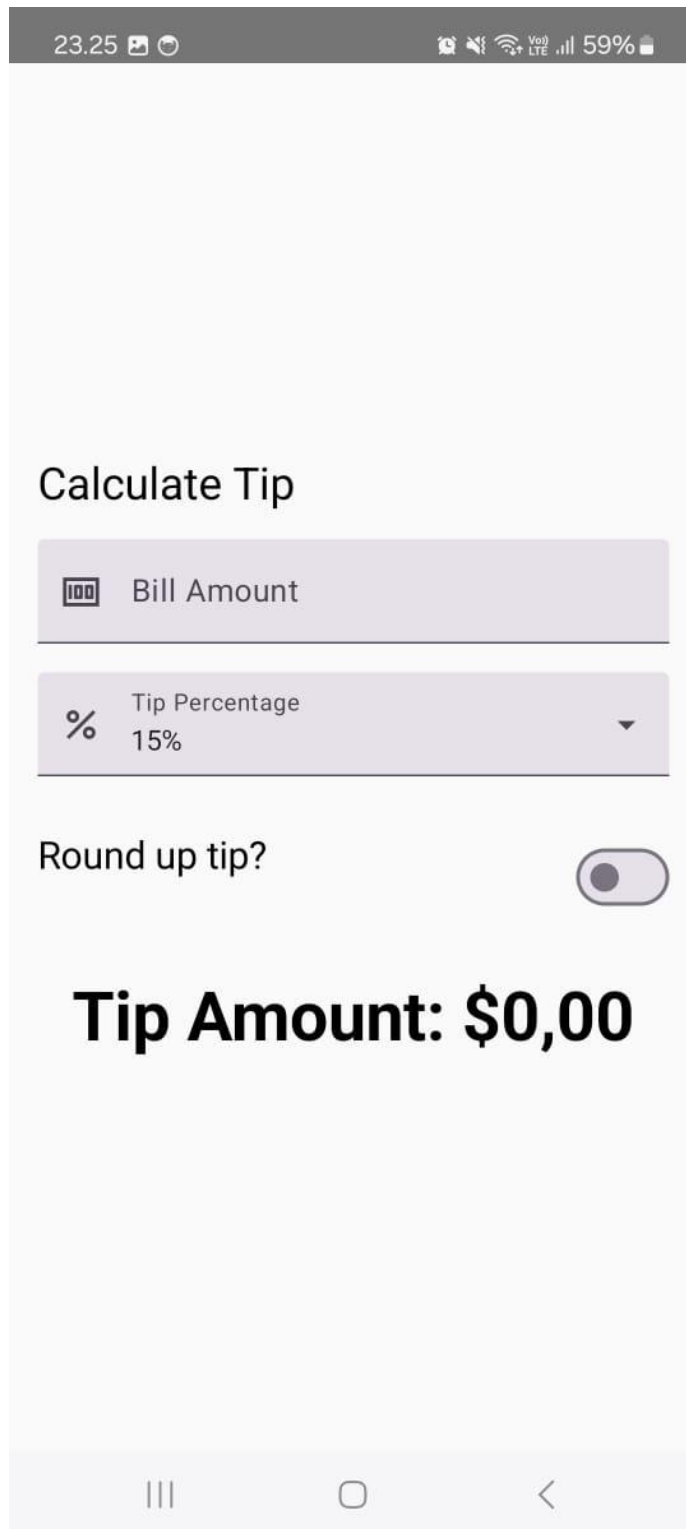
Gambar 12. Screenshot Hasil Jawaban Soal 1 XML bagian b Modul 2



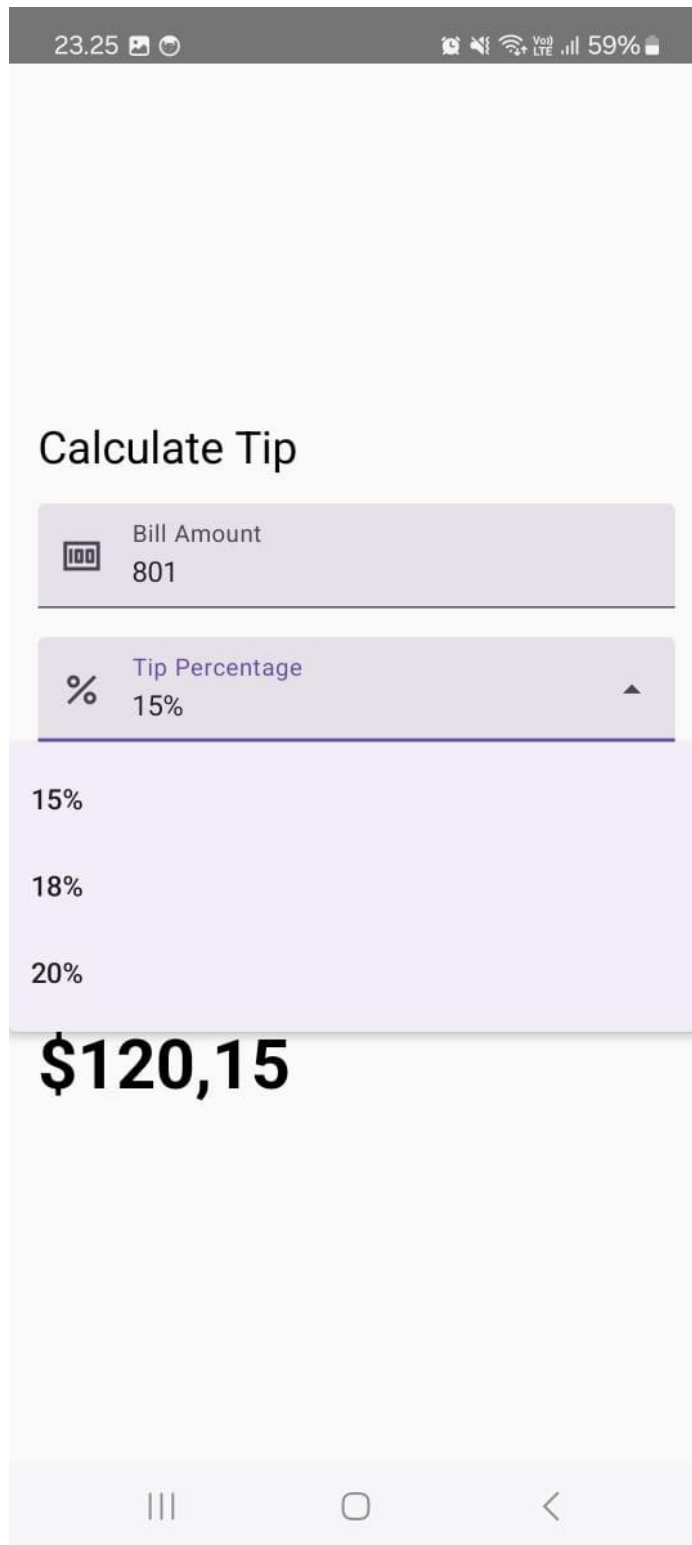
Gambar 13. Screenshot Hasil Jawaban Soal 1 XML bagian c Modul 2



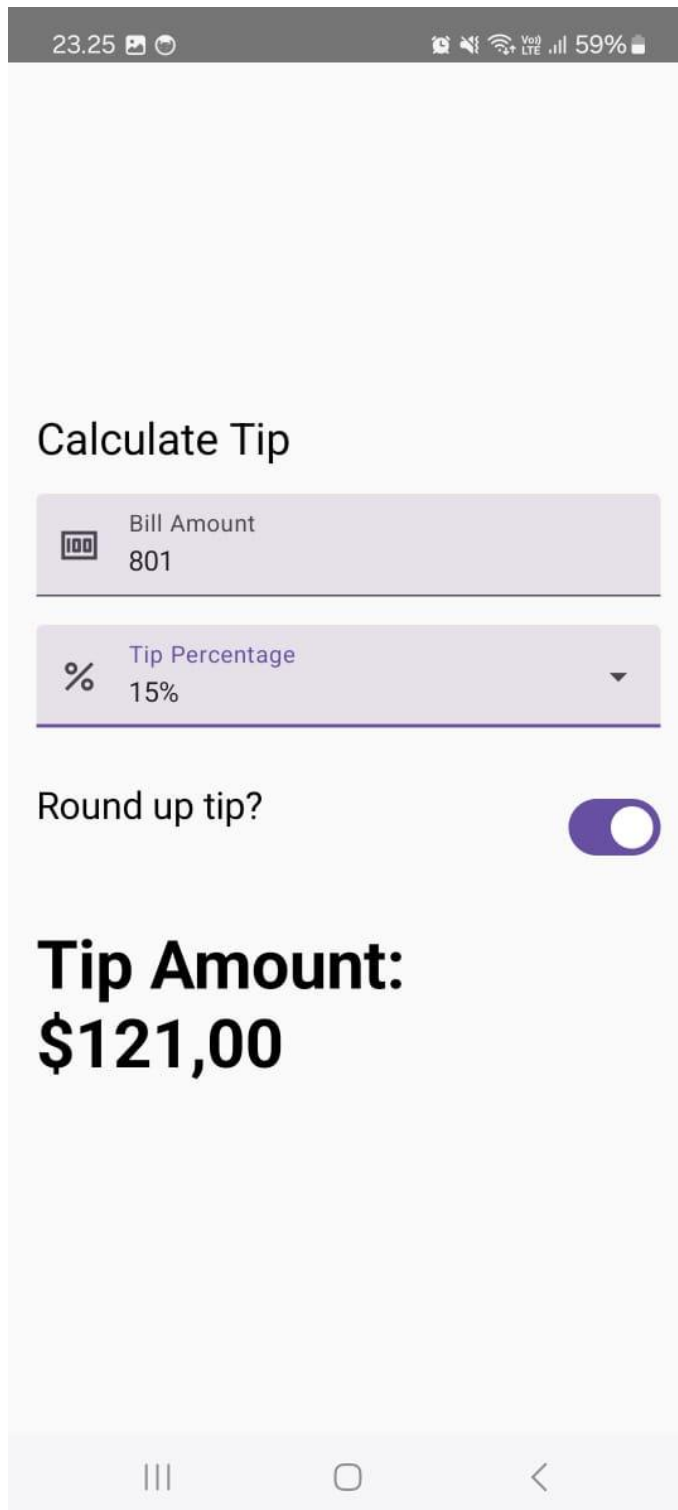
Gambar 14. Screenshot Hasil Jawaban Soal 1 XML bagian d Modul 2



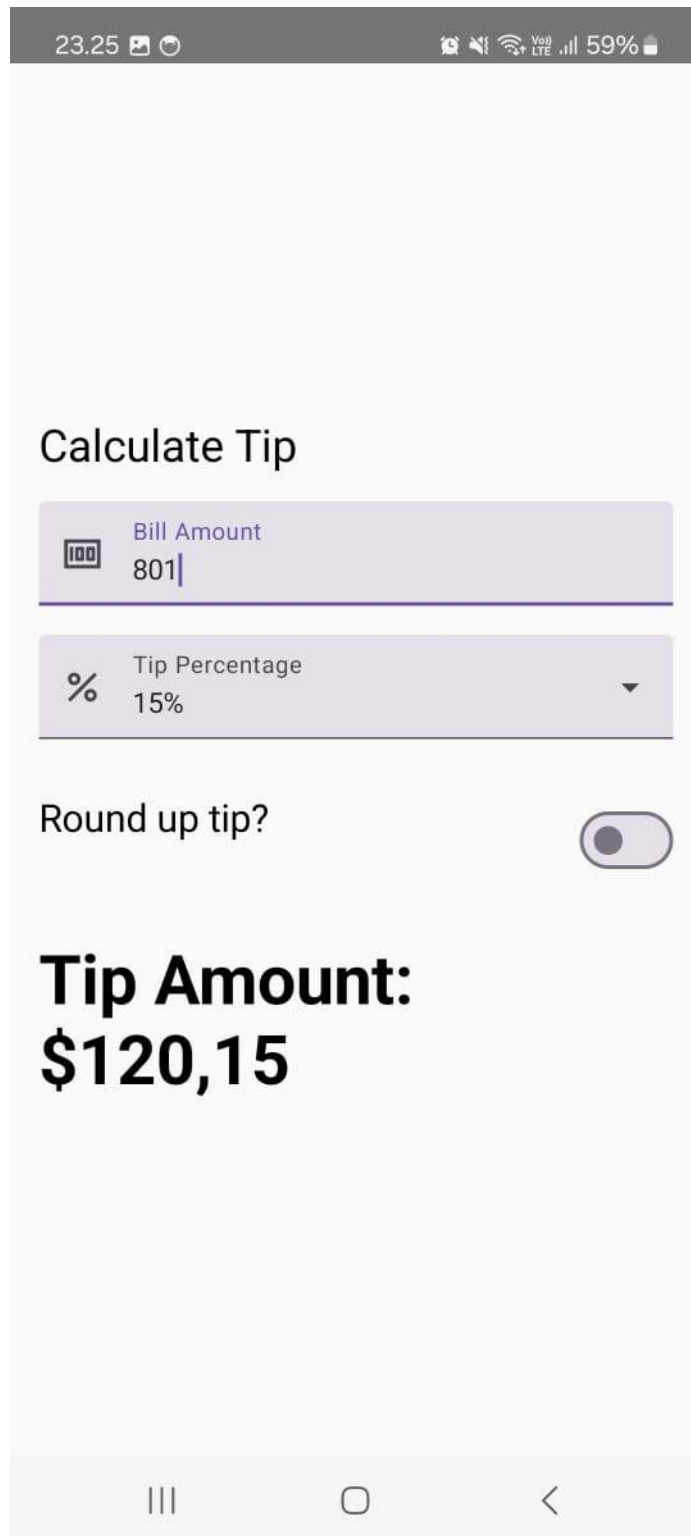
Gambar 15. Screenshot Hasil Jawaban Soal 1 compose bagian a Modul 2



Gambar 16. Screenshot Hasil Jawaban Soal 1 compose bagian b Modul 2



Gambar 17. Screenshot Hasil Jawaban Soal 1 compose bagian c Modul 2



Gambar 18. Screenshot Hasil Jawaban Soal 1 compose bagian d Modul 2

C. Pembahasan

XML :

MainActivity.kt:

Pada baris ke [1] terdapat package `com.pemrogramanmobile.mycalculatoripapp` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file.

Pada baris ke [3] sampai [8], berfungsi untuk mengimport beberapa kelas yang akan digunakan, yaitu `Bundle`, `Editable`, `TextWatcher`, `AutoCompleteTextView`, `EditText`, `Switch`, `TextView`, `ArrayAdapter`, `AppCompatActivity` dan `ceil` dari `kotlin.math`.

Pada baris ke [10] terdapat class `MainActivity : AppCompatActivity()` yang berfungsi untuk membuat class `MainActivity` yang merupakan activity utama aplikasi ini.

Pada baris ke [12] sampai [15] berfungsi untuk membuat variabel yang akan menyimpan inputan pengguna seperti jumlah tagihan, dropdown pilihan tip, switch pembulatan, dan hasil tip. (`lateinit` digunakan untuk menunda inisialisasi sebelum digunakan).

Pada baris ke [17] terdapat `private val tipOptions = listOf("15%", "18%", "20%")` yang berfungsi untuk membuat daftar pilihan persentase tip yang akan ditampilkan di dropdown.

Pada baris ke [19] sampai [21] berfungsi untuk memanggil activity yang pertama kali dibuat, kemudian juga menampilkan layout yang diambil dari `activity_main.xml`.

Pada baris ke [23] sampai [26] berfungsi untuk menghubungkan variabel di kode ini ke komponen yang ada di file xml berdasarkan id-nya.

Pada baris ke [28] berfungsi untuk membuat adapter yang menampilkan daftar persentase tip pada dropdown.

Pada baris ke [29] berfungsi untuk menetapkan adapter tersebut ke dropdown `tipPercentageDropdown`.

Pada baris ke [31] berfungsi untuk mengatur nilai default dari dropdown menjadi "15%" saat aplikasi dibuka pertama kali.

Pada baris ke [33] sampai [35] berfungsi untuk menampilkan pilihan dropdown ketika `tipPercentageDropdown` diklik oleh pengguna.

Pada baris ke [37] sampai [39] berfungsi untuk memanggil fungsi `calculateTip()` saat pengguna memilih salah satu pilihan dari dropdown.

Pada baris ke [41] sampai [48] berfungsi untuk memantau perubahan teks pada input jumlah tagihan, dan setiap kali teks berubah, fungsi `calculateTip()` akan dipanggil.

Pada baris ke [50] sampai [52] berfungsi untuk memantau status switch pembulatan, jika pengguna mengaktifkan atau menonaktifkan switch, maka fungsi `calculateTip()` akan dijalankan.

Pada baris ke [54] terdapat `calculateTip()` yang berfungsi untuk menghitung nilai tip secara langsung ketika aplikasi pertama kali dibuka.

Pada baris ke [58] berfungsi untuk mengambil nilai dari input tagihan dan mengubahnya menjadi angka, jika kosong maka diisi dengan nilai 0.0.

Pada baris ke [59] sampai [60] berfungsi untuk mengambil persentase tip dari dropdown dan menghapus karakter "%" lalu mengubahnya menjadi angka desimal.

Pada baris ke [62] berfungsi untuk menghitung nilai tip berdasarkan jumlah tagihan dan persentase tip.

Pada baris ke [63] sampai [65] berfungsi untuk membulatkan nilai tip ke atas jika switch pembulatan diaktifkan.

Pada baris ke [67] berfungsi untuk menampilkan hasil perhitungan tip ke dalam TextView dengan format dua angka di belakang koma.

activity_main.xml:

Pada baris ke [1] terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan.

Pada baris ke [2] sampai [7] terdapat elemen `ScrollView` yang berfungsi sebagai kontainer gulir vertikal agar isi layout bisa di-scroll jika melebihi layar, dengan lebar dan tinggi disesuaikan ke layar penuh, padding 24dp, dan latar belakang berwarna #F8FFFB.

Pada baris ke [9] sampai [13] terdapat `LinearLayout` sebagai wadah utama yang menampung seluruh komponen, dengan orientasi vertikal dan gravitasi ke tengah.

Pada baris ke [15] sampai [23] terdapat `TextView` dengan id `title` yang menampilkan teks “Calculate Tip” sebagai judul, dengan warna teks #090909, ukuran teks 20sp, dan margin bawah 16dp.

Pada baris ke [25] sampai [56] terdapat `LinearLayout` yang digunakan sebagai wadah input jumlah tagihan, dengan tinggi 70dp, margin bawah 16dp, latar belakang dari `drawable box_background`, orientasi horizontal, dan padding 10dp. Di dalamnya terdapat `ImageView` dengan ikon uang dan `TextInputLayout` yang menampung `TextInputEditText` untuk memasukkan jumlah tagihan dengan `inputType numberDecimal` dan padding vertikal 8dp.

Pada baris ke [58] sampai [91] terdapat `LinearLayout` serupa yang digunakan untuk memilih persentase tip, dengan struktur dan gaya yang hampir sama seperti input jumlah tagihan, namun berisi ikon persen dan `AutoCompleteTextView` untuk menampilkan daftar pilihan tip.

Pada baris ke [93] sampai [111] terdapat `LinearLayout` dengan tinggi 50dp dan margin bawah 24dp, yang menampilkan `TextView` “Round up tip?” di sebelah kiri dan `Switch` dengan id `roundUpSwitch` di sebelah kanan yang digunakan untuk mengaktifkan atau menonaktifkan pembulatan nilai tip.

Pada baris ke [113] sampai [120] terdapat `TextView` dengan id `tipResult` yang menampilkan hasil perhitungan tip awal yaitu “Tip Amount: \$0.00” dengan ukuran teks 34sp, tebal (bold), dan warna teks hitam.

COMPOSE :

MainActivity.kt:

Pada baris ke [1] terdapat package `com.pemrogramanmobile.mycalculatoripapp` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file.

Pada baris ke [3] sampai [34], berfungsi untuk mengimpor beberapa komponen penting dari Android seperti `Bundle`, `layout`, `icon`, `textfield`, dan komponen `Material3` yang digunakan dalam aplikasi ini.

Pada baris ke [37] terdapat class `MainActivity : ComponentActivity()` yang berfungsi untuk membuat class `MainActivity` sebagai activity utama aplikasi ini.

Pada baris ke [38] sampai [44] berfungsi untuk membuat fungsi `onCreate` yang akan dijalankan saat activity dimulai, lalu menampilkan `composable` bernama `Mycalculatorip` sebagai isi tampilan aplikasi.

Pada baris ke [47] terdapat anotasi `@OptIn(ExperimentalMaterial3Api::class)` yang digunakan untuk mengaktifkan fitur eksperimental dari Material3.

Pada baris ke [49] sampai [159] berfungsi untuk membuat fungsi composable bernama `MyCalculatorTip` yang akan menjadi UI utama aplikasi ini.

Pada baris ke [50] terdapat variabel options yang berisi daftar pilihan persentase tip.

Pada baris ke [51] sampai [53] digunakan untuk mendeklarasikan dan mengingat state dari dropdown, nilai pilihan yang dipilih, dan jumlah tagihan.

Pada baris ke [54] terdapat variabel `roundUp` yang menyimpan status switch untuk membulatkan tip atau tidak.

Pada baris ke [56] sampai [61] berfungsi untuk menghitung nilai tip berdasarkan persentase yang dipilih dan jumlah tagihan, lalu jika `roundUp` bernilai true maka tip akan dibulatkan ke atas.

Pada baris ke [63] sampai [67] digunakan untuk menambahkan layout Box sebagai wadah utama dengan padding 16dp.

Pada baris ke [68] sampai [71] berfungsi untuk membuat kolom dengan konten yang ditampilkan secara vertikal dan berada di tengah layar.

Pada baris ke [72] sampai [77] digunakan untuk menampilkan judul aplikasi yaitu "Calculate Tip" dengan ukuran font 24sp.

Pada baris ke [79] sampai [92] berfungsi untuk menampilkan TextField untuk input jumlah tagihan, yang disertai icon uang, hanya menerima input angka, dan tampilannya diatur dengan padding dan lebar penuh.

Pada baris ke [94] sampai [100] digunakan untuk membuat `ExposedDropDownMenuBox` sebagai tampilan pilihan persentase tip.

Pada baris ke [101] sampai [116] berfungsi untuk menampilkan TextField readonly yang menunjukkan pilihan persentase, dengan icon persen, trailing icon dropdown, dan menggunakan warna default.

Pada baris ke [118] sampai [132] digunakan untuk menampilkan daftar `DropDownMenuItem` dari setiap item yang ada di list options, yang akan menutup menu dan menyimpan pilihan saat diklik.

Pada baris ke [136] sampai [147] digunakan untuk menampilkan Row yang berisi teks "Round up tip?" dan switch untuk mengaktifkan pembulatan, dengan jarak antar item disesuaikan.

Pada baris ke [149] sampai [156] digunakan untuk menampilkan hasil kalkulasi tip dalam format dua angka desimal, dengan ukuran font 38sp, tebal, dan berada di tengah secara horizontal.

SOAL 2

Soal Praktikum:

2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan dari implementasi XML dan Jetpack Compose, pada XML code bersifat imperativ Dimana tiap fungsi harus dibuat sendiri supaya membentuk suatu box yang diinginkan, sedangkan untuk compose lebih ke declarative Dimana code sudah banyak tersedia beberapa fungsi yang berguna untuk mengerjakan soal seperti `ExposedDropdownMenuBox`.

Kelebihan

XML : lebih fleksibel dalam membentuk tampilannya.

COMPOSE : lebih cepat dalam pengerjaan codenya.

Kekurangan

XML : lebih memakan waktu dalam pengerjaannya.

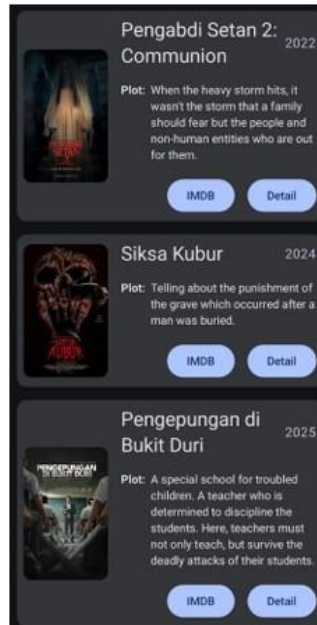
COMPOSE : tidak terlalu fleksible dibandingkan compose.

MODUL 3 : BUILD A SCROLLABLE LIST

SOAL 1

Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
 2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
 3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
 4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
 5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
 6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
 7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
 8. Aplikasi berbasis XML harus menggunakan ViewBinding
- UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 19. Contoh UI List Modul 3

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 20. Contoh UI List Model 3

A. Source Code

XML :

Games.kt

Table 7. source code soal 1 Games.kt (xml) Modul 3

1	package com.pemrogamanmobile.myfavoritegames.data
2	
3	import java.io.Serializable
4	
5	data class Games(6 val detailResourceId: Int, 7 val imageResourceId: Int, 8 val steamLinkResourceId: Int, 9 val yearResourceId: Int, 10 val titleResourceId: Int 11): Serializable

DataSource.kt

Table 8. source code soal 1 DataSource.kt (xml) Model 3

1	package com.pemrogamanmobile.myfavoritegames.data
2	
3	import com.pemrogamanmobile.myfavoritegames.R
4	
5	object DataSource { 6 fun loadGames(): List<Games> = listOf(7 Games(R.string.detail1, R.drawable.image1, R.string.link1, R.string.year1, R.string.title1), 8 Games(R.string.detail2, R.drawable.image2, R.string.link2, R.string.year2, R.string.title2), 9 Games(R.string.detail3, R.drawable.image3, R.string.link3, R.string.year3, R.string.title3), 10 Games(R.string.detail4, R.drawable.image4, R.string.link4, R.string.year4, R.string.title4), 11 Games(R.string.detail5, R.drawable.image5, R.string.link5, R.string.year5, R.string.title5), 12 Games(R.string.detail6, R.drawable.image6, R.string.link6, R.string.year6, R.string.title6), 13 Games(R.string.detail7, R.drawable.image7, R.string.link7, R.string.year7, R.string.title7), 14 Games(R.string.detail8, R.drawable.image8, R.string.link8, R.string.year8, R.string.title8), 15 Games(R.string.detail9, R.drawable.image9, R.string.link9, R.string.year9, R.string.title9),

16	Games(R.string.detail10, R.drawable.image10, R.string.link10,
17	R.string.year10, R.string.title10)
18	)
	}

GamesAdapter.kt

Table 9. source code soal 1 GamesAdapter.kt (xml) Modul 3

1	package com.pemrogamanmobile.myfavoritegames.ui
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.view.LayoutInflater
6	import android.view.ViewGroup
7	import androidx.recyclerview.widget.RecyclerView
8	import com.pemrogamanmobile.myfavoritegames.data.Games
9	import
	com.pemrogamanmobile.myfavoritegames.databinding.ItemGamesBinding
10	
11	class GamesAdapter(
12	private val items: List<Games>,
13	private val onDetailClick: (Games) -> Unit
14	): RecyclerView.Adapter<GamesAdapter.ViewHolder>() {
15	
16	inner class ViewHolder(val binding: ItemGamesBinding)
17	: RecyclerView.ViewHolder(binding.root)
18	
19	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
	ViewHolder {
20	val binding = ItemGamesBinding.inflate(
21	LayoutInflater.from(parent.context), parent, false
22	)
23	return ViewHolder(binding)
24	}
25	
26	override fun onBindViewHolder(holder: ViewHolder, position: Int) {
27	val item = items[position]
28	with(holder.binding) {
29	
30	val layoutParams = root.layoutParams as
	ViewGroup.MarginLayoutParams
31	layoutParams.bottomMargin = 60
32	root.layoutParams = layoutParams
33	
34	imageView.setImageResource(item.imageResourceId)

35	titleText.text =
36	holder.itemView.context.getString(item.titleResourceId)
	yearText.text =
37	holder.itemView.context.getString(item.yearResourceId)
	detailText.text =
38	holder.itemView.context.getString(item.detailResourceId)
39	
40	viewButton.setOnClickListener {
	val url =
41	holder.itemView.context.getString(item.steamLinkResourceId)
42	holder.itemView.context.startActivity(
43	Intent(Intent.ACTION_VIEW, Uri.parse(url))
44	)
45	}
46	detailButton.setOnClickListener {
47	onDetailClick(item)
48	}
49	}
50	}
51	
52	override fun getItemCount(): Int = items.size
4	}

GamesListFragment.kt

Table 10. source code soal 1 GamesListFragment.kt (xml) Modul 3

1	package com.pemrogamanmobile.myfavoritegames.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.findNavController
9	import com.pemrogamanmobile.myfavoritegames.data.DataSource
10	import
	com.pemrogamanmobile.myfavoritegames.databinding.FragmentGamesListBinding
11	
12	class GamesListFragment : Fragment() {
13	private var _binding: FragmentGamesListBinding? = null
14	private val binding get() = _binding!!
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container: ViewGroup?, saved: Bundle?
18	) = FragmentGamesListBinding.inflate(inflater, container, false)

19	.also { _binding = it }
20	.root
21	
22	override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
23	val games = DataSource.loadGames()
24	binding.recyclerView.adapter = GamesAdapter(games) { game ->
25	val action = GamesListFragmentDirections
26	.actionListToDetail(game)
27	findNavController().navigate(action)
28	}
29	}
30	
31	override fun onDestroyView() {
32	super.onDestroyView()
33	_binding = null
34	}
35	}
5	

DetailFragment.kt

Table 11. source code soal 1 DetailFragment.kt (xml) Modul 3

1	package com.pemrogamanmobile.myfavoritegames.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.navArgs
9	import com.pemrogamanmobile.myfavoritegames.databinding.FragmentDetailBinding
10	class DetailFragment : Fragment() {
11	private var _binding: FragmentDetailBinding? = null
12	private val binding get() = _binding!!
13	
14	private val args: DetailFragmentArgs by navArgs()
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?
18	) = FragmentDetailBinding.inflate(inflater, container, false)
19	.also { _binding = it }
20	.root
21	
22	override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
23	val games = args.games
24	with(binding) {
25	detailImageView.setImageResource(games.imageResourceId)
26	detailTitleText.text = getString(games.titleResourceId)
27	detailYearText.text = getString(games.yearResourceId)
28	detailDescText.text = getString(games.detailResourceId)
29	}
30	}
31	
32	override fun onDestroyView() {
33	super.onDestroyView()
34	_binding = null
35	}
36	}
37	}
37	}

MainActivity.kt

Table 12. source code soal 1 mainactivity.kt (xml) Modul 3

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
	com.pemrogamanmobile.myfavoritegames.databinding.ActivityMainBinding
6	
7	class MainActivity : AppCompatActivity() {
8	private lateinit var binding: ActivityMainBinding
9	
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	binding = ActivityMainBinding.inflate(layoutInflater)
13	setContentView(binding.root)
14	}
15	}

activity_main.xml

Table 13. source code soal 1 activity_main.xml (xml) Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_host_fragment"
6	android:name="androidx.navigation.fragment.NavHostFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:navGraph="@navigation/nav_graph"
10	app:defaultNavHost="true"/>

Fragment_games_list.xml

Table 14. source code soal 1 fragment_games_list.xml (xml) Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:background="#000000"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/recyclerView"

11	android:contentDescription="@string/games_list_desc"
12	android:layout_width="0dp"
13	android:layout_height="0dp"
14	android:padding="8dp"
15	app:layoutManager="LinearLayoutManager"
16	app:layout_constraintTop_toTopOf="parent"
17	app:layout_constraintBottom_toBottomOf="parent"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintEnd_toEndOf="parent"/>
20	</androidx.constraintlayout.widget.ConstraintLayout>

Fragment_detail.xml

Table 15. source code soal 1 fragment_detail.xml (xml) Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:background="#131418"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent">
7	
8	<LinearLayout
9	android:orientation="vertical"
10	android:padding="16dp"
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content">
13	
14	<com.google.android.material.imageview.ShapeableImageView
15	android:id="@+id/detailImageView"
16	android:layout_width="match_parent"
17	android:layout_height="400dp"
18	android:scaleType="centerCrop"
19	app:shapeAppearanceOverlay="@style/ShapeAppearance.App.Rounded6dp"
20	/>
21	
22	<LinearLayout
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content"
25	android:orientation="horizontal"
26	android:gravity="center_vertical"
27	android:layout_marginTop="12dp">
28	
29	<TextView
30	android:id="@+id/detailTitleText"

31	android:layout_width="0dp"
32	android:layout_weight="1"
33	android:layout_height="wrap_content"
34	android:textColor="#FFFFFF"
35	android:textSize="26sp"
36	android:textStyle="bold"/>
37	
38	<TextView
39	android:id="@+id/detailYearText"
40	android:layout_width="wrap_content"
41	android:layout_height="wrap_content"
42	android:textColor="#7C7C86"
43	android:textSize="14sp"/>
44	</LinearLayout>
45	
46	<TextView
47	android:layout_marginTop="8dp"
48	android:layout_width="wrap_content"
49	android:layout_height="wrap_content"
50	android:text="tentang game ini:"
51	android:textColor="#FFFFFF"
52	android:textSize="14sp"
53	android:textStyle="bold"/>
54	
55	<TextView
56	android:id="@+id/detailDescText"
57	android:layout_marginTop="4dp"
58	android:layout_width="match_parent"
59	android:layout_height="wrap_content"
60	android:textColor="#FFFFFF"
61	android:textSize="14sp"/>
62	</LinearLayout>
	</ScrollView>

Item_games.kt

Table 16. source code soal 1 item_games.xml (xml) Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	app:cardCornerRadius="6dp"
8	app:cardBackgroundColor="#343539">
9	

10	<LinearLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	android:orientation="horizontal"
14	android:padding="8dp"
15	android:gravity="center_vertical">
16	
17	<com.google.android.material.imageview.ShapeableImageView
18	android:id="@+id/imageView"
19	android:layout_width="100dp"
20	android:layout_height="140dp"
21	android:layout_marginEnd="12dp"
22	android:scaleType="centerCrop"
23	
24	app:shapeAppearanceOverlay="@style/ShapeAppearance.App.Rounded6dp"
25	/>
26	<LinearLayout
27	android:layout_width="0dp"
28	android:layout_weight="1"
29	android:orientation="vertical"
30	android:layout_height="wrap_content">
31	
32	<LinearLayout
33	android:layout_width="match_parent"
34	android:layout_height="wrap_content"
35	android:orientation="horizontal"
36	android:gravity="center_vertical">
37	
38	<TextView
39	android:id="@+id/titleText"
40	android:layout_width="0dp"
41	android:layout_weight="1"
42	android:layout_height="wrap_content"
43	android:textColor="@android:color/white"
44	android:textSize="20sp"
45	android:textStyle="bold"/>
46	
47	<TextView
48	android:id="@+id/yearText"
49	android:layout_width="wrap_content"
50	android:layout_height="wrap_content"
51	android:textColor="#7C7C86"
52	android:textSize="14sp"/>

53	</LinearLayout>	
54		
55	<TextView	
56	android:layout_marginTop="8dp"	
57	android:layout_width="wrap_content"	
58	android:layout_height="wrap_content"	
59	android:text="tentang game ini:"	
60	android:textColor="@android:color/white"	
61	android:textSize="14sp"	
62	android:textStyle="bold"/>	
63		
64	<TextView	
65	android:id="@+id/detailText"	
66	android:layout_marginTop="4dp"	
67	android:layout_width="wrap_content"	
68	android:layout_height="wrap_content"	
69	android:textColor="@android:color/white"	
70	android:textSize="14sp"/>	
71		
72	<LinearLayout	
73	android:layout_width="match_parent"	
74	android:layout_height="wrap_content"	
75	android:orientation="horizontal"	
76	android:gravity="end"	
77	android:layout_marginTop="12dp">	
78		
79	<Button	
80	android:id="@+id/viewButton"	
81	android:layout_width="100dp"	
82	android:layout_height="44dp"	
83	android:text="View"	
84	android:backgroundTint="#b0c4ff"	
85	android:textColor="#000"/>	
86		
87	<Space	android:layout_width="8dp"
88	android:layout_height="wrap_content"/>	
89	<Button	
90	android:id="@+id/detailButton"	
91	android:layout_width="100dp"	
92	android:layout_height="44dp"	
93	android:text="Detail"	
94	android:backgroundTint="#b0c4ff"	
95	android:textColor="#000"/>	

96	</LinearLayout>
97	</LinearLayout>
98	</LinearLayout>
99	</androidx.cardview.widget.CardView>

strings.xml

Table 17. source code soal 1 strings.xml (xml) Modul 3

1	<resources>
2	<string name="app_name">My favorite games</string>
3	
4	<string name="games_list_desc">List of favorite games</string>
5	
6	<string name="title1">Terraria</string>
7	<string name="title2">Mobile Legends</string>
8	<string name="title3">Firewatch</string>
9	<string name="title4">The Elder Scrolls : Skyrim</string>
10	<string name="title5">Monster Hunter World</string>
11	<string name="title6">Genshin Impact</string>
12	<string name="title7">Delta Force</string>
13	<string name="title8">Magic Chess Go Go</string>
14	<string name="title9">STAR WARS Jedi: Fallen Order</string>
15	<string name="title10">Minecraft</string>
16	
17	<string name="year1">2011</string>
18	<string name="year2">2016</string>
19	<string name="year3">2016</string>
20	<string name="year4">2016</string>
21	<string name="year5">2018</string>
22	<string name="year6">2020</string>
23	<string name="year7">2025</string>
24	<string name="year8">2025</string>
25	<string name="year9">2019</string>
26	<string name="year10">2011</string>
27	
28	<string name="detail1">Dig, fight, explore, build! Nothing is impossible in this action-packed Pack also available!</string>
29	<string name="detail2">Bergabung dengan teman Anda di Mobile Legends: Bang Bang, Game MOBA 5v5 terbaru, dan bertarung melawan Player sungguhan! Pilih Hero favorit Anda dan bentuk tim yang sempurna dengan teman Anda! Matchmaking 10 detik, pertandingan 10 menit. Laning, Jungling, Team Fight, dan menghancurkan Turret, semua kesenangan dari Game aksi dan MOBA PC berada dalam genggamannya Anda! Tunjukkan semangat Esports Anda!</string>
30	<string name="detail3">Firewatch is a single-player first-person mystery set in the Wyoming wilderness, where your only emotional lifeline is the person on the other end of a hand.

31	<string name="detail4">Winner of more than 200 Game of the Year Awards, The Elder Scrolls V: Skyrim Special Edition brings the epic fantasy to life in stunning detail. The Special Edition includes the critically acclaimed add-ons with all-new features.</string>
32	<string name="detail5">Welcome to a new world! In Monster Hunter: World, the latest installment in the series, you can enjoy the ultimate hunting experience, using everything at your disposal to hunt monsters in a vast world with surprises and excitement.</string>
33	<string name="detail6">Genshin Impact adalah game RPG Open World terbaru. Kamu adalah seorang penjelajah sebuah dunia fantasi yang teramat luas, kamu dapat menjelajahi tujuh bangsa, bertempur dengan kemampuan dan berkepribadian unik, melawan musuh-musuh kuat, dan berkelana mencari harta karun. Di dunia yang luas ini, biarkan rasa keingintahuan membawamu menguak misteri-misteri yang tersembunyi. Akhirnya kamu dapat bertemu kembali dengan saudaramu, dan menyaksikan akhir dari kisah perjalananmu.</string>
34	<string name="detail7">Pertempuran Epic Warfare 24vs24, Rasakan map besar, puluhan server, dan gameplay permainan menarik, dan update yang terus menerus! Warfare membawa kalian ke medan perang yang penuh kekacauan dan kehancuran environment. Baik kalian menembaki musuh di darat, laut, dan udara atau sebagai rekan satu tim sebagai medic, pilih role kalian dan raih kemenangan!</string>
35	<string name="detail8">Magic Chess: Go Go - Game strategi multiplayer baru yang terinspirasi dari Go dan Bang Bang. Dengan gameplay seperti catur, game ini kasual dan mudah dimainkan kapan saja, di mana saja, dengan teman-teman! Di sini, kemenangan bergantung pada strategi dan sedikit keberuntungan dan kontrol yang ketat. Di setiap babak, kamu bisa mengendalikan Commander untuk membeli dan menempatkan unit, menyusun Sinergi, menggunakan equipment, dan menempatkan hero dengan cerdas untuk mengalahkan 7 player lain secara bertahap untuk memenangkan pertandingan.</string>
36	<string name="detail9">A galaxy-spanning adventure awaits in Star Wars Jedi: Fallen Order, an action-adventure title from Respawn. An abandoned Padawan must complete his training, develop new powers, and master the art of the lightsaber - all while staying one step ahead of the Empire.</string>
37	<string name="detail10">Bangun, jelajahi, bertahan hidup! Main dengan teman dan buat dunia baru di dunia terbuka dengan membangun, membuat item, dan bertahan hidup di game sandbox membangun. Dapatkan sumber daya, coba bertahan hidup di malam hari, dan rancang petualangan epik balok demi balok. Lakukan apa saja sesuai selera di dunia yang terbuka sepenuhnya tempat kamu bisa bermain dengan teman, memelihara hewan, membuka peternakan, menambang jauh ke bawah tanah, menghadapi musuh misterius, atau sekadar berpetualangan sejauh imajinasi membawamu!</string>
38	<string name="link1"> https://store.steampowered.com/app/105600/Terraria/ </string>
39	<string name="link2"> https://play.google.com/store/apps/details?id=com.mobile.legends </string>
40	<string name="link3"> https://store.steampowered.com/app/383870/Firewatch/ </string>
41	<string name="link4"> https://store.steampowered.com/app/489830/The_Elder_Scrolls_V_Skyrim_Special_Edition/ </string>
42	<string name="link5"> https://store.steampowered.com/app/582010/Monster_Hunter_World/ </string>
43	<string name="link6"> https://play.google.com/store/apps/details?id=com.miHoYo.GenshinImpact </string>
44	<string name="link7"> https://play.google.com/store/apps/details?id=com.garena.game.df </string>
45	<string name="link8"> https://play.google.com/store/apps/details?id=com.mobilechess.gp </string>
46	<string name="link9"> https://store.steampowered.com/app/1172380/STAR_WARS_Jedi_Fallen_Order/ </string>
47	<string name="link10"> https://play.google.com/store/apps/details?id=com.mojang.minecraftpe </string>
48	</resources>
49	

styles.xml

Table 18. source code soal 1 style.xml (xml) Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	<style name="ShapeAppearance.App.Rounded6dp" parent="">
4	<item name="cornerFamily">rounded</item>
5	<item name="cornerSize">6dp</item>
6	</style>
7	</resources>

COMPOSE :

MainActivity.kt

Table 19. source code soal 1 mainactivity.kt (compose) Modul 3

1	package com.pemrogamanmobile.myfavoritgames
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.material3.Surface
7	import androidx.compose.ui.graphics.Color
8	import androidx.navigation.NavType
9	import androidx.navigation.compose.NavHost
10	import androidx.navigation.compose.composable
11	import androidx.navigation.compose.rememberNavController
12	import androidx.navigation.navArgument
13	import
	com.pemrogamanmobile.myfavoritgames.ui.theme.MyFavorite
	GamesTheme
14	
15	class MainActivity : ComponentActivity() {
16	override fun onCreate(savedInstanceState: Bundle?) {
17	super.onCreate(savedInstanceState)
18	setContent {
19	MyFavoriteGamesTheme {
20	Surface(color = Color(0xFF131418)) {
21	val navController =
	rememberNavController()
22	
23	NavHost(navController =
	navController, startDestination = "home") {
24	composable("home") {
25	

	HomeScreen(onDetailClick = {
26	game ->
	navController.navigate(
27	"detail/\${game.titleResourceId}/\${game.imageResourceId}/
28	\${game.yearResourceId}/\${game.detailResourceId}"
29	)
30	})
31	}
32	
33	composable(
	route
	"detail/{titleResId}/{imageResId}/{yearResId}/{detailRes
34	Id}",
	arguments = listOf(
35	navArgument("titleResId") { type = NavType.IntType },
36	navArgument("imageResId") { type = NavType.IntType },
37	navArgument("yearResId")
	{ type = NavType.IntType },
38	navArgument("detailResId") { type = NavType.IntType }
39	)
40	) { backStackEntry ->
41	val titleResId
	backStackEntry.arguments?.getInt("titleResId") ?: 0
42	val imageResId
	backStackEntry.arguments?.getInt("imageResId") ?: 0
43	val yearResId
	backStackEntry.arguments?.getInt("yearResId") ?: 0
44	val detailResId
	backStackEntry.arguments?.getInt("detailResId") ?: 0
45	
46	DetailScreen(
47	titleResId = titleResId,
48	imageResId = imageResId,
49	yearResId = yearResId,
50	detailResId
	detailResId
51	)
52	}
53	}
54	}
55	}

56	}
57	}
58	}

HomeScreen.kt

Table 20. source code soal 1 HomeScreen.kt (compose) Modul 3

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import androidx.compose.foundation.lazy.LazyColumn
4	import androidx.compose.foundation.lazy.items
5	import androidx.compose.runtime.Composable
6	import androidx.compose.ui.Modifier
7	
8	import
	com.pemrogamanmobile.myfavoritegames.data.Datasource
9	import
10	com.pemrogamanmobile.myfavoritegames.model.Games
11	
12	@Composable
13	fun HomeScreen(
14	modifier: Modifier = Modifier,
15	onDetailClick: (Games) -> Unit
16	) {
17	val games = Datasource().loadGames()
18	
19	LazyColumn(modifier = modifier) {
20	items(games) { game ->
	GameCard(games = game, onDetailClick =
21	onDetailClick)
22	}
23	}
	}

DetailScreen.kt

Table 21. source code soal 1 DetailScreen.kt (compose) Modul 3

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.rememberScrollState


```

6  import
   androidx.compose.foundation.shape.RoundedCornerShape
7  import androidx.compose.foundation.verticalScroll
8  import androidx.compose.material3.*
9  import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.draw.clip
13 import androidx.compose.ui.graphics.Color
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.res.stringResource
16 import androidx.compose.ui.unit.dp
17 import androidx.compose.ui.layout.ContentScale
18 import androidx.compose.ui.text.TextStyle
19 import androidx.compose.ui.text.font.FontWeight
20 import androidx.compose.ui.unit.sp
21
22 @Composable
23 fun DetailScreen(titleResId: Int, imageResId: Int,
   yearResId: Int, detailResId: Int) {
24     Surface(
25         modifier = Modifier.fillMaxSize(),
26         color = Color(0xFF131418)
27     ) {
28
29         val scrollState = rememberScrollState()
30
31         Column(
32             modifier = Modifier
33                 .verticalScroll(scrollState)
34                 .padding(16.dp)
35         ) {
36
37             Image(
38                 painter
39                 painterResource(imageResId),
40                 contentDescription
41                 stringResource(detailResId),
42                 modifier = Modifier
43                     .fillMaxWidth()
44                     .height(400.dp)
45                     .clip(RoundedCornerShape(6.dp)),
46                 contentScale = ContentScale.Crop
47             )

```

48	Column(modifier	=
49	Modifier.fillMaxWidth()) {	
50	Row(	
	modifier	=
51	Modifier.fillMaxWidth(),	
52	horizontalArrangement	=
53	Arrangement.SpaceBetween	
	) {	
54	Text(	
	text	=
	stringResource(titleResId),	
55	style = TextStyle(fontSize =	=
56	26.sp, fontWeight = FontWeight.Bold, color =	=
57	Color.White),	
58	modifier	=
	Modifier.weight(1f)	
59	)	
	Text(	
60	text	=
	stringResource(yearResId),	
61	style = TextStyle(fontSize =	=
62	16.sp, color = Color(0xFF7C7C86)),	
63	modifier	=
64	Modifier.align(Alignment.CenterVertically)	
65	)	
66	}	
67		
68	Spacer(modifier	=
	Modifier.height(8.dp))	
69		
70	Text(	
71	text = "tentang game ini:",	
72	style = TextStyle(fontSize =	=
73	14.sp, fontWeight = FontWeight.Bold, color =	=
	Color.White)	
74	)	
75	Text(	
76	text	=
77	stringResource(detailResId),	
78	style = TextStyle(fontSize =	=
79	14.sp, color = Color.White),	
80	modifier = Modifier.padding(top =	
81	4.dp)	
82	)	

83	}
84	}
85	}
86	}

GameCard.kt

Table 22. source code soal 1 GameCard.kt (compose) Modul 3

1	package com.pemrogamanmobile.myfavoritgames
2	
3	import android.content.Intent
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.*
6	import
	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.material3.*
8	import androidx.compose.runtime.Composable
9	import androidx.compose.ui.Alignment
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.draw.clip
12	import androidx.compose.ui.graphics.Color
13	import androidx.compose.ui.platform.LocalContext
14	import androidx.compose.ui.res.painterResource
15	import androidx.compose.ui.res.stringResource
16	import androidx.compose.ui.text.TextStyle
17	import androidx.compose.ui.text.font.FontWeight
18	import androidx.compose.ui.unit.dp
19	import androidx.compose.ui.unit.sp
20	import androidx.core.net.toUri
21	import
22	com.pemrogamanmobile.myfavoritgames.model.Games
23	import androidx.compose.ui.layout.ContentScale
24	
25	@Composable
26	fun GameCard(
27	games: Games,
28	modifier: Modifier = Modifier,
29	onDetailClick: (Games) -> Unit
30	) {
31	val context = LocalContext.current
32	
33	Card(
34	modifier = modifier.padding(12.dp),

35	colors	=
36	CardDefaults.cardColors(containerColor	=
37	Color(0xFF343539))	
38	) {	
39	Row(modifier = Modifier.fillMaxWidth()) {	
40	Image(	
	painter	=
41	painterResource(games.imageResourceId),	
42	contentDescription	=
43	stringResource(games.titleResourceId),	
44	modifier = Modifier	
45	.padding(start = 12.dp)	
46	.width(100.dp)	
47	.height(140.dp)	
48	.clip(RoundedCornerShape(6.dp))	
49		
50	.align(Alignment.CenterVertically),	
51	contentScale = ContentScale.Crop	
52	)	
53		
54	Column(	
	modifier = Modifier	
55	.padding(16.dp)	
56	.fillMaxWidth(),	
57	verticalArrangement	=
58	Arrangement.SpaceBetween	
	) {	
59	Row(	
60	modifier	=
61	Modifier.fillMaxWidth(),	
62	horizontalArrangement	=
	Arrangement.SpaceBetween,	
63	verticalAlignment	=
	Alignment.Top	
	) {	
64	Text(	
65	text	=
66	context.getString(games.titleResourceId),	
67	style = TextStyle(fontSize =	=
	26.sp, fontWeight = FontWeight.Bold, color =	=
68	Color.White),	
69	modifier	=
70	Modifier.weight(1f)	
	)	

71	Spacer(modifier	=
72	Modifier.width(8.dp))	
73	Text(	
	text	=
74	context.getString(games.yearResourceId),	
75	style = TextStyle(fontSize =	
76	16.sp, color = Color(0xFF7C7C86)),	
77	modifier = Modifier	
78	.wrapContentWidth(Alignment.End)	
79	.align(Alignment.CenterVertically)	
80	)	
81	}	
82		
83	Spacer(modifier	=
	Modifier.height(8.dp))	
84	Column(modifier	=
85	Modifier.fillMaxWidth()) {	
86	Text(	
	text = "tentang game ini:",	
87	style = TextStyle(fontSize =	
	14.sp, fontWeight = FontWeight.Bold, color =	
88	Color.White)	
	)	
89	Text(	
90	text	=
91	context.getString(games.detailResourceId),	
92	style = TextStyle(fontSize =	
	14.sp, color = Color.White),	
93	modifier	=
94	Modifier.padding(top = 4.dp)	
95	)	
96	}	
97	Spacer(modifier	=
	Modifier.height(12.dp))	
98		
99	Row(	
100	modifier	=
101	Modifier.fillMaxWidth(),	

102	horizontalArrangement	=
	Arrangement.End,	
	verticalAlignment	=
103	Alignment.CenterVertically	
104	) {	
105	Button(	
	onClick = {	
106	val url	=
	context.getString(games.steamLinkIdResourceId)	
	val intent	=
107	Intent(Intent.ACTION_VIEW, url.toUri())	
108	context.startActivity(intent)	
109	},	
110	modifier	=
111	Modifier.width(100.dp).height(44.dp),	
	colors	=
112	ButtonDefaults.buttonColors(containerColor	=
113	Color(0xFFb0c4ff), contentColor = Color.White)	
114	) {	
	Text("View", style	=
115	TextStyle(fontSize = 16.sp, color = Color.Black))	
	}	
116	Spacer(modifier	=
	Modifier.width(8.dp))	
117	Button(	
118	onClick = {	
119	onDetailClick(games) },	
120	modifier	=
121	Modifier.width(100.dp).height(44.dp),	
122	colors	=
123	ButtonDefaults.buttonColors(containerColor	=
124	Color(0xFFb0c4ff), contentColor = Color.White)	
	) {	
	Text("Detail", style	=
	TextStyle(fontSize = 16.sp, color = Color.Black))	
	}	
	}	
	}	
	}	
	}	

Datasource.kt

Table 23. source code soal 1 Datasource.kt (compose) Modul 3

1	package com.pemrogamanmobile.myfavoritgames.data
2	
3	import com.pemrogamanmobile.myfavoritgames.R
4	import
5	com.pemrogamanmobile.myfavoritgames.model.Games
6	
7	class Datasource {
8	fun loadGames(): List<Games> {
9	return listOf<Games>(Games(R.string.detail1, 10 R.drawable.image1, R.string.link1, R.string.year1, R.string.title1), 11 Games(R.string.detail2, R.drawable.image2, R.string.link2, R.string.year2, 12 R.string.title2), Games(R.string.detail3, 13 R.drawable.image3, R.string.link3, R.string.year3, R.string.title3), 14 Games(R.string.detail4, R.drawable.image4, R.string.link4, R.string.year4, 15 R.string.title4), Games(R.string.detail5, 16 R.drawable.image5, R.string.link5, R.string.year5, R.string.title5), 17 Games(R.string.detail6, R.drawable.image6, R.string.link6, R.string.year6, 18 R.string.title6), Games(R.string.detail7, 19 R.drawable.image7, R.string.link7, R.string.year7, R.string.title7), 20 Games(R.string.detail8, R.drawable.image8, R.string.link8, R.string.year8, R.string.title8), 21 Games(R.string.detail9, R.drawable.image9, R.string.link9, R.string.year9, R.string.title9), 22 Games(R.string.detail10, R.drawable.image10, R.string.link10, R.string.year10, R.string.title10))

23	}
24	}

strings.xml

Table 24. source code soal 1 strings.xml (compose) Modul 3

1	<resources>
2	<string name="app_name">my favorite games</string>
3	
4	<string name="title1">Terraria</string>
5	<string name="title2">Mobile Legends</string>
6	<string name="title3">Firewatch</string>
7	<string name="title4">The Elder Scrolls : Skyrim</string>
8	<string name="title5">Monster Hunter World</string>
	<string name="title6">Genshin Impact</string>
9	<string name="title7">Delta Force</string>
10	<string name="title8">Magic Chess Go Go</string>
11	<string name="title9">STAR WARS Jedi: Fallen Order</string>
12	<string name="title10">Minecraft</string>
13	
14	<string name="year1">2011</string>
15	<string name="year2">2016</string>
16	<string name="year3">2016</string>
17	<string name="year4">2016</string>
18	<string name="year5">2018</string>
19	<string name="year6">2020</string>
20	<string name="year7">2025</string>
21	<string name="year8">2025</string>
22	<string name="year9">2019</string>
23	<string name="year10">2011</string>
24	
25	<string name="detail1">Dig, fight, explore, build! Nothing is impossible in this action-packed adventure game. Four Pack also available!</string>
26	<string name="detail2">Bergabung dengan teman Anda di Mobile Legends: Bang Bang, Game MOBA 5v5 terbaru, dan bertarung melawan Player sungguhan! Pilih Hero favorit Anda dan bentuk tim yang sempurna dengan teman Anda! Matchmaking 10 detik, pertandingan 10 menit. Laning, Jungling, Team Fight, dan menghancurkan Turret, semua kesenangan dari Game aksi dan MOBA PC berada dalam genggaman Anda! Tunjukkan semangat Esports Anda!</string>
27	

28	<string name="detail3">Firewatch is a single-player first-person mystery set in the Wyoming wilderness, where your only emotional lifeline is the person on the other end of a handheld radio.</string>
29	<string name="detail4">Winner of more than 200 Game of the Year Awards, The Elder Scrolls V: Skyrim Special Edition brings the epic fantasy to life in stunning detail. The Special Edition includes the critically acclaimed game and add-ons with all-new features.</string>
30	<string name="detail5">Welcome to a new world! In Monster Hunter: World, the latest installment in the series, you can enjoy the ultimate hunting experience, using everything at your disposal to hunt monsters in a new world teeming with surprises and excitement.</string>
31	<string name="detail6">Genshin Impact adalah game RPG Open World terbaru. Kamu adalah seorang pengembara yang menjelajah sebuah dunia fantasi yang teramat luas, kamu dapat menjelajahi tujuh bangsa, bertemu berbagai karakter berkemampuan dan berkepribadian unik, melawan musuh-musuh kuat, dan berkelana mencari saudaramu yang hilang. Di dunia yang luas ini, biarkan rasa keingintahuan membawamu menguak misteri-misteri yang tersembunyi, sampai akhirnya kamu dapat bertemu kembali dengan saudaramu, dan menyaksikan akhir dari kisah perjalananmu.</string>
32	<string name="detail7">Pertempuran Epic Warfare 24vs24, Rasakan map besar, puluhan senjata, berbagai mode permainan menarik, dan update yang terus menerus! Warfare membawa kalian ke medan perang dinamis yang penuh kekacauan dan kehancuran environment. Baik kalian menembaki musuh di darat, laut, dan udara, atau menyelamatkan rekan satu tim sebagai medic, pilih role kalian dan raih kemenangan!</string>
33	<string name="detail8">Magic Chess: Go Go - Game strategi multiplayer baru yang terinspirasi dari Mobile Legends: Bang Bang. Dengan gameplay seperti catur, game ini kasual dan mudah dimainkan kapan saja, di mana saja bersama teman-teman! Di sini, kemenangan bergantung pada strategi dan sedikit keberuntungan daripada keterampilan pengendalian mikro. Di setiap babak, kamu bisa mengendalikan Commander untuk membeli dan meningkatkan Hero, menyusun Sinergi, menggunakan equipment, dan menempatkan hero dengan cerdas untuk mengalahkan lawan. Kalahkan 7 player lain secara bertahap untuk memenangkan pertandingan.</string>

34	<pre> <string name="detail9">A galaxy-spanning adventure awaits in Star Wars Jedi: Fallen Order, a 3rd person action-adventure title from Respawn. An abandoned Padawan must complete his training, develop new powerful Force abilities, and master the art of the lightsaber - all while staying one step ahead of the Empire.</string> <string name="detail10">Bangun, jelajahi, bertahan hidup! Main dengan teman dan buat duniamu sendiri! Jelajahi dunia terbuka dengan membangun, membuat item, dan bertahan hidup di game sandbox membangun terbaik. Kumpulkan sumber daya, coba bertahan hidup di malam hari, dan rancang petualangan epik balok demi balok. Jelajahi dan buat item sesuka hati di dunia yang terbuka sepenuhnya tempat kamu bisa bermain dengan teman, membangun kota balok, membuka peternakan, menambang jauh ke bawah tanah, menghadapi musuh misterius, atau sekadar bereksperimen sejauh imajinasi membawamu!</string> <string name="link1">https://store.steampowered.com/app/105600/ Terraria/</string> <string name="link2">https://play.google.com/store/apps/details ?id=com.mobile.legends</string> <string name="link3">https://store.steampowered.com/app/383870/ Firewatch/</string> <string name="link4">https://store.steampowered.com/app/489830/ The_Elder_Scrolls_V_Skyrim_Special_Edition/</string> <string name="link5">https://store.steampowered.com/app/582010/ Monster_Hunter_World/</string> <string name="link6">https://play.google.com/store/apps/details ?id=com.miHoYo.GenshinImpact</string> <string name="link7">https://play.google.com/store/apps/details ?id=com.garena.game.df</string> <string name="link8">https://play.google.com/store/apps/details ?id=com.mobilechess.gp</string> <string name="link9">https://store.steampowered.com/app/1172380 /STAR_WARS_Jedi_Fallen_Order/</string> </pre>
----	--

43	<pre> <string name="link10">https://play.google.com/store/apps/detail s?id=com.mojang.minecraftpe</string> </resources> </pre>
----	--

Games.kt

Table 25. source code soal 1 Games.kt (compose) Modul 3

1	package com.pemrogamanmobile.myfavoritegames.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Games(
7	@StringRes val detailResourceId: Int,
8	@DrawableRes val imageResourceId: Int,
9	@StringRes val steamLinkResourceId: Int,
10	@StringRes val yearResourceId: Int,
11	@StringRes val titleResourceId: Int,
12	)

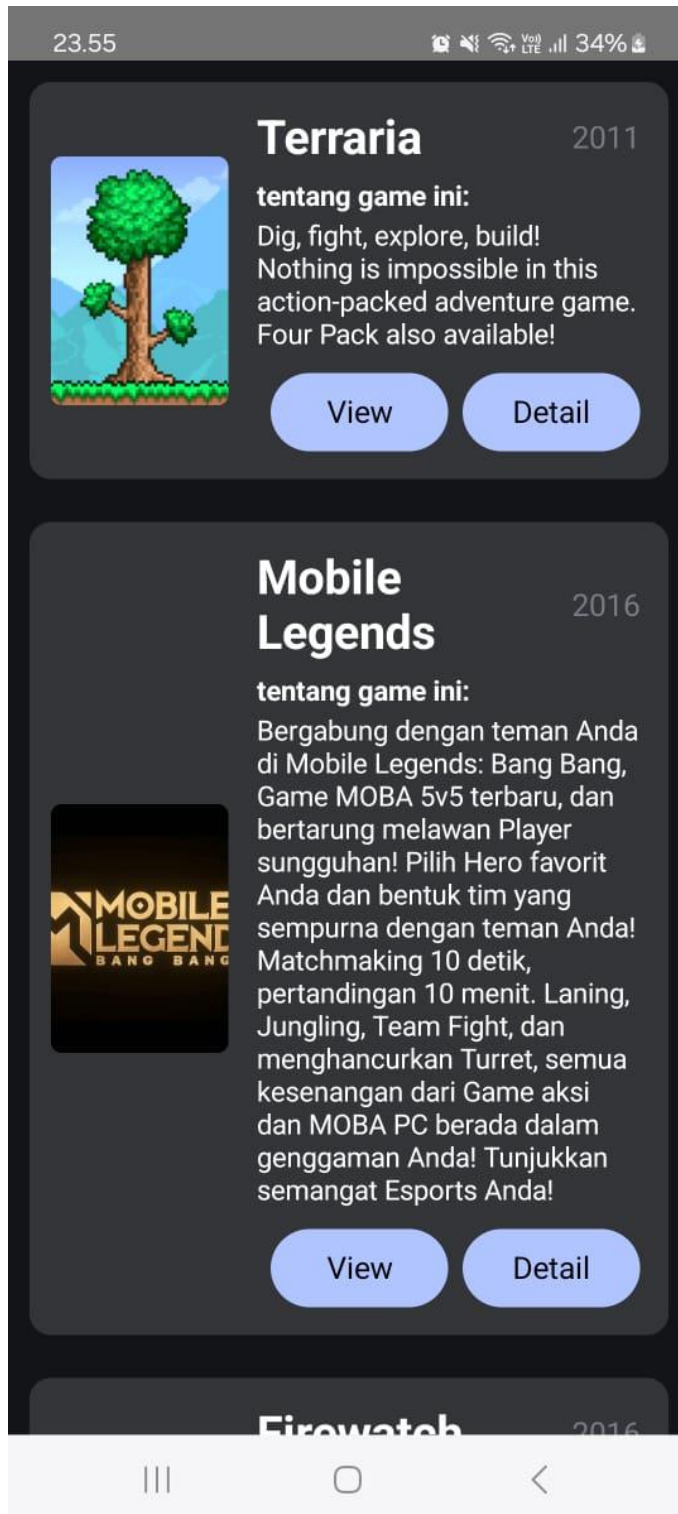
B. Output Program



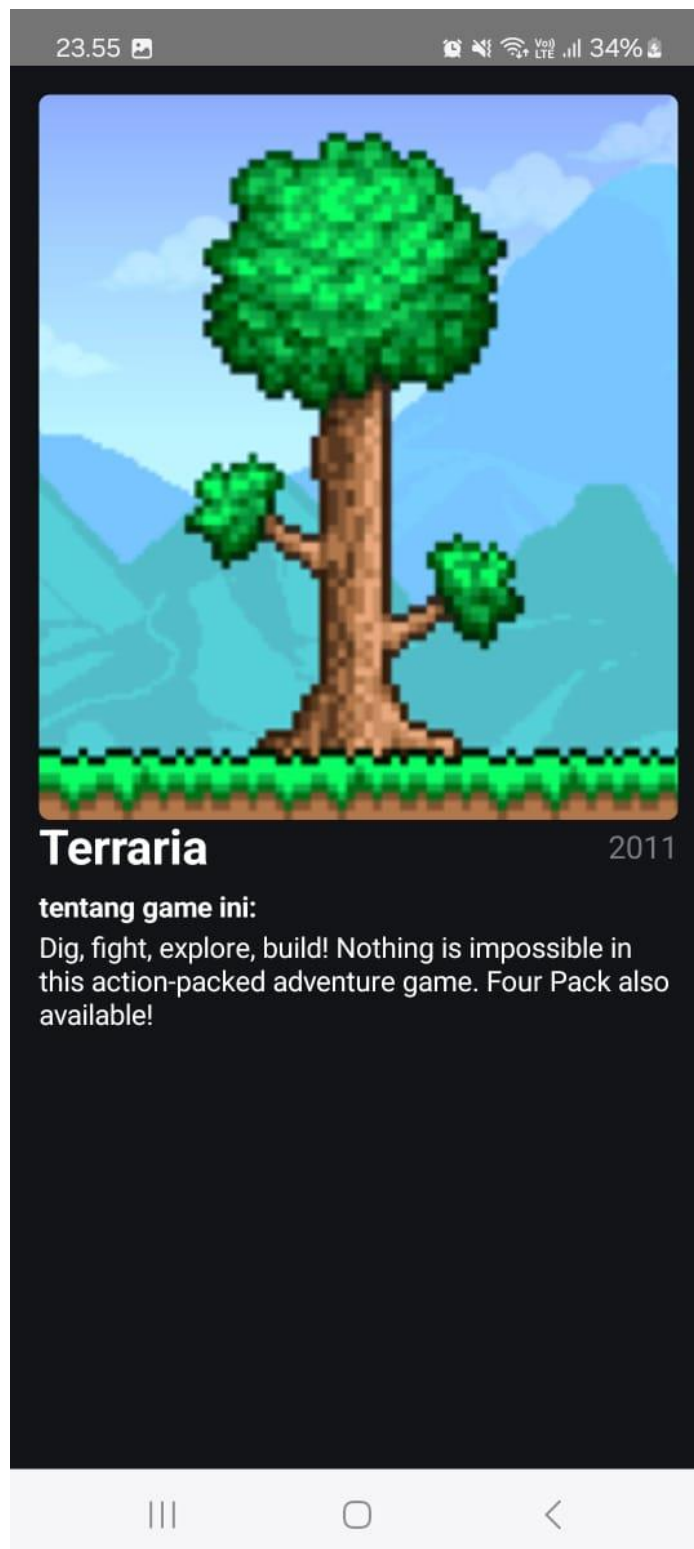
Gambar 21. Screenshot Hasil Jawaban Soal 1 XML bagian a Modul 3



Gambar 22. Screenshot Hasil Jawaban Soal 1 XML bagian b Modul 3



Gambar 23. Screenshot Hasil Jawaban Soal 1 compose bagian a Modul 3



Gambar 24. Screenshot Hasil Jawaban Soal 1 compose bagian b Modul 3

C. Pembahasan

XML :

Games.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.data` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file ini agar terorganisir sesuai struktur proyek Android.

Pada baris ke [3] terdapat import `java.io.Serializable` yang berfungsi untuk mengimport kelas `Serializable`, agar objek dari kelas ini bisa dikirim antar activity melalui intent.

Pada baris ke [5] terdapat data class `Games` yang berfungsi untuk membuat kelas data bernama `Games`, yang secara otomatis menyediakan fungsi seperti `toString()`, `equals()`, `hashCode()`, dan `copy()`.

Pada baris ke [6] sampai [10] berfungsi untuk mendeklarasikan properti dalam kelas `Games`, yaitu `detailResourceId`, `imageResourceId`, `steamLinkResourceId`, `yearResourceId`, dan `titleResourceId` yang masing-masing bertipe `Int` dan berfungsi sebagai ID dari resource yang menyimpan informasi detail teks, gambar game, tautan Steam, tahun rilis, serta judul game yang akan digunakan pada aplikasi.

Pada baris ke [11] terdapat `) : Serializable` yang berfungsi untuk menjadikan kelas `Games` sebagai objek yang dapat diserialisasi agar bisa dikirim melalui Intent.

Datasource.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.data` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file ini agar terorganisir sesuai struktur proyek Android.

Pada baris ke [3] terdapat import `com.pemrogamanmobile.myfavoritegames.R` yang berfungsi untuk mengakses resource seperti string dan drawable yang terdapat di dalam file res aplikasi Android ini.

Pada baris ke [5] terdapat object `DataSource` yang berfungsi untuk mendefinisikan object singleton bernama `DataSource`, artinya hanya ada satu instance dari objek ini yang akan digunakan sepanjang aplikasi berjalan.

Pada baris ke [6] terdapat fun `loadGames(): List<Games> = listOf()` yang berfungsi untuk mendeklarasikan fungsi bernama `loadGames` yang mengembalikan daftar dari objek `Games` dalam bentuk `List`.

Pada baris ke [7] sampai [16] berfungsi untuk mengisi list tersebut dengan sepuluh objek `Games` yang masing-masing berisi lima parameter, yaitu detail, image, link ke Steam, tahun rilis, dan judul game. Kelima parameter tersebut merujuk pada ID resource yang disimpan dalam file `strings.xml` dan drawable di folder `res`. Setiap baris merepresentasikan satu game favorit yang akan ditampilkan di aplikasi.

Pada baris ke [17] terdapat tanda `)` yang berfungsi untuk menutup fungsi `listOf` yang telah dibuat sebelumnya.

GamesAdapter.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.ui` yang berfungsi sebagai nama paket dari file ini dan menunjukkan bahwa file ini berada dalam folder `ui` (user interface).

Pada baris ke [3] sampai [9], berfungsi untuk mengimpor beberapa kelas yang digunakan, yaitu Intent dan Uri untuk membuka tautan eksternal, LayoutInflater dan ViewGroup untuk membuat tampilan, RecyclerView untuk membuat daftar scrollable, Games sebagai data model, dan ItemGamesBinding untuk menghubungkan data dengan layout XML menggunakan View Binding.

Pada baris ke [11] terdapat class GamesAdapter(yang berfungsi untuk mendeklarasikan kelas GamesAdapter sebagai adapter untuk RecyclerView.

Pada baris ke [12] sampai [13] berfungsi untuk mendefinisikan dua parameter konstruktor utama adapter, yaitu items sebagai list dari objek Games, dan onDetailClick sebagai lambda function yang dijalankan ketika tombol detail ditekan.

Pada baris ke [14] terdapat) : RecyclerView.Adapter<GamesAdapter.ViewHolder>() yang berfungsi untuk menyatakan bahwa kelas ini merupakan subclass dari RecyclerView.Adapter dan akan menggunakan inner class ViewHolder untuk menampung tampilan item.

Pada baris ke [16] terdapat inner class ViewHolder(val binding: ItemGamesBinding) yang berfungsi untuk mendefinisikan inner class ViewHolder yang menyimpan objek binding terhadap layout XML item.

Pada baris ke [17] terdapat : RecyclerView.ViewHolder(binding.root) yang berfungsi untuk menghubungkan root view dari layout item dengan superclass ViewHolder.

Pada baris ke [19] terdapat fungsi onCreateViewHolder yang berfungsi untuk membuat ViewHolder baru setiap kali RecyclerView membutuhkan item baru untuk ditampilkan.

Pada baris ke [20] sampai [21] digunakan LayoutInflater.from(parent.context) untuk mengakses context dan ItemGamesBinding.inflate untuk mengikat tampilan.

Pada baris ke [23] mengembalikan ViewHolder dengan binding tersebut.

Pada baris ke [26] terdapat fungsi onBindViewHolder yang berfungsi untuk menampilkan data ke dalam setiap item yang ada di RecyclerView.

Pada baris ke [27] terdapat val item = items[position] yang berfungsi untuk mengambil data Games berdasarkan posisi item.

Pada baris ke [28] terdapat with(holder.binding) untuk mempermudah akses komponen dalam layout XML.

Pada baris ke [29] sampai [32] berfungsi untuk mengatur isi komponen seperti imageView, titleText, yearText, dan detailText dengan data dari objek Games menggunakan getString() untuk mengambil string dari ID resource.

Pada baris ke [34] sampai [39] berfungsi untuk mengatur tombol viewButton agar saat diklik akan membuka link Steam yang sesuai dengan data item menggunakan Intent dengan action ACTION_VIEW dan Uri.parse().

Pada baris ke [40] sampai [42] berfungsi untuk mengatur tombol detailButton agar saat diklik akan menjalankan fungsi onDetailClick yang sebelumnya dikirim dari parameter konstruktor adapter.

Pada baris ke [46] terdapat override fun getItemCount(): Int = items.size yang berfungsi untuk mengembalikan jumlah total item dalam list yang akan ditampilkan oleh RecyclerView.

GamesListFragment.kt

Pada baris ke [1] terdapat package `com.pemrogramanmobile.myfavoritegames.ui` yang berfungsi sebagai nama paket dari file ini dan menunjukkan bahwa file ini berada dalam folder `ui` (user interface).

Pada baris ke [3] sampai [10], berfungsi untuk mengimport beberapa kelas yang akan digunakan, yaitu `Bundle`, `LayoutInflater`, `View`, `ViewGroup`, `Fragment`, `findNavController`, `DataSource`, dan `FragmentGamesListBinding`. Import ini mendukung pengelolaan tampilan fragment, navigasi antar fragment, pengambilan data dari `DataSource`, dan binding layout XML dengan `View`.

Pada baris ke [12] terdapat class `GamesListFragment : Fragment()` yang berfungsi untuk mendeklarasikan kelas `GamesListFragment` sebagai subclass dari `Fragment`, artinya kelas ini merepresentasikan salah satu tampilan layar (screen) dalam aplikasi.

Pada baris ke [13] terdapat `private var _binding: FragmentGamesListBinding? = null` yang berfungsi untuk mendeklarasikan variabel `_binding` yang menyimpan objek binding dari layout fragment ini, namun masih bernilai `null` untuk mencegah memory leak.

Pada baris ke [14] terdapat `private val binding get() = _binding!!` yang berfungsi untuk membuat properti akses terhadap `_binding` dengan jaminan bahwa nilainya tidak `null`, menggunakan not-null assertion (!!).

Pada baris ke [16] sampai [19] terdapat fungsi `onCreateView` yang berfungsi untuk meng-inflate layout `fragment_Games_list.xml` menggunakan `FragmentGamesListBinding`, dan mengatur nilai `_binding`.

Pada baris ke [20] berfungsi untuk mengembalikan `root` sebagai tampilan utama dari fragment ini.

Pada baris ke [22] terdapat fungsi `onViewCreated` yang berfungsi untuk menangani logika setelah tampilan fragment berhasil dibuat.

Pada baris ke [23] terdapat `val games = DataSource.loadGames()` yang berfungsi untuk mengambil daftar data `Games` dari `DataSource`.

Pada baris ke [24] berfungsi untuk mengatur adapter dari `recyclerView` di layout fragment ini dengan objek `GamesAdapter`, dan juga menetapkan lambda function yang akan dipanggil saat tombol detail ditekan.

Pada baris ke [25] sampai [26] terdapat `val action = GamesListFragmentDirections.actionListToDetail(Games)` yang berfungsi untuk membuat objek `action` berdasarkan `safe args` yang membawa data `Games` ke fragment detail.

Pada baris ke [27] terdapat `findNavController().navigate(action)` yang berfungsi untuk menavigasi pengguna ke fragment detail menggunakan `NavController` dan `action` yang telah dibuat.

Pada baris ke [31] sampai [34] terdapat fungsi `onDestroyView` yang berfungsi untuk menghapus referensi binding saat tampilan fragment dihancurkan, guna mencegah memory leak dengan mengatur `_binding = null`.

DetailFragment.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.ui` yang berfungsi sebagai nama paket dari file ini dan menunjukkan bahwa file ini berada di dalam folder `ui`.

Pada baris ke [3] sampai [9], berfungsi untuk mengimport beberapa kelas yang digunakan, yaitu `Bundle`, `LayoutInflater`, `View`, `ViewGroup`, `Fragment`, `navArgs`, dan `FragmentDetailBinding`. Import ini mendukung pembuatan tampilan fragment, navigasi menggunakan `Safe Args`, dan binding layout XML ke dalam kode.

Pada baris ke [11] terdapat class `DetailFragment : Fragment()` yang berfungsi untuk mendeklarasikan kelas `DetailFragment` sebagai subclass dari `Fragment`, yang merupakan bagian dari tampilan aplikasi.

Pada baris ke [12] terdapat `private var _binding: FragmentDetailBinding? = null` yang berfungsi untuk mendeklarasikan variabel `_binding` sebagai objek binding yang akan menghubungkan kode dengan layout fragment. Nilai awalnya adalah `null` untuk menghindari memory leak.

Pada baris ke [13] terdapat `private val binding get() = _binding!!` yang berfungsi untuk membuat properti binding yang pasti tidak `null` saat digunakan, menggunakan `not-null assertion (!!)`.

Pada baris ke [15] terdapat `private val args: DetailFragmentArgs by navArgs()` yang berfungsi untuk mengambil data dari argument navigasi menggunakan `Safe Args`, yaitu data `Games` yang dikirim dari fragment sebelumnya.

Pada baris ke [17] sampai [21] terdapat fungsi `onCreateView` yang berfungsi untuk meng-inflate layout `fragment_detail.xml` menggunakan `View Binding` dan mengembalikan tampilan utama dari fragment.

Pada baris ke [23] terdapat fungsi `onViewCreated` yang berfungsi untuk mengatur tampilan fragment setelah layout selesai dibuat.

Pada baris ke [24] terdapat `val Games = args.Games` yang berfungsi untuk mengambil objek `Games` dari argumen `Safe Args` yang dikirim melalui navigasi.

Pada baris ke [25] sampai [30] berfungsi untuk mengatur konten layout berdasarkan data dari objek `Games`, seperti gambar, judul, tahun rilis, dan deskripsi dengan menggunakan `getString()` berdasarkan ID resource masing-masing.

Pada baris ke [33] sampai [36] terdapat fungsi `onDestroyView` yang berfungsi untuk menghapus referensi binding saat tampilan fragment dihancurkan, untuk mencegah memory leak dengan mengatur `_binding = null`.

MainActivity.kt:

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames` yang berfungsi sebagai nama paket utama dari aplikasi ini dan menunjukkan bahwa file ini berada di bagian utama dari struktur folder aplikasi.

Pada baris ke [3] sampai [5], berfungsi untuk mengimpor beberapa kelas yang akan digunakan, yaitu `Bundle` dari `Android` untuk menyimpan data saat pembuatan activity, `AppCompatActivity` sebagai superclass activity, dan `ActivityMainBinding` untuk menghubungkan layout XML activity ke dalam kode melalui `View Binding`.

Pada baris ke [7] terdapat class `MainActivity : AppCompatActivity()` yang berfungsi untuk mendeklarasikan `MainActivity` sebagai activity utama aplikasi dan mewarisi perilaku dari `AppCompatActivity`.

Pada baris ke [8] terdapat `private lateinit var binding: ActivityMainBinding` yang berfungsi untuk mendeklarasikan variabel binding yang akan diinisialisasi nanti dan digunakan untuk mengakses komponen dalam layout activity melalui View Binding. Pada baris ke [10] sampai [14] terdapat fungsi `onCreate` yang merupakan lifecycle method yang dijalankan saat activity pertama kali dibuat. Pada baris ke [11] terdapat `super.onCreate(savedInstanceState)` yang berfungsi untuk memanggil implementasi `onCreate` dari superclass (`AppCompatActivity`). Pada baris ke [12] terdapat `binding = ActivityMainBinding.inflate(layoutInflater)` yang berfungsi untuk menginisialisasi objek binding menggunakan layout inflater agar layout XML bisa digunakan dalam kode. Pada baris ke [13] terdapat `setContentView(binding.root)` yang berfungsi untuk menampilkan layout utama activity ke layar dengan menggunakan root dari objek binding.

activity_main.xml:

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen `FragmentContainerView` yang berasal dari pustaka `androidx.fragment.app`. Elemen ini digunakan sebagai wadah untuk menampilkan fragmen.

Pada baris ke [3], terdapat deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android`, yang digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [4], terdapat deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto`, yang memungkinkan penggunaan atribut kustom dari pustaka `AndroidX`.

Pada baris ke [5], terdapat atribut `android:id="@+id/nav_host_fragment"`, yang menetapkan ID unik untuk elemen `FragmentContainerView` ini sebagai `nav_host_fragment`.

Pada baris ke [6], atribut `android:name="androidx.navigation.fragment.NavHostFragment"` mendefinisikan bahwa elemen ini adalah `NavHostFragment`, yang merupakan komponen utama untuk navigasi menggunakan `Jetpack Navigation`.

Pada baris ke [7], atribut `android:layout_width="match_parent"` digunakan untuk mengatur lebar elemen agar sesuai dengan lebar layar perangkat.

Pada baris ke [8], atribut `android:layout_height="match_parent"` digunakan untuk mengatur tinggi elemen agar sesuai dengan tinggi layar perangkat.

Pada baris ke [9], atribut `app:navGraph="@navigation/nav_graph"` menghubungkan elemen ini dengan file navigasi bernama `nav_graph`, yang berisi konfigurasi alur navigasi aplikasi.

Pada baris ke [10], atribut `app:defaultNavHost="true"` menetapkan elemen ini sebagai host navigasi utama, sehingga mampu menangani tindakan Back pada perangkat secara otomatis.

fragment_Games_list.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `ConstraintLayout` dari pustaka `androidx.constraintlayout.widget`. Elemen ini digunakan sebagai layout utama yang memungkinkan penempatan elemen-elemen anak dengan aturan constraint.

Pada baris ke [3], terdapat deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android`, yang digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [4], terdapat deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto`, yang memungkinkan penggunaan atribut kustom dari pustaka `AndroidX`.

Pada baris ke [5], atribut `android:background="#000000"` digunakan untuk mengatur latar belakang layout dengan warna hitam.

Pada baris ke [6], atribut `android:layout_width="match_parent"` digunakan untuk mengatur lebar layout agar sesuai dengan lebar layar perangkat.

Pada baris ke [7], atribut `android:layout_height="match_parent"` digunakan untuk mengatur tinggi layout agar sesuai dengan tinggi layar perangkat.

Pada baris ke [9], terdapat elemen `RecyclerView` dari pustaka `androidx.recyclerview.widget`. Elemen ini digunakan untuk menampilkan daftar data dengan performa tinggi.

Pada baris ke [10], atribut `android:id="@+id/recyclerView"` menetapkan ID unik untuk elemen ini sebagai `recyclerView`.

Pada baris ke [11], atribut `android:contentDescription="@string/Games_list_desc"` memberikan deskripsi konten yang bersifat aksesibilitas untuk `RecyclerView`, menggunakan nilai dari sumber string bernama `Games_list_desc`.

Pada baris ke [12] sampai [13], terdapat atribut `android:layout_width="0dp"` dan `android:layout_height="0dp"` mengatur lebar dan tinggi elemen ini agar dapat diatur sepenuhnya oleh constraint.

Pada baris ke [14], atribut `android:padding="8dp"` menambahkan jarak dalam sebesar 8dp di seluruh sisi elemen ini.

Pada baris ke [15], atribut `app:layoutManager="LinearLayoutManager"` menetapkan tata letak RecyclerView menggunakan `LinearLayoutManager`, yang menyusun elemen secara vertikal atau horizontal.

Pada baris ke [15] hingga [18], terdapat atribut constraint yang menghubungkan sisi atas, bawah, kiri, dan kanan RecyclerView dengan sisi parent, sehingga elemen ini berada di tengah layout secara proporsional.

fragment_detail.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `ScrollView` yang berasal dari pustaka bawaan Android. Elemen ini digunakan sebagai wadah untuk memungkinkan konten di dalamnya dapat digulir secara vertikal. deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android` digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [3], deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto` memungkinkan penggunaan atribut kustom dari pustaka `AndroidX` atau `Material`.

Pada baris ke [4], atribut `android:background="#131418"` mengatur latar belakang elemen dengan warna abu-abu gelap (#131418).

Pada baris ke [5] dan [6], atribut `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan untuk mengatur elemen agar mengisi seluruh lebar dan tinggi layar perangkat.

Pada baris ke [8] sampai [9], terdapat elemen `LinearLayout` dengan orientasi vertikal yang berfungsi sebagai kontainer utama untuk menampung elemen-elemen lainnya.

Pada baris ke [10] sampai [12], atribut `android:padding="16dp"` menambahkan jarak sebesar 16dp di semua sisi elemen ini untuk memberikan ruang antara konten dan tepi layar.

Pada baris ke [14] hingga [19], terdapat elemen `ShapeableImageView` dari pustaka `Material Components`. Elemen ini menampilkan gambar dengan ID `detailImageView`, lebar penuh, tinggi 400dp, skala `centerCrop`, dan sudut membulat dengan gaya yang diatur oleh `@style/ShapeAppearance.App.Rounded6dp`.

Pada baris ke [21], terdapat `LinearLayout` horizontal yang digunakan untuk menempatkan teks secara berdampingan.

Pada baris ke [25], atribut `android:gravity="center_vertical"` digunakan untuk menyelaraskan konten di dalam elemen secara vertikal di tengah.

Pada baris ke [26], atribut `android:layout_marginTop="12dp"` menambahkan jarak sebesar 12dp antara elemen ini dan elemen di atasnya.

Pada baris ke [28] sampai [35], terdapat elemen `TextView` dengan ID `detailTitleText`, yang digunakan untuk menampilkan judul. Elemen ini menggunakan warna teks putih, ukuran teks 26sp, dan gaya teks tebal (bold).

Pada baris ke [37] hingga [42], terdapat elemen `TextView` dengan ID `detailYearText`, yang digunakan untuk menampilkan tahun. Elemen ini menggunakan warna teks abu-abu terang, ukuran teks 14sp, dan tinggi sesuai konten.

Pada baris ke [45] hingga [52], terdapat elemen `TextView` tanpa ID yang menampilkan teks “tentang game ini:” sebagai label. Elemen ini menggunakan warna teks putih, ukuran teks 14sp, dan gaya teks tebal.

Pada baris ke [54] hingga [60], terdapat elemen `TextView` dengan ID `detailDescText`, yang digunakan untuk menampilkan deskripsi detail. Elemen ini menggunakan warna teks putih, ukuran teks 14sp, dan margin atas sebesar 4dp untuk memberi jarak dari elemen di atasnya.

item_Games.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `CardView` dari pustaka `androidx.cardview.widget`. Elemen ini digunakan untuk menampilkan konten dalam bentuk kartu dengan sudut membulat dan latar belakang yang dapat disesuaikan.

Pada baris ke [3], deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android` digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [4], deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto` memungkinkan penggunaan atribut kustom dari pustaka AndroidX.

Pada baris ke [5] hingga [6], atribut `android:layout_width="match_parent"` dan `android:layout_height="wrap_content"` digunakan untuk mengatur elemen agar memenuhi lebar layar perangkat dengan tinggi yang menyesuaikan kontennya.

Pada baris ke [7], atribut `android:layout_margin="4dp"` menambahkan jarak luar sebesar 4dp di seluruh sisi elemen ini.

Pada baris ke [8], atribut `app:cardCornerRadius="6dp"` digunakan untuk mengatur radius sudut kartu menjadi 6dp agar tampak membulat.

Pada baris ke [9], atribut `app:cardBackgroundColor="#343539"` mengatur warna latar belakang kartu dengan warna abu-abu gelap (#343539).

Pada baris ke [11], terdapat elemen `LinearLayout` dengan orientasi horizontal yang digunakan untuk menata elemen-elemen anak secara berdampingan.

Pada baris ke [15], atribut `android:padding="8dp"` menambahkan jarak dalam sebesar 8dp di seluruh sisi elemen ini.

Pada baris ke [18] hingga [25], terdapat elemen `ShapeableImageView` dari pustaka `Material Components`, yang menampilkan gambar dengan ID `imageView`. Elemen ini memiliki lebar 100dp, tinggi 140dp, margin kanan 12dp, skala gambar `centerCrop`, dan sudut membulat sesuai dengan gaya `@style/ShapeAppearance.App.Rounded6dp`.

Pada baris ke [27] sampai [37], terdapat `LinearLayout` vertikal yang digunakan untuk menata elemen-elemen teks dan tombol secara bertumpuk di dalamnya.

Pada baris ke [29], atribut `android:layout_weight="1"` digunakan untuk membuat elemen ini mengisi ruang yang tersisa dalam `LinearLayout` horizontal induknya.

Pada baris ke [39] hingga [53], terdapat elemen `TextView` untuk menampilkan judul dan tahun. Teks judul menggunakan ID `titleText`, warna putih, ukuran teks 20sp, dan gaya tebal, sedangkan teks tahun menggunakan ID `yearText`, warna abu-abu terang, dan ukuran teks 14sp.

Pada baris ke [56] hingga [63], terdapat elemen `TextView` tanpa ID yang menampilkan teks "tentang game ini:" sebagai label dengan warna putih, ukuran teks 14sp, dan gaya tebal.

Pada baris ke [65] hingga [71], terdapat elemen TextView dengan ID detailText yang digunakan untuk menampilkan deskripsi tambahan dengan warna putih, ukuran teks 14sp, dan margin atas 4dp.

Pada baris ke [73] hingga [78], terdapat LinearLayout horizontal yang digunakan untuk menempatkan dua tombol di sisi kanan bawah kartu.

Pada baris ke [80] hingga [86], terdapat tombol pertama dengan ID viewButton yang memiliki teks "View", lebar 100dp, tinggi 44dp, warna latar biru terang (#b0c4ff), dan warna teks hitam.

Pada baris ke [88], terdapat elemen Space dengan lebar 8dp yang digunakan untuk memberi jarak antara kedua tombol.

Pada baris ke [90] hingga [96], terdapat tombol kedua dengan ID detailButton yang memiliki teks "Detail", lebar dan tinggi yang sama seperti tombol pertama, warna latar biru terang, dan warna teks hitam.

strings.xml

Pada baris ke [1], terdapat elemen root <resources> yang digunakan untuk menampung semua sumber daya string dalam aplikasi Android.

Pada baris ke [2], terdapat elemen <string> dengan atribut name="app_name" yang menyimpan nama aplikasi, yaitu "My favorite games".

Pada baris ke [4], terdapat elemen <string> dengan atribut name="Games_list_desc" yang berfungsi sebagai deskripsi aksesibilitas untuk daftar permainan favorit, yaitu "List of favorite games".

Pada baris ke [6] sampai [15], terdapat sepuluh elemen <string> dengan atribut name="titleX" yang masing-masing menyimpan judul permainan favorit berdasarkan nomor urut, seperti "Terraria", "Mobile Legends", dan lainnya.

Pada baris ke [17] sampai [26], terdapat sepuluh elemen <string> dengan atribut name="yearX" yang masing-masing menyimpan tahun rilis permainan berdasarkan nomor urut, seperti "2011" untuk year1 dan "2020" untuk year6.

Pada baris ke [28] sampai [37], terdapat sepuluh elemen <string> dengan atribut name="detailX" yang masing-masing menyimpan deskripsi detail permainan berdasarkan nomor urut. Contohnya, detail1 berisi deskripsi "Terraria", dan detail6 berisi deskripsi "Genshin Impact".

Pada baris ke [39] sampai [48], terdapat sepuluh elemen <string> dengan atribut name="linkX" yang masing-masing menyimpan URL terkait permainan berdasarkan

nomor urut. Contohnya, link1 berisi URL ke halaman Steam "Terraria", dan link6 berisi URL ke halaman Google Play "Genshin Impact".

styles.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `<resources>` yang digunakan untuk menampung semua sumber daya (style) dalam aplikasi Android.

Pada baris ke [3], terdapat elemen `<style>` dengan atribut `name="ShapeAppearance.App.Rounded6dp"` yang mendefinisikan gaya bernama ShapeAppearance.App.Rounded6dp. Gaya ini digunakan untuk mengatur tampilan sudut elemen tertentu menjadi membulat dengan radius 6dp. atribut `parent=""` tidak memiliki nilai tertentu, menunjukkan bahwa gaya ini tidak mewarisi properti dari gaya lain.

Pada baris ke [4], terdapat elemen `<item>` dengan atribut `name="cornerFamily"` dan nilai `rounded`, yang menentukan bahwa elemen-elemen yang menggunakan gaya ini akan memiliki sudut membulat.

Pada baris ke [5], terdapat elemen `<item>` dengan atribut `name="cornerSize"` dan nilai `6dp`, yang menentukan ukuran radius sudut membulat sebesar 6dp.

Pada baris ke [6], elemen `</style>` menutup definisi gaya ShapeAppearance.App.Rounded6dp.

Pada baris ke [7], elemen `</resources>` menutup elemen root `<resources>`.

COMPOSE :

Datasource.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.data` yang berfungsi sebagai deklarasi nama paket tempat file ini disimpan dalam struktur proyek aplikasi Android.

Pada baris ke [2] berfungsi untuk mengimpor file R, yaitu file auto-generated yang berisi referensi ke semua sumber daya (seperti string, drawable, dan layout) dalam aplikasi.

Pada baris ke [3] digunakan untuk mengimpor kelas Games dari package model, yaitu model data yang digunakan untuk merepresentasikan setiap item dalam daftar favorit.

Pada baris ke [5] terdapat deklarasi class Datasource yang berfungsi sebagai sumber data untuk aplikasi ini.

Pada baris ke [6] sampai [7] terdapat fungsi loadGames() yang akan mengembalikan sebuah List<Games>, yaitu daftar data yang berisi 10 objek Games.

Pada baris ke [8] sampai [17] setiap item Games diisi menggunakan referensi dari resource R.string, R.drawable, dan R.string lainnya. Setiap objek menyimpan detail teks, gambar, link, tahun rilis, dan judul dari game favorit.

Games.kt:

Pada baris ke [1] terdapat package com.pemrogramanmobile.myfavoritegames.model yang berfungsi sebagai deklarasi nama paket tempat file ini disimpan, dan menunjukkan bahwa file ini berada di dalam struktur folder model.

Pada baris ke [2] sampai [3], dilakukan impor terhadap anotasi @DrawableRes dan @StringRes dari AndroidX. Anotasi ini digunakan untuk memberikan informasi bahwa parameter yang digunakan harus berupa ID dari resource drawable dan string, sehingga membantu Android Studio melakukan validasi dan memberikan peringatan bila terjadi kesalahan tipe resource.

Pada baris ke [5] terdapat deklarasi data class Games yang merupakan sebuah kelas data yang berfungsi untuk menyimpan informasi dari satu item game favorit. Dengan menggunakan data class, Kotlin secara otomatis akan menyediakan fungsi toString, equals, dan hashCode yang mempermudah pengelolaan data dalam list atau tampilan antarmuka.

Pada baris ke [6] sampai [10] terdapat lima properti yang dideklarasikan di dalam konstruktor Games, yaitu detailResourceId, imageResourceId, steamLinkResourceId, yearResourceId, dan titleResourceId. Semuanya bertipe Int karena merujuk pada resource ID yang didefinisikan dalam file res/values/strings.xml dan res/drawable. Properti detailResourceId, steamLinkResourceId, yearResourceId, dan titleResourceId masing-masing diberi anotasi @StringRes untuk menandakan bahwa nilai tersebut adalah ID dari string resource. Sedangkan imageResourceId menggunakan anotasi @DrawableRes karena nilainya berupa ID dari gambar (drawable).

GamesCard.kt

Pada baris ke [1] terdapat package com.pemrogramanmobile.myfavoritegames yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file.

Pada baris ke [3] sampai [22], dilakukan impor berbagai komponen penting dari Android Jetpack Compose seperti layout, gambar, teks, warna, bentuk sudut, serta intent untuk membuka halaman baru. Impor ini juga mencakup LocalContext dan stringResource untuk mengambil data dari resource string dan drawable berdasarkan ID.

Pada baris ke [25] sampai [29], terdapat fungsi composable bernama GamesCard yang menerima tiga parameter, yaitu objek Games dari tipe data Games, modifier yang

bersifat opsional dengan nilai default Modifier, dan `onDetailClick` sebagai fungsi lambda yang dijalankan saat tombol Detail ditekan.

Pada baris ke [30] terdapat deklarasi `val context = LocalContext.current` yang digunakan untuk mengambil context saat ini, yang diperlukan ketika akan membuat intent untuk membuka halaman link.

Pada baris ke [32] sampai [123] digunakan untuk membentuk struktur tampilan kartu menggunakan komponen Card dari Material3 dengan padding 12.dp dan warna latar belakang abu-abu gelap.

Pada baris ke [36] digunakan Row agar isi kartu tersusun secara horizontal.

Pada baris ke [38] sampai [48] digunakan Image untuk menampilkan gambar dari game, dengan pengambilan resource dari `Games.imageResourceId`. Gambar diberi padding kiri, ukuran lebar 100.dp dan tinggi 140.dp, serta sudut yang dibulatkan menggunakan `clip(RoundedCornerShape(6.dp))`. Gambar juga disejajarkan secara vertikal di tengah dan menggunakan `ContentScale.Crop` agar gambar menyesuaikan ukuran kotaknya.

Pada baris ke [50] sampai [121] digunakan Column sebagai kolom utama di sebelah kanan gambar yang berisi judul, tahun rilis, deskripsi singkat, dan dua tombol. Kolom diberi padding 16.dp dan diatur agar memenuhi lebar yang tersedia.

Pada baris ke [56] sampai [76] digunakan Row di bagian atas kolom untuk menampilkan judul game dan tahun rilis secara berdampingan. Judul menggunakan `fontSize 26.sp` dan `FontWeight.Bold`, sedangkan tahun rilis menggunakan warna teks abu-abu muda dan ukuran 16.sp.

Pada baris ke [80] hingga [90] digunakan Column yang menampilkan label teks "tentang game ini:" dan dilanjutkan dengan teks deskripsi dari resource `Games.detailResourceId`. Seluruh teks menggunakan warna putih dengan ukuran font 14.sp.

Pada baris ke [94] hingga [120] digunakan Row untuk menyusun dua buah tombol secara horizontal di bagian kanan bawah kartu. Tombol pertama diberi label "View" dan ketika ditekan akan membuka link Steam dari `Games.steamLinkResourceId` menggunakan `Intent.ACTION_VIEW` dan method `startActivity`. Tombol kedua diberi label "Detail" dan ketika ditekan akan menjalankan lambda `onDetailClick` untuk menampilkan informasi lebih lengkap. Kedua tombol memiliki ukuran lebar 100.dp dan tinggi 44.dp, serta menggunakan warna latar biru muda dengan teks berwarna hitam.

DetailScreen.kt

Pada baris ke [1] terdapat package `com.pemrogramanmobile.myfavoritegames` yang berfungsi sebagai penamaan paket dari aplikasi dan sebagai tempat penyimpanan file `DetailScreen.kt`.

Pada baris ke [3] sampai [18], berfungsi untuk mengimpor berbagai komponen penting dari Jetpack Compose seperti Image, Column, Row, Text, Surface, Modifier, Color, dan lainnya yang digunakan untuk membuat tampilan layar detail game.

Pada baris ke [20] terdapat anotasi `@Composable` yang menunjukkan bahwa fungsi `DetailScreen` merupakan fungsi composable, yaitu fungsi yang digunakan untuk membangun UI di Jetpack Compose.

Pada baris ke [21] terdapat deklarasi fungsi `DetailScreen` dengan parameter `titleResId`, `imageResId`, `yearResId`, dan `detailResId` bertipe `Int`, yang digunakan untuk menampilkan detail game berdasarkan resource ID yang diberikan.

Pada baris ke [22] sampai [25] digunakan `Surface` sebagai wadah utama tampilan yang memenuhi ukuran layar penuh dan memiliki warna latar belakang `Color(0xFF131418)`.

Pada baris ke [26] terdapat `Column` yang digunakan untuk menyusun komponen secara vertikal, dengan jarak padding sebesar 16dp di sekeliling konten.

Pada baris ke [28] sampai [36] digunakan `Image` untuk menampilkan gambar game berdasarkan `imageResId`. Gambar memenuhi lebar penuh (`fillMaxWidth`), tinggi 400dp, memiliki sudut membulat, dan menggunakan `ContentScale.Crop` agar gambar menyesuaikan ukuran.

Pada baris ke [38] sampai [53] digunakan `Column` dan `Row` untuk menampilkan judul dan tahun rilis game. Judul menggunakan ukuran font 26sp dan tebal dengan warna putih, sedangkan tahun menggunakan ukuran font 16sp dengan warna abu-abu keunguan.

Pada baris ke [55] berfungsi untuk menjadi `Spacer` untuk memberi jarak vertikal 8dp antara bagian judul dan deskripsi.

Pada baris ke [57] sampai [66] digunakan `Text` untuk menampilkan label "tentang game ini:" dan isi detail deskripsi dari game. Label ditampilkan dalam font 14sp tebal berwarna putih, sedangkan isi detail ditampilkan dalam font 14sp biasa juga berwarna putih, dengan padding atas sebesar 4dp.

HomeScreen.kt

Pada baris ke [1] terdapat package `com.pemrogramanmobile.myfavoritegames` yang berfungsi untuk menentukan lokasi file ini dalam struktur proyek.

Pada baris ke [3] sampai [6] digunakan `import` untuk memanggil fungsi dan komponen penting dari Jetpack Compose, yaitu `LazyColumn`, `items`, dan `Composable`, yang digunakan untuk membuat tampilan daftar secara efisien dan dapat diskrol.

Pada baris ke [8] sampai [9] diimpor `Datasource` dan `Games` dari package `data` dan `model`, yang masing-masing menyediakan data dummy dan struktur data untuk ditampilkan di layar.

Pada baris ke [11] terdapat anotasi `@Composable` yang menunjukkan bahwa `HomeScreen` adalah fungsi composable yang digunakan untuk membuat UI layar utama dari aplikasi.

Pada baris ke [12] sampai [15] terdapat deklarasi fungsi `HomeScreen` dengan parameter modifier (opsional untuk pengaturan tampilan) dan `onDetailClick`, yaitu lambda function yang menangani aksi ketika pengguna mengklik salah satu item untuk melihat detailnya.

Pada baris ke [16] dipanggil fungsi `loadGames()` dari `Datasource()` untuk memuat daftar data Games.

Pada baris ke [18] sampai [22] digunakan `LazyColumn` untuk menampilkan daftar secara vertikal dan efisien. Setiap item dalam daftar Games akan ditampilkan dengan memanggil fungsi `GamesCard`, dan saat diklik, akan memanggil lambda `onDetailClick` dengan data item tersebut.

MainActivity.kt:

Pada baris ke [1] terdapat package `com.pemrogramanmobile.myfavoritegames` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file.

Pada baris ke [3] sampai [13], berfungsi untuk mengimpor beberapa komponen penting dari Android seperti `Bundle`, `ComponentActivity`, `setContent`, `Surface`, `Color`, dan komponen-komponen navigasi dari Jetpack Compose yang digunakan dalam aplikasi ini.

Pada baris ke [15] terdapat class `MainActivity : ComponentActivity()` yang berfungsi untuk membuat class `MainActivity` sebagai activity utama aplikasi ini.

Pada baris ke [16] sampai [19] berfungsi untuk membuat fungsi `onCreate` yang akan dijalankan saat activity dimulai, lalu menampilkan tampilan aplikasi menggunakan tema `MyFavoriteGamesTheme`.

Pada baris ke [20] sampai [54] digunakan untuk menampilkan `Surface` sebagai latar belakang aplikasi dengan warna `Color(0xFF131418)`.

Pada baris ke [21] terdapat deklarasi `NavController` menggunakan `rememberNavController()` yang berfungsi untuk mengatur navigasi antar layar dalam aplikasi.

Pada baris ke [23] sampai [53] digunakan untuk membuat `NavHost` dengan `startDestination` bernama "home", yang berisi dua composable: "home" dan "detail".

Pada baris ke [25] sampai [29] digunakan untuk menampilkan `HomeScreen` pada rute "home", dengan parameter `onDetailClick` yang menerima objek Games lalu

menavigasi ke layar detail sambil membawa beberapa nilai resource ID seperti title, image, year, dan detail.

Pada baris ke [32] sampai [40] digunakan untuk menampilkan composable "detail" dengan parameter yang dikirim dari navigasi berupa titleResId, imageResId, yearResId, dan detailResId, yang semuanya bertipe Int.

Pada baris ke [40] sampai [51] digunakan untuk mengambil nilai-nilai argumen dari backStackEntry dan menampilkannya pada DetailScreen.

strings.xml

Pada baris ke [1], terdapat elemen root <resources> yang digunakan untuk menampung semua sumber daya string dalam aplikasi Android.

Pada baris ke [2], terdapat elemen <string> dengan atribut name="app_name" yang menyimpan nama aplikasi, yaitu "My favorite games".

Pada baris ke [4] sampai [13], terdapat sepuluh elemen <string> dengan atribut name="titleX" yang masing-masing menyimpan judul permainan favorit berdasarkan nomor urut, seperti "Terraria", "Mobile Legends", dan lainnya.

Pada baris ke [15] sampai [24], terdapat sepuluh elemen <string> dengan atribut name="yearX" yang masing-masing menyimpan tahun rilis permainan berdasarkan nomor urut, seperti "2011" untuk year1 dan "2020" untuk year6.

Pada baris ke [26] sampai [35], terdapat sepuluh elemen <string> dengan atribut name="detailX" yang masing-masing menyimpan deskripsi detail permainan berdasarkan nomor urut. Contohnya, detail1 berisi deskripsi "Terraria", dan detail6 berisi deskripsi "Genshin Impact".

Pada baris ke [37] sampai [46], terdapat sepuluh elemen <string> dengan atribut name="linkX" yang masing-masing menyimpan URL terkait permainan berdasarkan nomor urut. Contohnya, link1 berisi URL ke halaman Steam "Terraria", dan link6 berisi URL ke halaman Google Play "Genshin Impact".

SOAL 2

Soal Praktikum:

2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?.

A. Pembahasan

RecyclerView masih umum digunakan padahal memiliki kode yang panjang dan bersifat boiler-plate dibandingkan LazyColumn dengan kode yang lebih singkat dikarenakan beberapa hal berikut :

1. Kompatibilitas dan dukungannya yang cukup luas

RecyclerView masih ada di dalam Pustaka android.recyclerview dan didukung secara luas dalam berbagai versi aplikasi android. Selain itu, aplikasi yang dibuat sebelum mengenal jetpack compose banyak menggunakan RecyclerView sehingga contohnya banyak tersedia.

2. Fleksibel dan memiliki kendali penuh

RecyclerView memberikan pengguna kemampuan untuk mengontrol tampilannya, secara fleksibel contohnya sendiri seperti mengatur tata letak, animasinya, atau perilaku scroll.

3. Dukungan yang sangat stabil

RecyclerView merupakan komponen yang stabil digunakan karena telah melewati banyak uji coba, sehingga banyak digunakan dalam aplikasi produksi. Selain itu RecyclerView bisa dimasukkan Solusi atau library pihak ketiga, dan dapat terintegrasi dengan baik.

MODUL 4 : VIEWMODEL AND DEBUGGING

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada

Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.

b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel

c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment

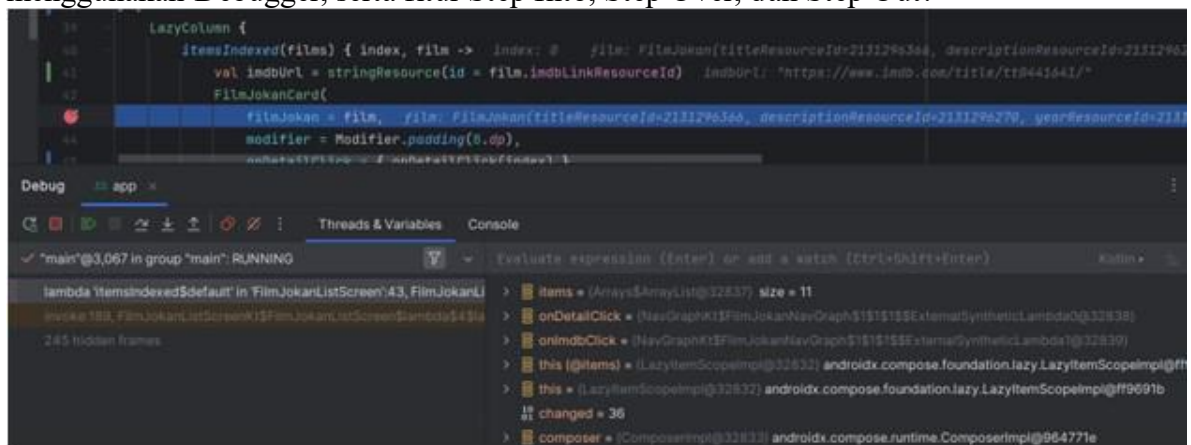
d. Install dan gunakan library Timber untuk logging event berikut:

a. Log saat data item masuk ke dalam list

b. Log saat tombol Detail dan tombol Explicit Intent ditekan

c. Log data dari list yang dipilih ketika berpindah ke halaman Detail

e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.



Gambar 25. Contoh Penggunaan Debugger Modul 4

A. Source Code

XML :

Games.kt

Table 26. source code soal 1 Games.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritegames.data
2	
3	import java.io.Serializable
4	
5	data class Games(6 val detailResourceId: Int, 7 val imageResourceId: Int, 8 val steamLinkResourceId: Int, 9 val yearResourceId: Int, 10 val titleResourceId: Int, 11 val category: String 12) : Serializable

DataSource.kt

Table 27. source code soal 1 DataSource.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritegames.data
2	
3	import com.pemrogamanmobile.myfavoritegames.R
4	
5	object DataSource { 6 fun loadGames(): List<Games> = listOf(7 Games(R.string.detail1, R.drawable.image1, R.string.link1, 8 Games(R.string.detail2, R.drawable.image2, R.string.link2, 9 Games(R.string.detail3, R.drawable.image3, R.string.link3, 10 Games(R.string.detail4, R.drawable.image4, R.string.link4, 11 Games(R.string.detail5, R.drawable.image5, R.string.link5, 12 Games(R.string.detail6, R.drawable.image6, R.string.link6, 13 Games(R.string.detail7, R.drawable.image7, R.string.link7, 14 Games(R.string.detail8, R.drawable.image8, R.string.link8, 15 Games(R.string.detail8, R.drawable.image8, R.string.link8,

16	Games(R.string.detail9, R.drawable.image9, R.string.link9, R.string.year9, R.string.title9, "All"), Games(R.string.detail10, R.drawable.image10, R.string.link10, R.string.year10, R.string.title10, "All")) }
----	--

GamesAdapter.kt

Table 28. source code soal 1 GamesAdapter.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritgames.ui
2	
3	import android.annotation.SuppressLint
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.recyclerview.widget.RecyclerView
7	import com.pemrogamanmobile.myfavoritgames.data.Games
8	import com.pemrogamanmobile.myfavoritgames.databinding.ItemGamesBinding
9	import timber.log.Timber
10	
11	class GamesAdapter(12 private val onViewClicked: (Games) -> Unit, 13 private val onDetailClicked: (Games) -> Unit 14) : RecyclerView.Adapter<GamesAdapter.ViewHolder>() { 15 16 private val items = mutableListOf<Games>() 17 18 inner class ViewHolder(val binding: ItemGamesBinding) : RecyclerView.ViewHolder(binding.root) 19 20 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder { 21 val binding = ItemGamesBinding.inflate(22 LayoutInflater.from(parent.context), parent, false 23) 24 return ViewHolder(binding) 25 } 26 27 override fun onBindViewHolder(holder: ViewHolder, position: Int) { 28 val item = items[position] 29 with(holder.binding) { 30 31

	// tambah padding(entah knp di xmlnya gak mau nambah
32	padding ke bawah, mau gak mau ya lewat sini yang ulun tau)
	val layoutParams = root.layoutParams as
33	ViewGroup.MarginLayoutParams
34	layoutParams.bottomMargin = 60
35	root.layoutParams = layoutParams
36	
37	imageView.setImageResource(item.imageResourceId)
	titleText.text =
38	holder.itemView.context.getString(item.titleResourceId)
	yearText.text =
39	holder.itemView.context.getString(item.yearResourceId)
	detailText.text =
40	holder.itemView.context.getString(item.detailResourceId)
41	
42	viewButton.setOnClickListener {
	val title =
43	holder.itemView.context.getString(item.titleResourceId)
44	Timber.d("View button clicked: \$title")
45	onViewClicked(item)
46	}
47	
48	detailButton.setOnClickListener {
	val title =
49	holder.itemView.context.getString(item.titleResourceId)
50	Timber.d("Detail button clicked: \$title")
51	onDetailClicked(item)
52	}
53	}
54	}
55	
56	override fun getItemCount(): Int = items.size
57	
58	@SuppressWarnings("NotifyDataSetChanged")
59	fun submitList(newItems: List<Games>) {
60	items.clear()
61	items.addAll(newItems)
62	notifyDataSetChanged()
63	}
64	}

GamesListFragment.kt

Table 29. source code soal 1 GamesListFragment.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritegames.ui
---	---

```

2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.view.LayoutInflater
7 import android.view.View
8 import android.view.ViewGroup
9 import androidx.fragment.app.Fragment
10 import androidx.fragment.app.viewModels
11 import androidx.lifecycle.Lifecycle
12 import androidx.lifecycle.LifecycleScope
13 import androidx.lifecycle.repeatOnLifecycle
14 import androidx.navigation.fragment.findNavController
15 import com.pemrogamanmobile.myfavoritegames.GameViewModel
16 import com.pemrogamanmobile.myfavoritegames.GameViewModelFactory
17 import
18 com.pemrogamanmobile.myfavoritegames.databinding.FragmentGamesList
19 import kotlinx.coroutines.launch
20 import timber.log.Timber
21
22 class GamesListFragment : Fragment() {
23     private var _binding: FragmentGamesListBinding? = null
24     private val binding get() = _binding!!
25
26     private val viewModel: GameViewModel by viewModels {
27         GameViewModelFactory("All") // string pakai All
28     }
29
30     override fun onCreateView(
31         inflater: LayoutInflater, container: ViewGroup?,
32         savedInstanceState: Bundle?
33     ) = FragmentGamesListBinding.inflate(inflater, container, false)
34         .also { _binding = it }
35         .root
36
37     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
38         val adapter = GamesAdapter(
39             onViewClicked = { game ->
40                 val title =
41                     requireContext().getString(game.titleResourceId)
42                     Timber.d("Launching external app with title: $title")
43
44                 // Intent ke aplikasi lain menggunakan URI
45                 val url =
46                     requireContext().getString(game.steamLinkResourceId)

```

```

43         Timber.d("Intent to other app, and the game is: $
44         val intent = Intent(Intent.ACTION_VIEW, Uri.parse
45         startActivity(intent)
46     },
47     onDetailClicked = { game ->
48         val title =
49         requireContext().getString(game.titleResourceId)
50         Timber.d("Navigating to DetailFragment for game:
51         viewModel.onGameClicked(game)
52         val action =
53         GamesListFragmentDirections.actionListToDetail(game)
54         findNavController().navigate(action)
55     }
56 )
57 binding.recyclerView.adapter = adapter
58
59     viewLifecycleOwner.lifecycleScope.launch {
60     viewLifecycleOwner.lifecycle.repeatOnLifecycle(Lifecycle.State.ST
61         viewModel.gamesState.collect { games ->
62             Timber.d("Games list updated: $games")
63             adapter.submitList(games)
64         }
65     }
66 }
67
68 override fun onDestroyView() {
69     super.onDestroyView()
70     _binding = null
71 }

```

DetailFragment.kt

Table 30. source code soal 1 DetailFragment.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritegames.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.navArgs
9	import com.pemrogamanmobile.myfavoritegames.databinding.FragmentDetailBinding
10	class DetailFragment : Fragment() {
11	private var _binding: FragmentDetailBinding? = null
12	private val binding get() = _binding!!
13	
14	private val args: DetailFragmentArgs by navArgs()
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?
18	) = FragmentDetailBinding.inflate(inflater, container, false)
19	.also { _binding = it }
20	.root
21	
22	override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
23	val games = args.games
24	with(binding) {
25	detailImageView.setImageResource(games.imageResourceId)
26	detailTitleText.text = getString(games.titleResourceId)
27	detailYearText.text = getString(games.yearResourceId)
28	detailDescText.text = getString(games.detailResourceId)
29	}
30	}
31	
32	override fun onDestroyView() {
33	super.onDestroyView()
34	_binding = null
35	}
36	}
37	}
37	

MainActivity.kt

Table 31. source code soal 1 mainactivity.kt (xml) Modul 4

```
1 package com.pemrogamanmobile.myfavoritegames
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import
6     com.pemrogamanmobile.myfavoritegames.databinding.ActivityMainBinding
7
8 class MainActivity : AppCompatActivity() {
9     private lateinit var binding: ActivityMainBinding
10
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         binding = ActivityMainBinding.inflate(layoutInflater)
14         setContentView(binding.root)
15     }
16 }
```

activity_main.xml

Table 32. source code soal 1 activity_main.xml (xml) Modul 4

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.fragment.app.FragmentContainerView
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     android:id="@+id/nav_host_fragment"
6
7     android:name="androidx.navigation.fragment.NavHostFragment"
8     android:layout_width="match_parent"
9     android:layout_height="match_parent"
10    app:navGraph="@navigation/nav_graph"
11    app:defaultNavHost="true"/>
```

Fragment_games_list.xml

Table 33. source code soal 1 fragment_games_list.xml (xml) Modul 4

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     android:background="#000000"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent">
8
9     <androidx.recyclerview.widget.RecyclerView
10         android:id="@+id/recyclerView"
11         android:contentDescription="@string/games_list_desc"
12         android:layout_width="0dp"
```


13	android:layout_height="0dp"
14	android:padding="8dp"
15	app:layoutManager="LinearLayoutManager"
16	app:layout_constraintTop_toTopOf="parent"
17	app:layout_constraintBottom_toBottomOf="parent"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintEnd_toEndOf="parent"/>
20	</androidx.constraintlayout.widget.ConstraintLayout>

Fragment_detail.xml

Table 34. source code soal 1 fragment_detail.xml (xml) Modul 4

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:background="#131418"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent">
7	
8	<LinearLayout
9	android:orientation="vertical"
10	android:padding="16dp"
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content">
13	
14	<com.google.android.material.imageview.ShapeableImageView
15	android:id="@+id/detailImageView"
16	android:layout_width="match_parent"
17	android:layout_height="400dp"
18	android:scaleType="centerCrop"
19	app:shapeAppearanceOverlay="@style/ShapeAppearance.App.Rounded6dp"
20	/>
21	
22	<LinearLayout
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content"
25	android:orientation="horizontal"
26	android:gravity="center_vertical"
27	android:layout_marginTop="12dp">
28	
29	<TextView
30	android:id="@+id/detailTitleText"
31	android:layout_width="0dp"
32	android:layout_weight="1"
33	android:layout_height="wrap_content"

34	android:textColor="#FFFFFF"
35	android:textSize="26sp"
36	android:textStyle="bold"/>
37	
38	<TextView
39	android:id="@+id/detailYearText"
40	android:layout_width="wrap_content"
41	android:layout_height="wrap_content"
42	android:textColor="#7C7C86"
43	android:textSize="14sp"/>
44	</LinearLayout>
45	
46	<TextView
47	android:layout_marginTop="8dp"
48	android:layout_width="wrap_content"
49	android:layout_height="wrap_content"
50	android:text="tentang game ini:"
51	android:textColor="#FFFFFF"
52	android:textSize="14sp"
53	android:textStyle="bold"/>
54	
55	<TextView
56	android:id="@+id/detailDescText"
57	android:layout_marginTop="4dp"
58	android:layout_width="match_parent"
59	android:layout_height="wrap_content"
60	android:textColor="#FFFFFF"
61	android:textSize="14sp"/>
62	</LinearLayout>
63	</ScrollView>

Item_games.kt

Table 35. source code soal 1 item_games.xml (xml) Modul 4

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	app:cardCornerRadius="6dp"
8	app:cardBackgroundColor="#343539">
9	
10	<LinearLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"

13	android:orientation="horizontal"
14	android:padding="8dp"
15	android:gravity="center_vertical">
16	
17	<com.google.android.material.imageview.ShapeableImageView
18	android:id="@+id/imageView"
19	android:layout_width="100dp"
20	android:layout_height="140dp"
21	android:layout_marginEnd="12dp"
22	android:scaleType="centerCrop"
23	app:shapeAppearanceOverlay="@style/ShapeAppearance.App.Rounded6dp"
24	/>
25	
26	<LinearLayout
27	android:layout_width="0dp"
28	android:layout_weight="1"
29	android:orientation="vertical"
30	android:layout_height="wrap_content">
31	
32	<LinearLayout
33	android:layout_width="match_parent"
34	android:layout_height="wrap_content"
35	android:orientation="horizontal"
36	android:gravity="center_vertical">
37	
38	<TextView
39	android:id="@+id/titleText"
40	android:layout_width="0dp"
41	android:layout_weight="1"
42	android:layout_height="wrap_content"
43	android:textColor="@android:color/white"
44	android:textSize="20sp"
45	android:textStyle="bold"/>
46	
47	<TextView
48	android:id="@+id/yearText"
49	android:layout_width="wrap_content"
50	android:layout_height="wrap_content"
51	android:textColor="#7C7C86"
52	android:textSize="14sp"/>
53	</LinearLayout>
54	
55	<TextView

56	android:layout_marginTop="8dp"	
57	android:layout_width="wrap_content"	
58	android:layout_height="wrap_content"	
59	android:text="tentang game ini:"	
60	android:textColor="@android:color/white"	
61	android:textSize="14sp"	
62	android:textStyle="bold"/>	
63		
64	<TextView	
65	android:id="@+id/detailText"	
66	android:layout_marginTop="4dp"	
67	android:layout_width="wrap_content"	
68	android:layout_height="wrap_content"	
69	android:textColor="@android:color/white"	
70	android:textSize="14sp"/>	
71		
72	<LinearLayout	
73	android:layout_width="match_parent"	
74	android:layout_height="wrap_content"	
75	android:orientation="horizontal"	
76	android:gravity="end"	
77	android:layout_marginTop="12dp">	
78		
79	<Button	
80	android:id="@+id/viewButton"	
81	android:layout_width="100dp"	
82	android:layout_height="44dp"	
83	android:text="View"	
84	android:backgroundTint="#b0c4ff"	
85	android:textColor="#000"/>	
86		
87	<Space	android:layout_width="8dp"
88	android:layout_height="wrap_content"/>	
89	<Button	
90	android:id="@+id/detailButton"	
91	android:layout_width="100dp"	
92	android:layout_height="44dp"	
93	android:text="Detail"	
94	android:backgroundTint="#b0c4ff"	
95	android:textColor="#000"/>	
96	</LinearLayout>	
97	</LinearLayout>	
98	</LinearLayout>	

99	</androidx.cardview.widget.CardView>
----	--------------------------------------

strings.xml

Table 36. source code soal 1 strings.xml (xml) Modul 4

1	<resources>
2	<string name="app_name">My favorite games</string>
3	
4	<string name="games_list_desc">List of favorite games</string>
5	
6	<string name="title1">Terraria</string>
7	<string name="title2">Mobile Legends</string>
8	<string name="title3">Firewatch</string>
9	<string name="title4">The Elder Scrolls : Skyrim</string>
10	<string name="title5">Monster Hunter World</string>
11	<string name="title6">Genshin Impact</string>
12	<string name="title7">Delta Force</string>
13	<string name="title8">Magic Chess Go Go</string>
14	<string name="title9">STAR WARS Jedi: Fallen Order</string>
15	<string name="title10">Minecraft</string>
16	
17	<string name="year1">2011</string>
18	<string name="year2">2016</string>
19	<string name="year3">2016</string>
20	<string name="year4">2016</string>
21	<string name="year5">2018</string>
22	<string name="year6">2020</string>
23	<string name="year7">2025</string>
24	<string name="year8">2025</string>
25	<string name="year9">2019</string>
26	<string name="year10">2011</string>
27	
28	<string name="detail1">Dig, fight, explore, build! Nothing is game. Four Pack also available!</string>
29	<string name="detail2">Bergabung dengan teman Anda di Mobile Game MOBA 5v5 terbaru, dan bertarung melawan Player sungguhan! Anda dan bentuk tim yang sempurna dengan teman Anda! Matchmaking 10 10 menit. Laning, Jungling, Team Fight, dan menghancurkan Turre dari Game aksi dan MOBA PC berada dalam genggamannya Anda! Tunjukk
30	Anda!</string>
	<string name="detail3">Firewatch is a single-player first-per
31	where your only emotional lifeline is the person on the other end
	<string name="detail4">Winner of more than 200 Game of the
	Special Edition brings the epic fantasy to life in stunning c
32	critically acclaimed game and add-ons with all-new features.</str

33	<pre> <string name="detail5">Welcome to a new world! In Monster Hunter series, you can enjoy the ultimate hunting experience, using every a new world teeming with surprises and excitement.</string> <string name="detail6">Genshin Impact adalah game RPG Open World yang menjelajah sebuah dunia fantasi yang teramat luas, kamu dapat karakter berkemampuan dan berkepribadian unik, melawan musuh-musuh yang hilang. Di dunia yang luas ini, biarkan rasa keingintahuanmu 34 tersembunyi, sampai akhirnya kamu dapat bertemu kembali dengan perjalananmu.</string> <string name="detail7">Pertempuran Epic Warfare 24vs24, Rasa mode permainan menarik, dan update yang terus menerus! Warfare me 35 penuh kekacauan dan kehancuran environment. Baik kalian menembak menyelamatkan rekan satu tim sebagai medic, pilih role kalian dan <string name="detail8">Magic Chess: Go Go - Game strategi multi Legends: Bang Bang. Dengan gameplay seperti catur, game ini kasual saja bersama teman-teman! Di sini, kemenangan bergantung pada keterampilan pengendalian mikro. Di setiap babak, kamu bisa m 36 meningkatkan Hero, menyusun Sinergi, menggunakan equipment, dan mengalahkan lawan. Kalahkan 7 player lain secara bertahap untuk m <string name="detail9">A galaxy-spanning adventure awaits in 37 action-adventure title from Respawn. An abandoned Padawan must co Force abilities, and master the art of the lightsaber - all Empire.</string> <string name="detail10">Bangun, jelajahi, bertahan hidup! Ma Jelajahi dunia terbuka dengan membangun, membuat item, dan bertahan Kumpulkan sumber daya, coba bertahan hidup di malam hari, dan r 38 Jelajahi dan buat item sesuka hati di dunia yang terbuka sepenuhnya 39 membangun kota balok, membuka peternakan, menambang jauh ke bawa 40 sekadar bereksperimen sejauh imajinasi membawamu!</string> 41 42 <string name="link1">https://store.steampowered.com/app/10560 43 <string name="link2">https://play.google.com/store/apps/detai 44 <string name="link3">https://store.steampowered.com/app/38387 45 <string name="link4">https://store.steampowered.com/app/489830/The_Elder_ 46 <string name="link5">https://store.steampowered.com/app/58201 47 <string name="link6">https://play.google.com/store/apps/detai 48 <string name="link7">https://play.google.com/store/apps/detai 49 <string name="link8">https://play.google.com/store/apps/detai 50 <string name="link9">https://store.steampowered.com/app/11723 51 <string name="link10">https://play.google.com/store/apps/deta 52 </resources> </pre>
----	---

styles.xml

Table 37. source code soal 1 style.xml (xml) Modul 4

1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	<style name="ShapeAppearance.App.Rounded6dp" parent="">
4	<item name="cornerFamily">rounded</item>
5	<item name="cornerSize">6dp</item>
6	</style>
7	</resources>

GamesViewModel.kt

Table 38. source code soal 1 GamesViewModel.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import androidx.lifecycle.ViewModel
4	import
	com.pemrogamanmobile.myfavoritegames.data.DataSource
5	import
6	com.pemrogamanmobile.myfavoritegames.data.Games
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.update
10	import timber.log.Timber
11	
12	class GamesViewModel(
13	private val gameCategory: String
14	) : ViewModel() {
15	
16	// StateFlow untuk menyimpan daftar game
17	private val _gamesState = MutableStateFlow(
	DataSource.loadGames().filter { it.category
18	== gameCategory }
19	)
	val gamesState: StateFlow<List<Games>> get() =
20	_gamesState
21	
22	// StateFlow untuk menyimpan game yang dipilih
	private val _selectedGame =
23	MutableStateFlow<Games?>(null)
	val selectedGame: StateFlow<Games?> get() =
24	_selectedGame
25	
26	init {

27	// Log data game yang di-load
28	Timber.d("Games data loaded: \${_gamesState.value}")
29	}
30	
31	// Fungsi untuk menangani klik pada item game
32	fun onGameClicked(game: Games) {
33	_selectedGame.update { game }
34	Timber.d("Game selected: \$game")
35	}
36	}

GamesViewModelFactory.kt

Table 39. source code soal 1 GamesViewModelFactory.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritgames
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class GamesViewModelFactory(
7	private val gameCategory: String
8	) : ViewModelProvider.Factory {
9	override fun <T : ViewModel>
10	create(modelClass: Class<T>): T {
11	if (modelClass.isAssignableFrom
12	(GamesViewModel::class.java)) {
13	@Suppress("UNCHECKED_CAST")
14	return GamesViewModel(gameCategory) as T
15	}
16	throw IllegalArgumentException("Unknown
	ViewModel class")
	}
	}

MyApplication.kt

Table 40. source code soal 1 MyApplication.kt (xml) Modul 4

1	package com.pemrogamanmobile.myfavoritgames
2	
3	import android.app.Application
4	import timber.log.Timber
5	import
6	com.pemrogamanmobile.myfavoritgames.BuildConfig
7	class MyApplication : Application() {

8	override fun onCreate() {
9	super.onCreate()
10	
11	// Inisialisasi Timber
12	if (BuildConfig.DEBUG) {
13	Timber.plant(Timber.DebugTree())
14	}
15	}
16	}

COMPOSE :

MainActivity.kt

Table 41. source code soal 1 mainactivity.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.material3.Surface
7	import androidx.compose.runtime.collectAsState
8	import androidx.compose.runtime.getValue
9	import androidx.compose.ui.graphics.Color
10	import androidx.lifecycle.viewmodel.compose.viewModel
11	import androidx.navigation.compose.NavHost
12	import androidx.navigation.compose.composable
13	import androidx.navigation.compose.rememberNavController
14	import
	com.pemrogamanmobile.myfavoritegames.data.Datasource
15	import
	com.pemrogamanmobile.myfavoritegames.ui.theme.MyFavoriteGamesTheme
16	import androidx.compose.runtime.LaunchedEffect
17	import timber.log.Timber
18	
19	class MainActivity : ComponentActivity() {
20	override fun onCreate(savedInstanceState: Bundle?) {
21	super.onCreate(savedInstanceState)
22	
23	// Log saat MainActivity dibuat
24	Timber.d("MainActivity created")
25	
26	setContent {
27	MyFavoriteGamesTheme {

```

28         Surface(color = Color(0xFF131418)) {
29             val navController =
rememberNavController()
30
31             // menggunakan parameter String "All"
32             val gamesViewModel: GamesViewModel =
viewModel(
33                 factory = GamesViewModelFactory(
34                     gameCategory = "All",
35                     datasource = Datasource()
36                 )
37             )
38
39             NavHost(navController =
navController, startDestination = "home") {
40                 composable("home") {
41                     Timber.d("Navigated to Home
Screen")
42
43                     HomeScreen(
44                         gamesViewModel =
gamesViewModel,
45                         onDetailClick = { game -
>
46                         gamesViewModel.onGameClicked(game) // set selectedGame di
ViewModel
47
48                         // Log saat tombol
Detail ditekan
49                         Timber.d("Detail
button clicked untuk game: $game")
50
51                         navController.navigate("detail")
52                     }
53                 )
54             }
55
56             composable("detail") {
57                 Timber.d("Navigated to
Detail Screen")
58
59                 val selectedGame by
gamesViewModel.selectedGame.collectAsState()

```


7	import androidx.compose.runtime.collectAsState	
8	import androidx.compose.runtime.getValue	
9	import androidx.compose.ui.Modifier	
10	import androidx.compose.ui.unit.dp	
11	import	
12	com.pemrogamanmobile.myfavoritegames.model.Games	
13		
14	@Composable	
15	fun HomeScreen(	
16	gamesViewModel: GamesViewModel,	
17	onDetailClick: (Games) -> Unit	
18	) {	
	val	games
		by
19	gamesViewModel.gamesState.collectAsState()	
20		
21	LazyColumn(	
22	verticalArrangement	=
	Arrangement.spacedBy(8.dp),	
23	modifier = Modifier.fillMaxSize()	
24	) {	
25	items(games) { game ->	
26	GameCard(	
27	games = game,	
28	onDetailClick = onDetailClick	
29	)	
30	}	
31	}	
32	}	

DetailScreen.kt

Table 43. source code soal 1 DetailScreen.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
6	androidx.compose.foundation.rememberScrollState
	import
7	androidx.compose.foundation.shape.RoundedCornerShape
8	import androidx.compose.foundation.verticalScroll
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.Alignment

```

12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.draw.clip
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.res.painterResource
16 import androidx.compose.ui.res.stringResource
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.layout.ContentScale
19 import androidx.compose.ui.text.TextStyle
20 import androidx.compose.ui.text.font.FontWeight
21 import androidx.compose.ui.unit.sp
22
23 @Composable
24 fun DetailScreen(titleResId: Int, imageResId: Int,
25 yearResId: Int, detailResId: Int) {
26     Surface(
27         modifier = Modifier.fillMaxSize(),
28         color = Color(0xFF131418)
29     ) {
30         val scrollState = rememberScrollState()
31
32         Column(
33             modifier = Modifier
34                 .verticalScroll(scrollState)
35                 .padding(16.dp)
36         ) {
37
38             Image(
39                 painter = painterResource(imageResId),
40                 contentDescription = stringResource(detailResId),
41                 modifier = Modifier
42                     .fillMaxWidth()
43                     .height(400.dp)
44                     .clip(RoundedCornerShape(6.dp)),
45                 contentScale = ContentScale.Crop
46             )
47
48             Column(modifier = Modifier.fillMaxWidth()) {
49                 Row(
50                     modifier = Modifier.fillMaxWidth(),
51

```

	horizontalArrangement	=
54	Arrangement.SpaceBetween	
	) {	
	Text(	
55	text	=
56	stringResource(titleResId),	
57	style = TextStyle(fontSize =	
58	26.sp, fontWeight = FontWeight.Bold, color =	
	Color.White),	
59	modifier	=
	Modifier.weight(1f)	
60	)	
	Text(	
61	text	=
62	stringResource(yearResId),	
63	style = TextStyle(fontSize =	
64	16.sp, color = Color(0xFF7C7C86)),	
65	modifier	=
66	Modifier.align(Alignment.CenterVertically)	
67	)	
68	}	
69	Spacer(modifier	=
70	Modifier.height(8.dp))	
71		
72	Text(	
	text = "tentang game ini:",	
73	style = TextStyle(fontSize =	=
	14.sp, fontWeight = FontWeight.Bold, color =	
74	Color.White)	
	)	
75		
76	Text(	
77	text	=
78	stringResource(detailResId),	
79	style = TextStyle(fontSize =	=
80	14.sp, color = Color.White),	
81	modifier = Modifier.padding(top =	
82	4.dp)	
83	)	
84	}	
85	}	
86	}	
87	}	

GameCard.kt

Table 44. source code soal 1 GameCard.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames	
2		
3	import android.content.Intent	
4	import androidx.compose.foundation.Image	
5	import androidx.compose.foundation.layout.*	
6	import	
	androidx.compose.foundation.shape.RoundedCornerShape	
7	import androidx.compose.material3.*	
8	import androidx.compose.runtime.Composable	
9	import androidx.compose.ui.Alignment	
10	import androidx.compose.ui.Modifier	
11	import androidx.compose.ui.draw.clip	
12	import androidx.compose.ui.graphics.Color	
13	import androidx.compose.ui.layout.ContentScale	
14	import androidx.compose.ui.platform.LocalContext	
15	import androidx.compose.ui.res.painterResource	
16	import androidx.compose.ui.res.stringResource	
17	import androidx.compose.ui.text.TextStyle	
18	import androidx.compose.ui.text.font.FontWeight	
19	import androidx.compose.ui.unit.dp	
20	import androidx.compose.ui.unit.sp	
21	import androidx.core.net.toUri	
22	import	
23	com.pemrogamanmobile.myfavoritegames.model.Games	
24	import timber.log.Timber	
25		
26	@Composable	
27	fun GameCard(	
28	games: Games,	
29	modifier: Modifier = Modifier,	
30	onDetailClick: (Games) -> Unit	
31	) {	
32	val context = LocalContext.current	
33		
34	Card(	
35	modifier = modifier.padding(12.dp),	
	colors	=
36	CardDefaults.cardColors(containerColor	=
37	Color(0xFF343539))	
38	) {	
39	Row(modifier = Modifier.fillMaxWidth()) {	
40		

41	Image (	
	painter	=
42	painterResource (games.imageResourceId),	
	contentDescription	=
43	stringResource (games.titleResourceId),	
44	modifier = Modifier	
45	.padding (start = 12.dp)	
46	.width (100.dp)	
47	.height (140.dp)	
48	.clip (RoundedCornerShape (6.dp))	
49		
50	.align (Alignment.CenterVertically),	
51	contentScale = ContentScale.Crop	
52	)	
53		
54	Column (	
55	modifier = Modifier	
	.padding (16.dp)	
56	.fillMaxWidth (),	
57	verticalArrangement	=
58	Arrangement.SpaceBetween	
59	) {	
	Row (	
60	modifier	=
61	Modifier.fillMaxWidth (),	
62	horizontalArrangement	=
63	Arrangement.SpaceBetween,	
	verticalAlignment	=
64	Alignment.Top	
	) {	
	Text (	
65	text	=
66	context.getString (games.titleResourceId),	
67	style = TextStyle (fontSize =	=
68	26.sp, fontWeight = FontWeight.Bold, color =	=
	Color.White),	
69	modifier	=
70	Modifier.weight (1f)	
71	)	
72	Spacer (modifier	=
	Modifier.width (8.dp))	
73		
	Text (	
74		

	text	=
75	context.getString(games.yearResourceId),	
76	style = TextStyle(fontSize =	
77	16.sp, color = Color(0xFF7C7C86)),	
78	modifier = Modifier	
79		
	.wrapContentWidth(Alignment.End)	
80		
81	.align(Alignment.CenterVertically)	
	)	
82	}	
83		
84	Spacer(modifier	=
	Modifier.height(8.dp))	
85	Column(modifier	=
86	Modifier.fillMaxWidth()) {	
87	Text(	
	text = "tentang game ini:",	
88	style = TextStyle(fontSize =	
	14.sp, fontWeight = FontWeight.Bold, color =	
89	Color.White)	
	)	
90	Text(	
91	text	=
92	context.getString(games.detailResourceId),	
93	style = TextStyle(fontSize =	
	14.sp, color = Color.White),	
94	modifier	=
95	Modifier.padding(top = 4.dp)	
96	)	
97	}	
98	Spacer(modifier	=
	Modifier.height(12.dp))	
99		
100	Row(	
101	modifier	=
102	Modifier.fillMaxWidth(),	
	horizontalArrangement	=
103	Arrangement.End,	
	verticalAlignment	=
104	Alignment.CenterVertically	
	) {	
	Button(	

105	onClick = {	
106	val url	=
107	context.getString(games.linkResourceId)	
	val intent	=
108	Intent(Intent.ACTION_VIEW, url.toUri())	
	Timber.d("Explicit	
	Intent triggered: Opening URL \$url") // Logging	
109	tombol "View"	
110	context.startActivity(intent)	
111	},	
112	modifier	=
113	Modifier.width(100.dp).height(44.dp),	
	colors	=
114	ButtonDefaults.buttonColors(containerColor	=
115	Color(0xFFb0c4ff), contentColor = Color.White)	
116	) {	
117	Text("View", style	=
	TextStyle(fontSize = 16.sp, color = Color.Black))	
	}	
118	Spacer(modifier	=
119	Modifier.width(8.dp))	
120		
	Button(	
121	onClick = {	
	Timber.d("Detail button	
	clicked for game(dengan judul?):	
122	\${context.getString(games.titleResourceId)}") //	
123	Logging tombol "Detail"	
	onDetailClick(games)	
124	},	
125	modifier	=
126	Modifier.width(100.dp).height(44.dp),	
127	colors	=
128	ButtonDefaults.buttonColors(containerColor	=
129	Color(0xFFb0c4ff), contentColor = Color.White)	
130	) {	
131	Text("Detail", style	=
132	TextStyle(fontSize = 16.sp, color = Color.Black))	
133	}	
134	}	
135	}	
136	}	
137	}	

138	}
-----	---

Datasource.kt

Table 45. source code soal 1 Datasource.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames.data
2	
3	import com.pemrogamanmobile.myfavoritegames.R
4	import
	com.pemrogamanmobile.myfavoritegames.model.Games
5	
6	class Datasource {
7	fun loadGames(): List<Games> {
8	return listOf(
9	Games(R.string.detail1,
	R.drawable.image1, R.string.link1, R.string.year1,
	R.string.title1, "All"),
10	Games(R.string.detail2,
	R.drawable.image2, R.string.link2, R.string.year2,
	R.string.title2, "All"),
11	Games(R.string.detail3,
	R.drawable.image3, R.string.link3, R.string.year3,
	R.string.title3, "All"),
12	Games(R.string.detail4,
	R.drawable.image4, R.string.link4, R.string.year4,
	R.string.title4, "All"),
13	Games(R.string.detail5,
	R.drawable.image5, R.string.link5, R.string.year5,
	R.string.title5, "All"),
14	Games(R.string.detail6,
	R.drawable.image6, R.string.link6, R.string.year6,
	R.string.title6, "All"),
15	Games(R.string.detail7,
	R.drawable.image7, R.string.link7, R.string.year7,
	R.string.title7, "All"),
16	Games(R.string.detail8,
	R.drawable.image8, R.string.link8, R.string.year8,
	R.string.title8, "All"),
17	Games(R.string.detail9,
	R.drawable.image9, R.string.link9, R.string.year9,
	R.string.title9, "All"),
18	Games(R.string.detail10,
	R.drawable.image10, R.string.link10,
	R.string.year10, R.string.title10, "All")
19	)
20	}

21	}
----	---

strings.xml

Table 46. source code soal 1 strings.xml (compose) Modul 4

1	<resources>
2	<string name="app_name">my favorite games</string>
3	
4	<string name="title1">Terraria</string>
5	<string name="title2">Mobile Legends</string>
6	<string name="title3">Firewatch</string>
7	<string name="title4">The Elder Scrolls : Skyrim</string>
8	<string name="title5">Monster Hunter World</string>
	<string name="title6">Genshin Impact</string>
9	<string name="title7">Delta Force</string>
10	<string name="title8">Magic Chess Go Go</string>
11	<string name="title9">STAR WARS Jedi: Fallen Order</string>
12	<string name="title10">Minecraft</string>
13	
14	<string name="year1">2011</string>
15	<string name="year2">2016</string>
16	<string name="year3">2016</string>
17	<string name="year4">2016</string>
18	<string name="year5">2018</string>
19	<string name="year6">2020</string>
20	<string name="year7">2025</string>
21	<string name="year8">2025</string>
22	<string name="year9">2019</string>
23	<string name="year10">2011</string>
24	
25	<string name="detail1">Dig, fight, explore, build! Nothing is impossible in this action-packed adventure game. Four Pack also available!</string>
26	<string name="detail2">Bergabung dengan teman Anda di Mobile Legends: Bang Bang, Game MOBA 5v5 terbaru, dan bertarung melawan Player sungguhan! Pilih Hero favorit Anda dan bentuk tim yang sempurna dengan teman Anda! Matchmaking 10 detik, pertandingan 10 menit. Laning, Jungling, Team Fight, dan menghancurkan Turret, semua kesenangan dari Game aksi dan MOBA PC berada dalam genggaman Anda! Tunjukkan semangat Esports Anda!</string>
27	<string name="detail3">Firewatch is a single-player first-person mystery set in the Wyoming wilderness, where
28	

	your only emotional lifeline is the person on the other end of a handheld radio.</string> <string name="detail4">Winner of more than 200 Game of the Year Awards, The Elder Scrolls V: Skyrim Special Edition brings the epic fantasy to life in stunning detail. The Special Edition includes the critically acclaimed game and add-ons with all-new features.</string>
29	
30	<string name="detail5">Welcome to a new world! In Monster Hunter: World, the latest installment in the series, you can enjoy the ultimate hunting experience, using everything at your disposal to hunt monsters in a new world teeming with surprises and excitement.</string>
31	<string name="detail6">Genshin Impact adalah game RPG Open World terbaru. Kamu adalah seorang pengembara yang menjelajah sebuah dunia fantasi yang teramat luas, kamu dapat menjelajahi tujuh bangsa, bertemu berbagai karakter berkemampuan dan berkepribadian unik, melawan musuh-musuh kuat, dan berkelana mencari saudaramu yang hilang. Di dunia yang luas ini, biarkan rasa keingintahuan membawamu menguak misteri-misteri yang tersembunyi, sampai akhirnya kamu dapat bertemu kembali dengan saudaramu, dan menyaksikan akhir dari kisah perjalananmu.</string>
32	<string name="detail7">Pertempuran Epic Warfare 24vs24, Rasakan map besar, puluhan senjata, berbagai mode permainan menarik, dan update yang terus menerus! Warfare membawa kalian ke medan perang dinamis yang penuh kekacauan dan kehancuran environment. Baik kalian menembaki musuh di darat, laut, dan udara, atau menyelamatkan rekan satu tim sebagai medic, pilih role kalian dan raih kemenangan!</string>
33	<string name="detail8">Magic Chess: Go Go - Game strategi multiplayer baru yang terinspirasi dari Mobile Legends: Bang Bang. Dengan gameplay seperti catur, game ini kasual dan mudah dimainkan kapan saja, di mana saja bersama teman-teman! Di sini, kemenangan bergantung pada strategi dan sedikit keberuntungan daripada keterampilan pengendalian mikro. Di setiap babak, kamu bisa mengendalikan Commander untuk membeli dan meningkatkan Hero, menyusun Sinergi, menggunakan equipment, dan menempatkan hero dengan cerdas untuk mengalahkan lawan. Kalahkan 7 player lain secara bertahap untuk memenangkan pertandingan.</string> <string name="detail9">A galaxy-spanning adventure awaits in Star Wars Jedi: Fallen Order, a 3rd person

34	action-adventure title from Respawn. An abandoned Padawan must complete his training, develop new powerful Force abilities, and master the art of the lightsaber - all while staying one step ahead of the Empire.</string> <string name="detail10">Bangun, jelajahi, bertahan hidup! Main dengan teman dan buat duniamu sendiri! Jelajahi dunia terbuka dengan membangun, membuat item, dan bertahan hidup di game sandbox membangun terbaik.
35	Kumpulkan sumber daya, coba bertahan hidup di malam hari, dan rancang petualangan epik balok demi balok. Jelajahi dan buat item sesuka hati di dunia yang terbuka sepenuhnya tempat kamu bisa bermain dengan teman, membangun kota balok, membuka peternakan, menambang jauh ke bawah tanah, menghadapi musuh misterius, atau sekadar bereksperimen sejauh imajinasi membawamu!</string> <string name="link1">https://store.steampowered.com/app/105600/Terraria/</string> <string name="link2">https://play.google.com/store/apps/details?id=com.mobile.legends</string>
36	<string name="link3">https://store.steampowered.com/app/383870/Firewatch/</string>
37	<string name="link4">https://store.steampowered.com/app/489830/The_Elder_Scrolls_V_Skyrim_Special_Edition/</string>
38	<string name="link5">https://store.steampowered.com/app/582010/Monster_Hunter_World/</string>
39	<string name="link6">https://play.google.com/store/apps/details?id=com.miHoYo.GenshinImpact</string>
40	<string name="link7">https://play.google.com/store/apps/details?id=com.garena.game.df</string>
41	<string name="link8">https://play.google.com/store/apps/details?id=com.mobilechess.gp</string>
42	<string name="link9">https://store.steampowered.com/app/1172380/STAR_WARS_Jedi_Fallen_Order/</string>
43	

44	<pre> <string name="link10">https://play.google.com/store/apps/detail s?id=com.mojang.minecraftpe</string> </resources> </pre>
----	--

Games.kt

Table 47. source code soal 1 Games.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Games(
7	@StringRes val detailResourceId: Int,
8	@DrawableRes val imageResourceId: Int,
9	@StringRes val linkResourceId: Int,
10	@StringRes val yearResourceId: Int,
11	@StringRes val titleResourceId: Int,
12	val category: String
13	)

GamesViewModel.kt

Table 48. source code soal 1 GamesViewModel.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import
7	com.pemrogamanmobile.myfavoritegames.data.Datasource
8	import
9	com.pemrogamanmobile.myfavoritegames.model.Games
10	import timber.log.Timber
11	class GamesViewModel(
12	private val gameCategory: String,
13	private val datasource: Datasource
14	) : ViewModel() {
15	
16	private val _gamesState = MutableStateFlow(

```

17         datasource.loadGames().filter { it.category
18 == gameCategory }
19     )
20     val gameState: StateFlow<List<Games>> get() =
21     _gameState
22
23     private val _selectedGame =
24     MutableStateFlow<Games?>(null)
25     val selectedGame: StateFlow<Games?> get() =
26     _selectedGame
27
28     init {
29         // Log saat data masuk ke list
30         Timber.d("Games data loaded:
31 ${_gameState.value}")
32     }
33
34     fun onGameClicked(game: Games) {
35         _selectedGame.value = game
36         // Log data game yang dipilih
37         Timber.d("Game selected: $game")
38     }
39 }

```

GamesViewModelFactory.kt

Table 49. source code soal 1 GamesViewModelFactory.kt (compose) Modul 4

```

1 package com.pemrogamanmobile.myfavoritegames
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import
6 com.pemrogamanmobile.myfavoritegames.data.Datasource
7
8 class GamesViewModelFactory(
9     private val gameCategory: String,
10    private val datasource: Datasource = Datasource()
11 ) : ViewModelProvider.Factory {
12     override fun <T : ViewModel>
13     create(modelClass: Class<T>): T {
14         if (modelClass.isAssignableFrom
15         (GamesViewModel::class.java)) {
16             @Suppress("UNCHECKED_CAST")
17

```

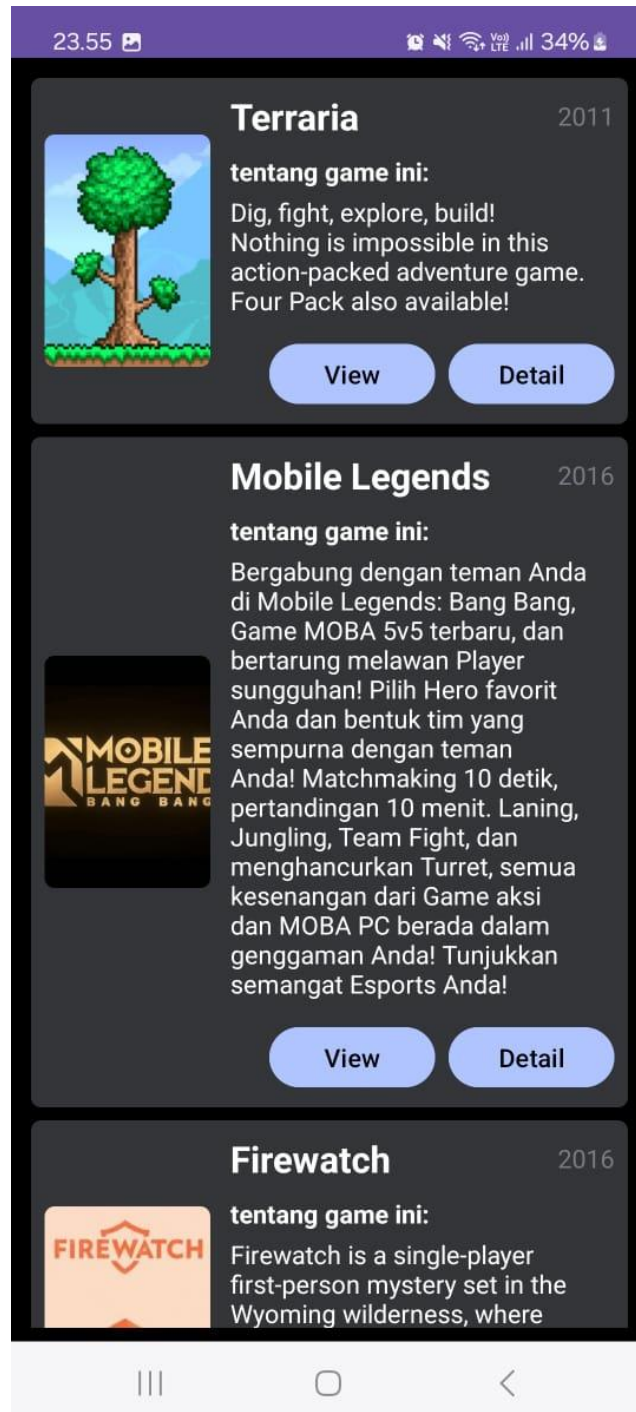

15	return GamesViewModel(gameCategory,
16	datasource) as T
17	throw IllegalArgumentException("Unknown
18	ViewModel class")
	}
	}

MyFavoriteGamesApplication.kt

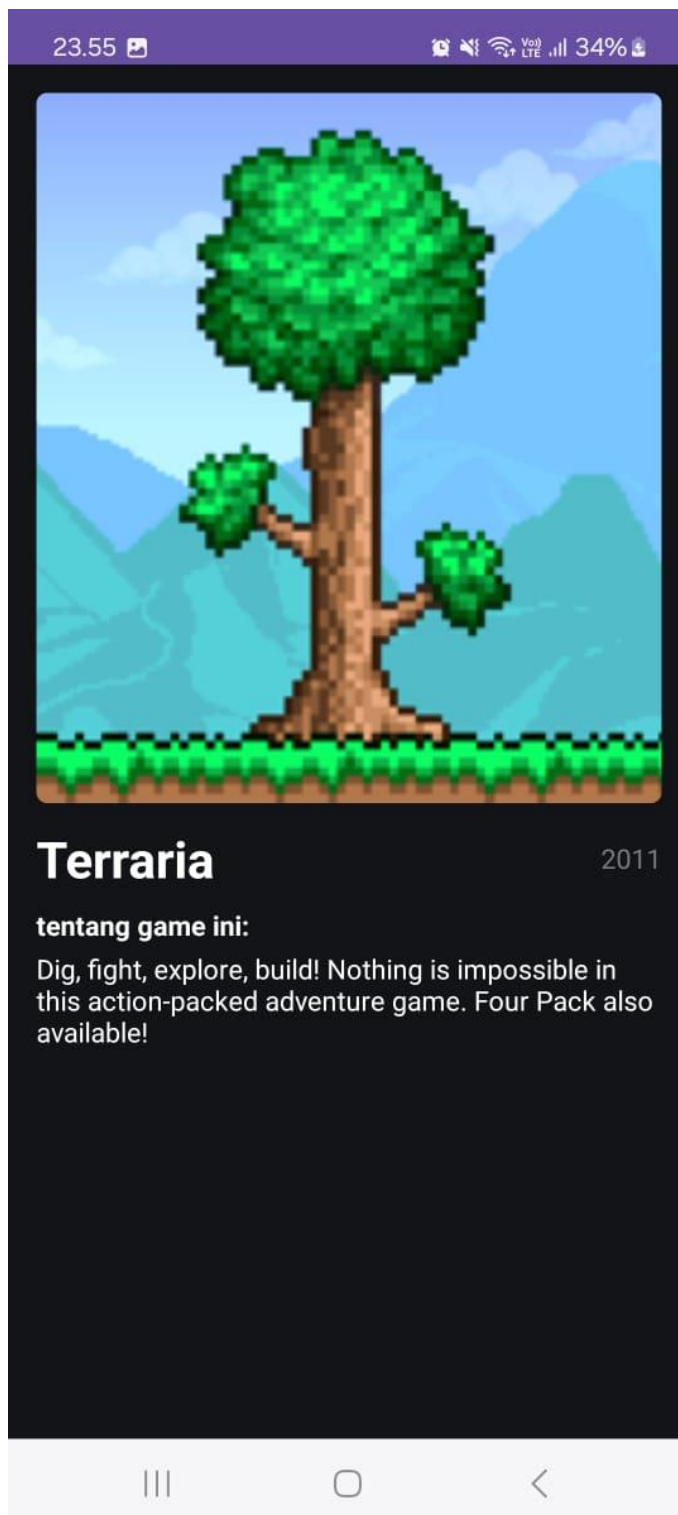
Table 50. source code soal 1 MyFavoriteGamesApplication.kt (compose) Modul 4

1	package com.pemrogamanmobile.myfavoritegames
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyFavoriteGamesApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	if (BuildConfig.DEBUG) {
10	Timber.plant(Timber.DebugTree())
11	}
12	}
13	}

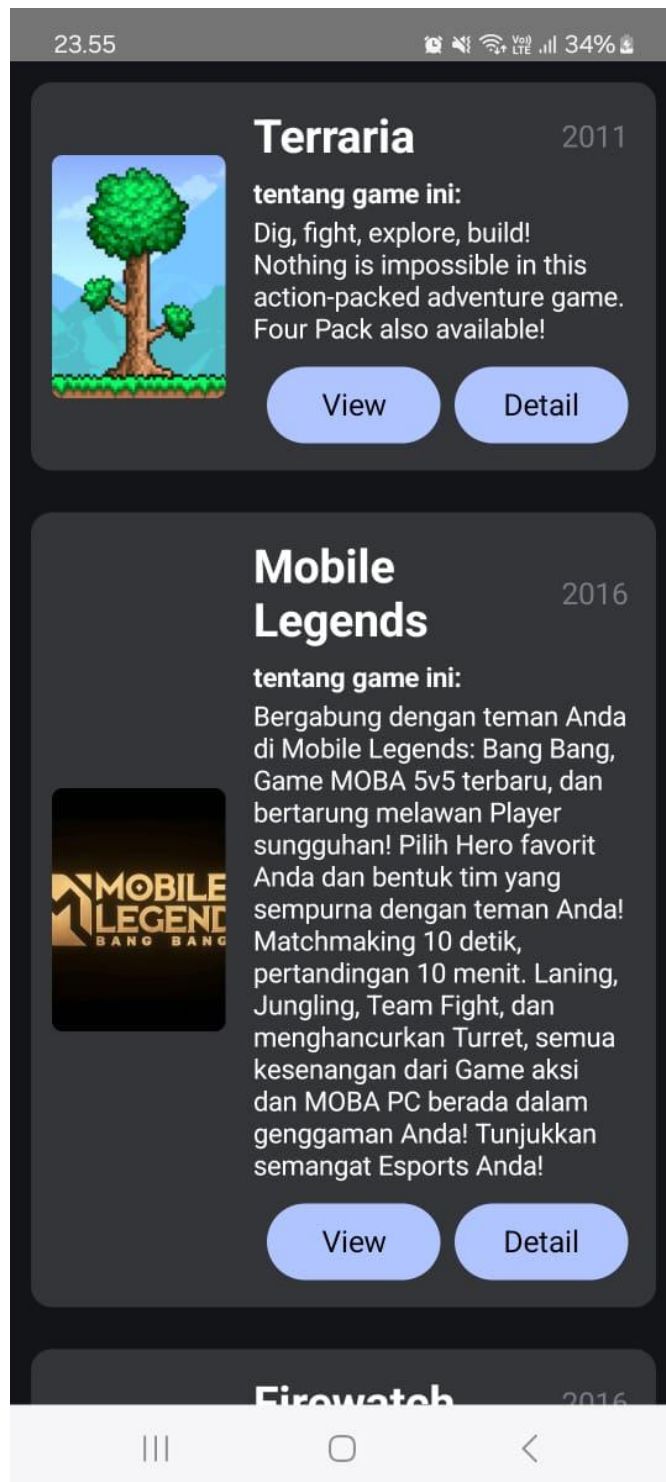
B. Output Program



Gambar 26. Screenshot Hasil Jawaban Soal 1 XML bagian a Modul 4



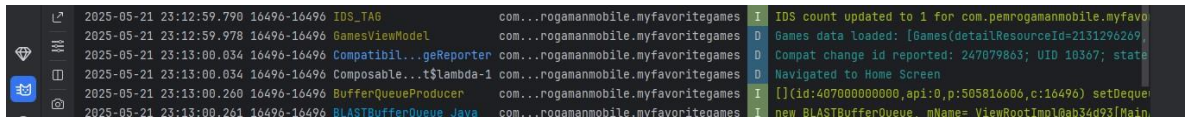
Gambar 27. Screenshot Hasil Jawaban Soal 1 XML bagian b Modul 4



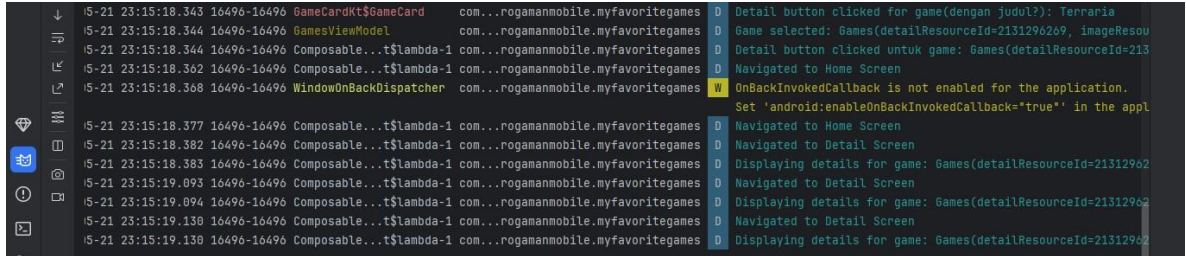
Gambar 28. Screenshot Hasil Jawaban Soal 1 compose bagian a Modul 4



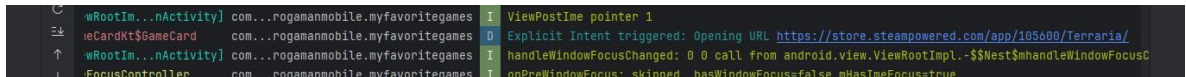
Gambar 29. Screenshot Hasil Jawaban Soal 1 compose bagian b Modul 4



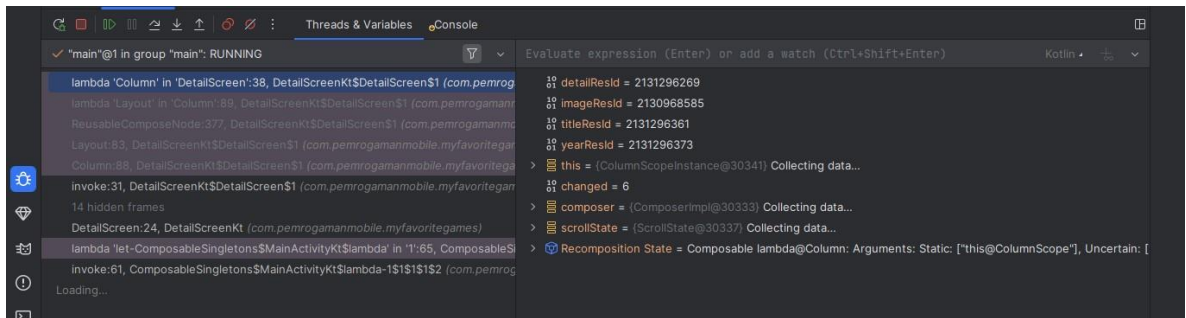
Gambar 30. Screenshot Hasil Jawaban Soal 1 compose Log saat data item masuk ke dalam list Modul 4



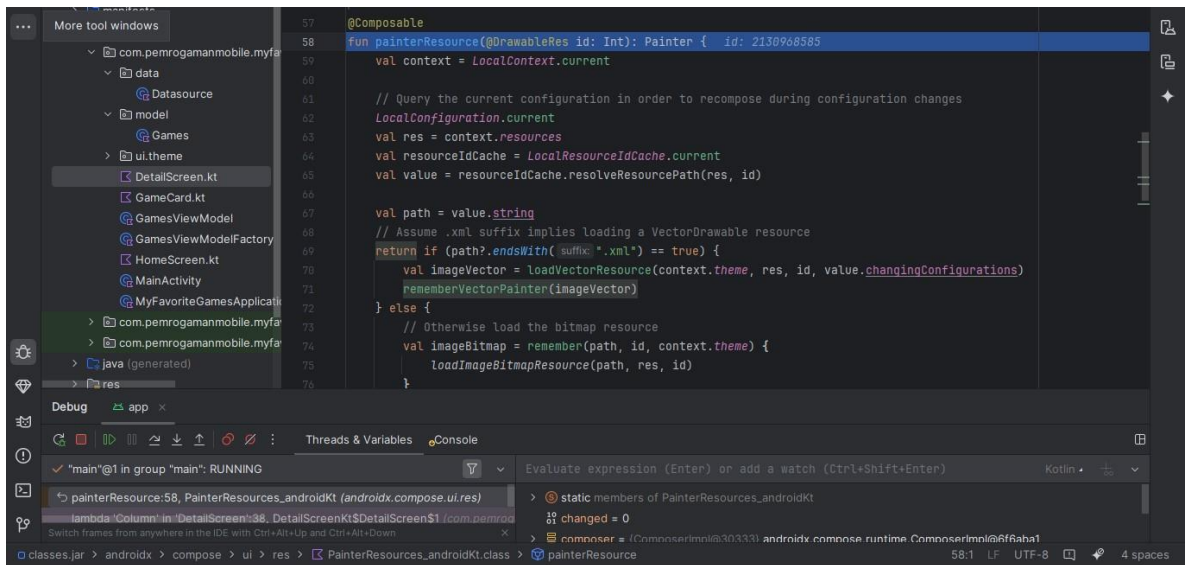
Gambar 31. Screenshot Hasil Jawaban Soal 1 compose Log saat tombol Detail dan Log data dari list yang dipilih ketika berpindah ke halaman Detail Modul 4



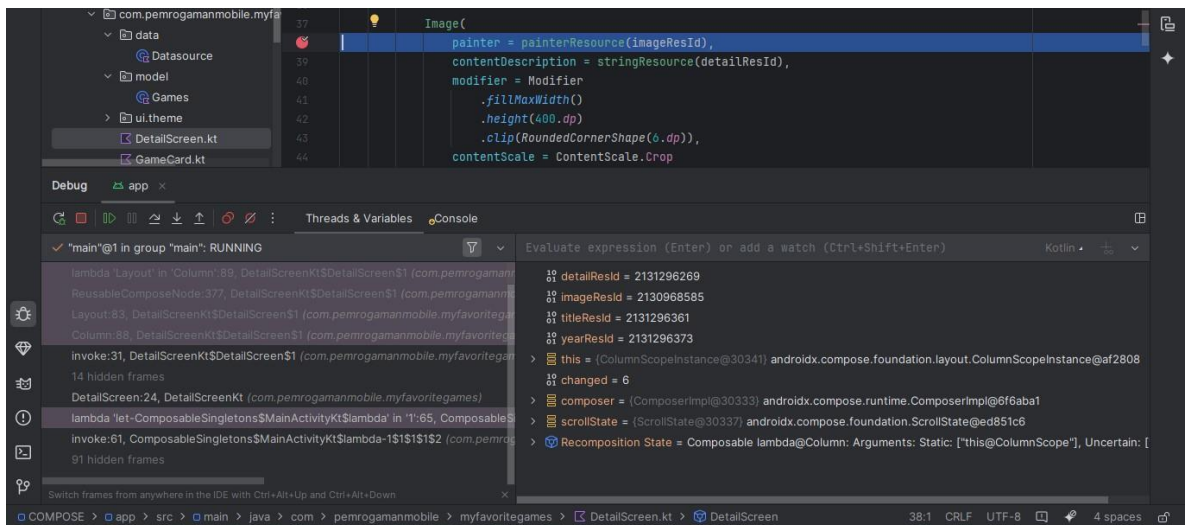
Gambar 32. Screenshot Hasil Jawaban Soal 1 compose Log saat tombol Explicit Intent ditekan Modul 4



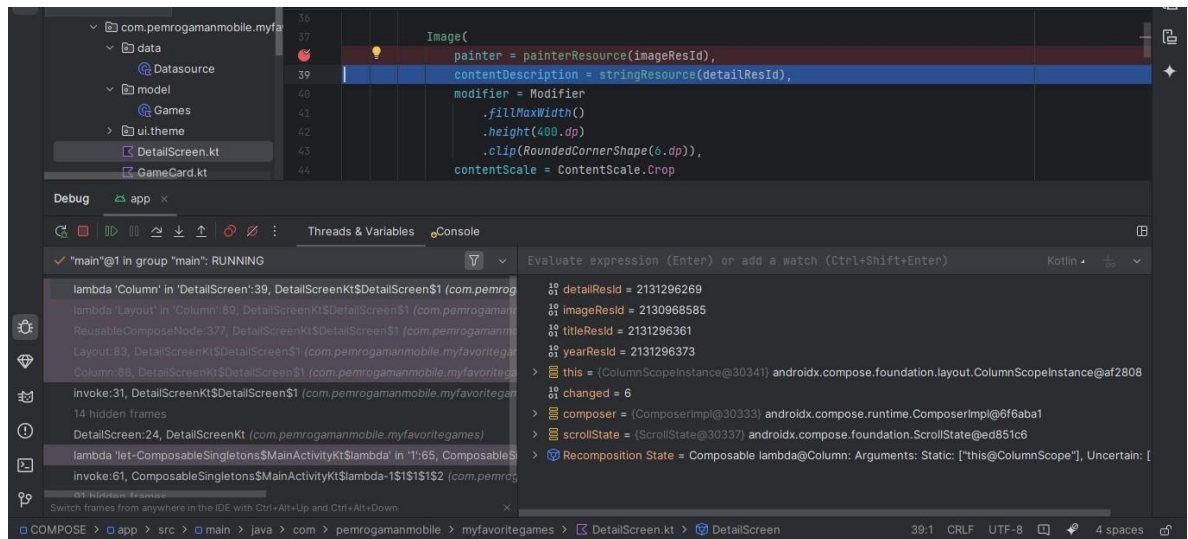
Gambar 33. Screenshot Hasil Jawaban Soal 1 compose debugger Modul 4



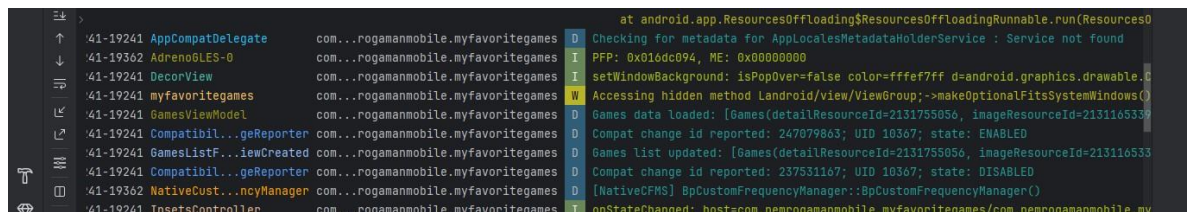
Gambar 34. Screenshot Hasil Jawaban Soal 1 compose debugger step into Modul 4



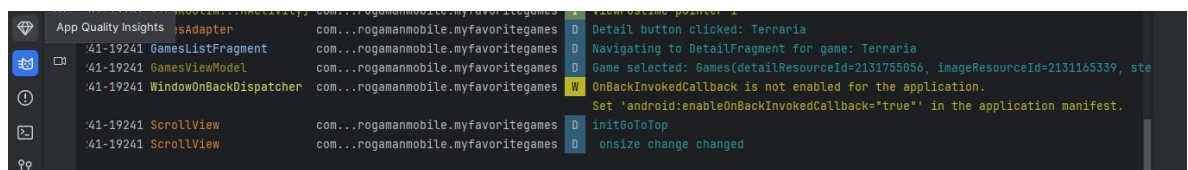
Gambar 35. Screenshot Hasil Jawaban Soal 1 compose debugger step out Modul 4



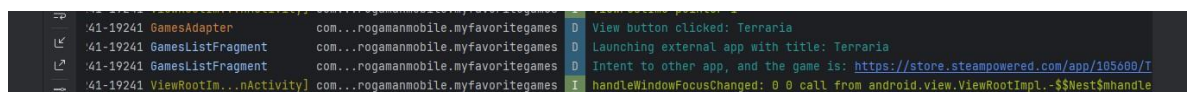
Gambar 36. Screenshot Hasil Jawaban Soal 1 compose debugger step over Modul 4



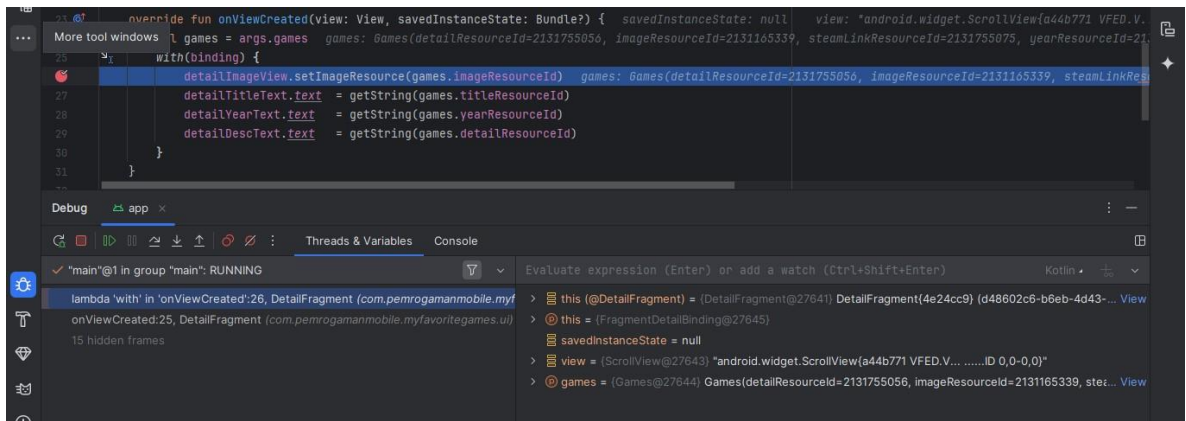
Gambar 37. Screenshot Hasil Jawaban Soal 1 xml Log saat data item masuk ke dalam list Modul 4



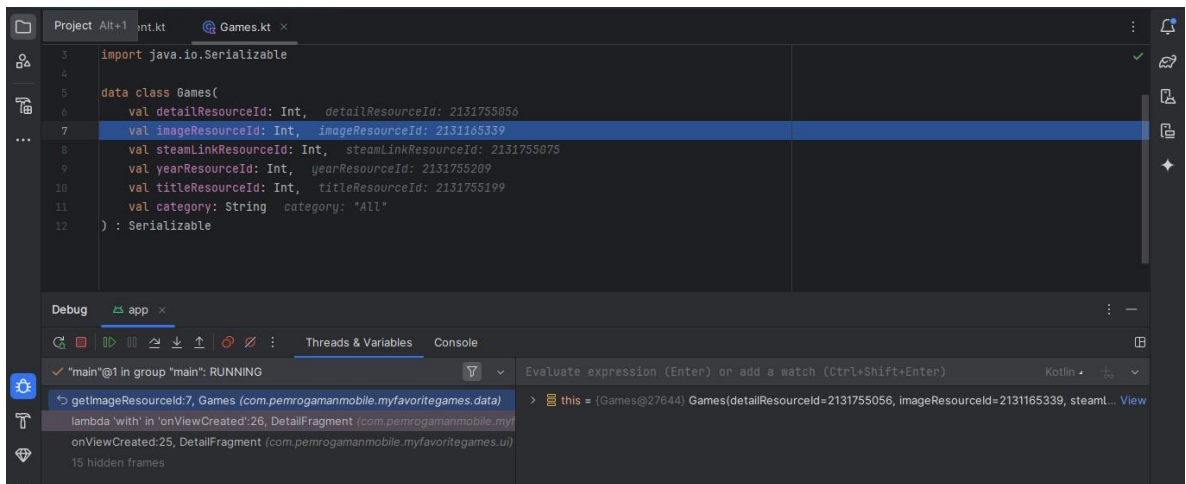
Gambar 38. Screenshot Hasil Jawaban Soal 1 xml Log saat tombol Detail dan Log data dari list yang dipilih ketika berpindah ke halaman Detail Modul 4



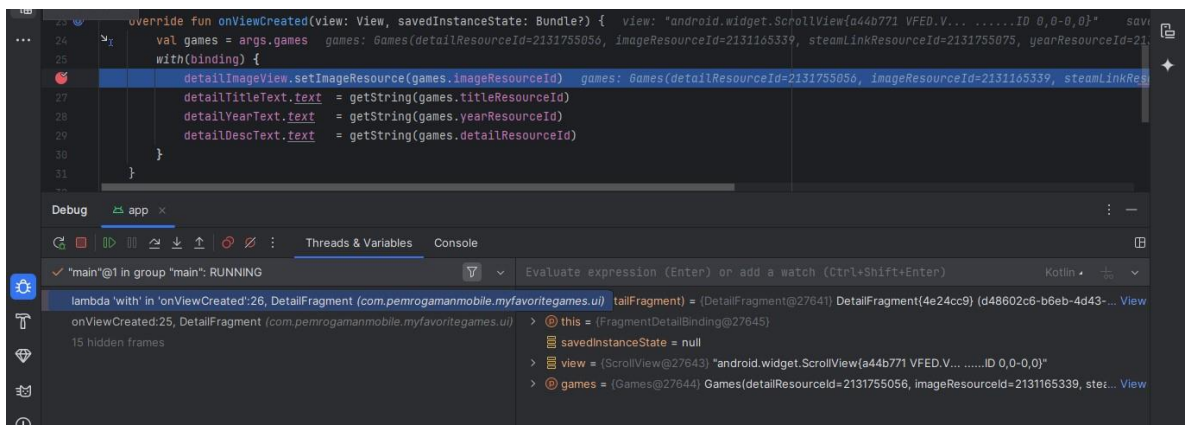
Gambar 39. Screenshot Hasil Jawaban Soal 1 xml Log saat tombol Explicit Intent ditekan Modul 4



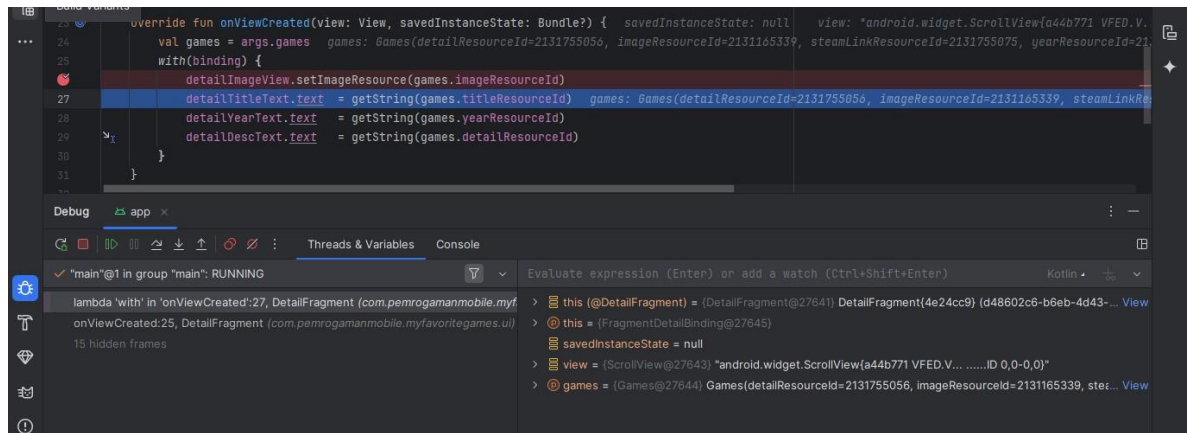
Gambar 40. Screenshot Hasil Jawaban Soal 1 xml debugger Modul 4



Gambar 41. Screenshot Hasil Jawaban Soal 1 xml step into Modul 4



Gambar 42. Screenshot Hasil Jawaban Soal 1 xml step out Modul 4



Gambar 43. Screenshot Hasil Jawaban Soal 1 xml step over Modul 4

C. Pembahasan

XML :

Games.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.data` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file ini agar terorganisir sesuai struktur proyek Android.

Pada baris ke [3] terdapat import `java.io.Serializable` yang berfungsi untuk mengimport kelas `Serializable`, agar objek dari kelas ini bisa dikirim antar activity melalui intent.

Pada baris ke [5] terdapat data class `Games` (yang berfungsi untuk membuat kelas data bernama `Games`, yang secara otomatis menyediakan fungsi seperti `toString()`, `equals()`, `hashCode()`, dan `copy()`).

Pada baris ke [6] sampai [11] berfungsi untuk mendeklarasikan properti dalam kelas `Games`, yaitu `detailResourceId`, `imageResourceId`, `steamLinkResourceId`, `yearResourceId`, `titleResourceId` dan `category`. yang masing-masing bertipe `Int` dan satu `string`, dan berfungsi sebagai ID dari resource yang menyimpan informasi detail teks, gambar game, tautan Steam, tahun rilis, judul game dan katagori yang akan digunakan pada aplikasi.

Pada baris ke [12] terdapat `Serializable` yang berfungsi untuk menjadikan kelas `Games` sebagai objek yang dapat diserialisasi agar bisa dikirim melalui Intent.

Datasource.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.data` yang berfungsi sebagai nama paket dari aplikasi dan tempat menyimpan file ini agar terorganisir sesuai struktur proyek Android.

Pada baris ke [3] terdapat import `com.pemrogamanmobile.myfavoritegames.R` yang berfungsi untuk mengakses resource seperti string dan drawable yang terdapat di dalam file res aplikasi Android ini.

Pada baris ke [5] terdapat object `DataSource` yang berfungsi untuk mendefinisikan object singleton bernama `DataSource`, artinya hanya ada satu instance dari objek ini yang akan digunakan sepanjang aplikasi berjalan.

Pada baris ke [6] terdapat fun loadGames(): List<Games> = listOf(yang berfungsi untuk mendeklarasikan fungsi bernama loadGames yang mengembalikan daftar dari objek Games dalam bentuk List.

Pada baris ke [7] sampai [16] berfungsi untuk mengisi list tersebut dengan sepuluh objek Games yang masing-masing berisi enam parameter, yaitu detail, image, link ke Steam, tahun rilis, judul game dan katagori. Keenam parameter ini merujuk pada ID resource yang disimpan dalam file strings.xml dan drawable di folder res dan ada kalimat “All”. Setiap baris merepresentasikan satu game favorit yang akan ditampilkan di aplikasi.

GamesAdapter.kt

Pada baris ke [1] terdapat deklarasi package com.pemrogamanmobile.myfavoritegames.ui yang menetapkan namespace tempat file ini berada dalam struktur proyek aplikasi Android

Pada baris ke [3] sampai [9] terdapat rangkaian import yang mencakup android annotation SuppressLint untuk menonaktifkan peringatan Lint tertentu, android view LayoutInflater untuk menghasilkan layout dari XML, android view ViewGroup sebagai wadah view, androidx recyclerview widget RecyclerView untuk membuat daftar geser, kelas Games dari modul data sebagai model item, ItemGamesBinding untuk binding layout item, dan Timber untuk logging terstruktur

Pada baris ke [11] sampai [14] terdapat deklarasi kelas GamesAdapter turunan RecyclerView Adapter dengan dua callback yaitu onViewClicked dan onDetailClicked yang akan dipanggil saat tombol view atau detail di dalam setiap item diklik

Pada baris ke [16] terdapat properti items sebagai mutable list of Games yang menyimpan data daftar game untuk ditampilkan

Pada baris ke [18] terdapat inner class ViewHolder yang menyimpan instance ItemGamesBinding dan mewarisi RecyclerView ViewHolder agar setiap item baris dapat diikat ke view binding

Pada baris ke [20] sampai [25] terdapat metode override onCreateViewHolder yang menggunakan LayoutInflater dari context parent untuk menginflate ItemGamesBinding kemudian mengembalikan instance ViewHolder berisi binding tersebut

Pada baris ke [27] sampai [53] terdapat metode override onBindViewHolder yang mengambil item berdasarkan posisi lalu melakukan konfigurasi view dengan menerapkan margin bawah melalui layoutParams, mengisi imageView dan teks title year dan detail dari resource ID di item, serta mendaftarkan listener pada viewButton dan detailButton untuk mencatat log menggunakan Timber dan memanggil callback onViewClicked atau onDetailClicked

Pada baris ke [55] terdapat override `getItemCount` yang mengembalikan ukuran list items sebagai jumlah baris yang akan ditampilkan

Pada baris ke [57] sampai [62] terdapat fungsi `submitList` dengan anotasi `SuppressWarnings` yang membersihkan konten items lalu menambahkan semua elemen `newItems` dan memanggil `notifyDataSetChanged` agar `RecyclerView` melakukan refresh seluruh tampilan

GamesListFragment.kt

Pada baris ke [1] terdapat deklarasi package `com.pemrogramanmobile.myfavoritegames.ui` yang menetapkan namespace tempat file ini berada dalam struktur proyek aplikasi Android

Pada baris ke [3] sampai [19] terdapat rangkaian import yang mencakup `Intent` dan `Uri` untuk navigasi antar aplikasi, `Bundle` untuk menyimpan state, `LayoutInflater`, `View`, dan `ViewGroup` untuk menginflate dan mengelola tampilan, `Fragment` dan `viewModels` dari `Jetpack` untuk lifecycle dan dependency injection, `Lifecycle`, `lifecycleScope`, dan `repeatOnLifecycle` dari `Lifecycle KTX` untuk coroutine yang terikat pada lifecycle, `findNavController` untuk navigasi antar fragment, `GamesViewModel` dan `GamesViewModelFactory` sebagai lapisan presentasi, `FragmentGamesListBinding` untuk view binding, `kotlinx.coroutines.launch` untuk coroutine scope, serta `Timber` untuk logging terstruktur

Pada baris ke [21] terdapat deklarasi class `GamesListFragment` yang merupakan subclass dari `Fragment` dan menjadi container UI daftar game

Pada baris ke [22] terdapat properti `_binding` sebagai nullable `FragmentGamesListBinding` yang akan menahan referensi binding sampai view dihancurkan

Pada baris ke [23] terdapat properti binding yang memanggil `_binding!!` untuk memperoleh instance binding non-null ketika view masih valid

Pada baris ke [25] sampai [27] terdapat inisialisasi `viewModel` dengan delegasi by `viewModels` menggunakan `GamesViewModelFactory` dan parameter “All” untuk memfilter data saat pembuatan `viewModel`

Pada baris ke [29] sampai [33] terdapat override `onCreateView` yang menginflate layout `FragmentGamesListBinding` lewat inflater dan container, menyimpan hasil binding ke `_binding`, lalu mengembalikan root view

Pada baris ke [35] sampai [54] terdapat override `onViewCreated` yang pertama membuat adapter `GamesAdapter` dengan dua lambda callback `onViewClicked` dan `onDetailClicked`; pada `onViewClicked` dibuat `Intent ACTION_VIEW` dengan URI dari resource `steamLink`, dicatat lognya dengan `Timber` lalu dipanggil `startActivity`; pada

onDetailClicked dicatat log, viewModel.onGameClicked dipanggil, kemudian navigasi ke DetailFragment melalui NavController dengan actionListToDetail

Pada baris ke [55] terdapat pengaturan adapter pada recyclerView di binding agar menampilkan daftar game

Pada baris ke [57] sampai [64] dijalankan coroutine di viewLifecycleOwner.lifecycleScope yang menggunakan repeatOnLifecycle pada state STARTED untuk mengumpulkan gameState dari viewModel, mencatat log setiap update, dan memanggil adapter.submitList agar RecyclerView memperbarui tampilannya

Pada baris ke [67] sampai [69] terdapat override onDestroyView yang memanggil super.onDestroyView dan menetapkan _binding ke null untuk mencegah memory leak.

DetailFragment.kt

Pada baris ke [1] terdapat package com.pemrogramanmobile.myfavoritegames.ui yang berfungsi sebagai nama paket dari file ini dan menunjukkan bahwa file ini berada di dalam folder ui.

Pada baris ke [3] sampai [9], berfungsi untuk mengimport beberapa kelas yang digunakan, yaitu Bundle, LayoutInflater, View, ViewGroup, Fragment, navArgs, dan FragmentDetailBinding. Import ini mendukung pembuatan tampilan fragment, navigasi menggunakan Safe Args, dan binding layout XML ke dalam kode.

Pada baris ke [11] terdapat class DetailFragment : Fragment() yang berfungsi untuk mendeklarasikan kelas DetailFragment sebagai subclass dari Fragment, yang merupakan bagian dari tampilan aplikasi.

Pada baris ke [12] terdapat private var _binding: FragmentDetailBinding? = null yang berfungsi untuk mendeklarasikan variabel _binding sebagai objek binding yang akan menghubungkan kode dengan layout fragment. Nilai awalnya adalah null untuk menghindari memory leak.

Pada baris ke [13] terdapat private val binding get() = _binding!! yang berfungsi untuk membuat properti binding yang pasti tidak null saat digunakan, menggunakan not-null assertion (!!).

Pada baris ke [15] terdapat private val args: DetailFragmentArgs by navArgs() yang berfungsi untuk mengambil data dari argument navigasi menggunakan Safe Args, yaitu data Games yang dikirim dari fragment sebelumnya.

Pada baris ke [17] sampai [21] terdapat fungsi onCreateView yang berfungsi untuk meng-inflate layout fragment_detail.xml menggunakan View Binding dan mengembalikan tampilan utama dari fragment.

Pada baris ke [23] terdapat fungsi onViewCreated yang berfungsi untuk mengatur tampilan fragment setelah layout selesai dibuat.

Pada baris ke [24] terdapat val Games = args.Games yang berfungsi untuk mengambil objek Games dari argumen Safe Args yang dikirim melalui navigasi.

Pada baris ke [25] sampai [30] berfungsi untuk mengatur konten layout berdasarkan data dari objek Games, seperti gambar, judul, tahun rilis, dan deskripsi dengan menggunakan getString() berdasarkan ID resource masing-masing.

Pada baris ke [33] sampai [36] terdapat fungsi onDestroyView yang berfungsi untuk menghapus referensi binding saat tampilan fragment dihancurkan, untuk mencegah memory leak dengan mengatur _binding = null.

MainActivity.kt:

Pada baris ke [1] terdapat package com.pemrogamanmobile.myfavoritegames yang berfungsi sebagai nama paket utama dari aplikasi ini dan menunjukkan bahwa file ini berada di bagian utama dari struktur folder aplikasi.

Pada baris ke [3] sampai [5], berfungsi untuk mengimpor beberapa kelas yang akan digunakan, yaitu Bundle dari Android untuk menyimpan data saat pembuatan activity, AppCompatActivity sebagai superclass activity, dan ActivityMainBinding untuk menghubungkan layout XML activity ke dalam kode melalui View Binding.

Pada baris ke [7] terdapat class MainActivity : AppCompatActivity() yang berfungsi untuk mendeklarasikan MainActivity sebagai activity utama aplikasi dan mewarisi perilaku dari AppCompatActivity.

Pada baris ke [8] terdapat private lateinit var binding: ActivityMainBinding yang berfungsi untuk mendeklarasikan variabel binding yang akan diinisialisasi nanti dan digunakan untuk mengakses komponen dalam layout activity melalui View Binding.

Pada baris ke [10] sampai [14] terdapat fungsi onCreate yang merupakan lifecycle method yang dijalankan saat activity pertama kali dibuat.

Pada baris ke [11] terdapat super.onCreate(savedInstanceState) yang berfungsi untuk memanggil implementasi onCreate dari superclass (AppCompatActivity).

Pada baris ke [12] terdapat binding = ActivityMainBinding.inflate(layoutInflater) yang berfungsi untuk menginisialisasi objek binding menggunakan layout inflater agar layout XML bisa digunakan dalam kode.

Pada baris ke [13] terdapat setContentView(binding.root) yang berfungsi untuk menampilkan layout utama activity ke layar dengan menggunakan root dari objek binding.

GamesViewModel.kt

Pada baris ke [1] terdapat deklarasi package com.pemrogamanmobile.myfavoritegames yang menetapkan namespace tempat file ini berada dalam struktur proyek aplikasi Android

Pada baris ke [2] sampai [9] terdapat rangkaian import yang mencakup ViewModel dari Jetpack lifecycle DataSource dan Games dari modul data MutableStateFlow StateFlow dan fungsi update dari kotlinox coroutines flow serta Timber untuk logging terstruktur

Pada baris ke [11] sampai [13] terdapat deklarasi class `GamesViewModel` dengan parameter privat `gameCategory` bertipe `String` dan pewarisan dari `ViewModel` sebagai lapisan presentasi untuk logika UI

Pada baris ke [16] sampai [23] terdapat deklarasi properti `_gamesState` sebagai `MutableStateFlow` yang diinisialisasi dengan daftar game hasil filter category sama dengan `gameCategory` serta `gamesState` sebagai `StateFlow` publik untuk observasi data dan properti `_selectedGame` sebagai `MutableStateFlow` nullable `Games` diinisialisasi null bersama `selectedGame` sebagai `StateFlow` publik untuk menyimpan pilihan pengguna

Pada baris ke [25] sampai [28] terdapat blok init yang mencatat log data game yang berhasil dimuat menggunakan `Timber` dengan nilai `_gamesState` value

Pada baris ke [31] sampai [34] terdapat fungsi `onGameClicked` dengan parameter game yang memperbarui nilai `_selectedGame` menggunakan `update` dan mencatat log game terpilih melalui `Timber`

GamesViewModelFactory.kt

Pada baris ke [1] terdapat deklarasi package `com.pemrogramanmobile.myfavoritegames` yang menetapkan namespace tempat file ini berada dalam struktur proyek aplikasi Android

Pada baris ke [3] sampai [4] terdapat import `ViewModel` dan `ViewModelProvider` dari `Jetpack Lifecycle` yang memungkinkan pembuatan dan penyediaan instance `ViewModel` secara dinamis

Pada baris ke [6] sampai [8] terdapat deklarasi class `GamesViewModelFactory` dengan parameter privat `gameCategory` bertipe `String` yang mengimplementasi `ViewModelProvider Factory` sebagai mekanisme injeksi dependensi untuk `GamesViewModel`

Pada baris ke [9] terdapat override fungsi `create` generik dengan parameter `modelClass` bertipe `Class T` yang mengembalikan instance `ViewModel` sesuai kelas yang diminta

Pada baris ke [10] sampai [12] terdapat blok kondisi yang memeriksa apakah `modelClass` dapat diassignable dari `GamesViewModel` kemudian mereturn instance `GamesViewModel` dengan `gameCategory` sambil menonaktifkan peringatan `unchecked cast` melalui anotasi `Suppress`

Pada baris ke [14] terdapat perintah `throw IllegalArgumentException` dengan pesan `Unknown ViewModel class` yang akan dieksekusi jika kelas `ViewModel` yang diminta tidak dikenali oleh factory

MyAplication.kt

Pada baris ke [1] terdapat deklarasi package com.pemrogramanmobile.myfavoritegames yang menetapkan namespace tempat file ini berada dalam struktur proyek aplikasi Android

Pada baris ke [3] sampai [5] terdapat import android app Application import Timber log pustaka untuk logging terstruktur dan import BuildConfig yang berisi informasi build agar kelas ini dapat menggunakan flag DEBUG

Pada baris ke [7] sampai [16] terdapat deklarasi kelas MyApplication turunan Application dengan override fungsi onCreate yang memanggil super onCreate lalu memeriksa jika BuildConfig.DEBUG bernilai true maka melakukan inisialisasi Timber dengan DebugTree agar log hanya muncul dalam mode debug

activity_main.xml:

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen `FragmentContainerView` yang berasal dari pustaka `androidx.fragment.app`. Elemen ini digunakan sebagai wadah untuk menampilkan fragmen.

Pada baris ke [3], terdapat deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android`, yang digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [4], terdapat deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto`, yang memungkinkan penggunaan atribut kustom dari pustaka `AndroidX`.

Pada baris ke [5], terdapat atribut `android:id="@+id/nav_host_fragment"`, yang menetapkan ID unik untuk elemen `FragmentContainerView` ini sebagai `nav_host_fragment`.

Pada baris ke [6], atribut `android:name="androidx.navigation.fragment.NavHostFragment"` mendefinisikan bahwa elemen ini adalah `NavHostFragment`, yang merupakan komponen utama untuk navigasi menggunakan `Jetpack Navigation`.

Pada baris ke [7], atribut `android:layout_width="match_parent"` digunakan untuk mengatur lebar elemen agar sesuai dengan lebar layar perangkat.

Pada baris ke [8], atribut `android:layout_height="match_parent"` digunakan untuk mengatur tinggi elemen agar sesuai dengan tinggi layar perangkat.

Pada baris ke [9], atribut `app:navGraph="@navigation/nav_graph"` menghubungkan elemen ini dengan file navigasi bernama `nav_graph`, yang berisi konfigurasi alur navigasi aplikasi.

Pada baris ke [10], atribut `app:defaultNavHost="true"` menetapkan elemen ini sebagai host navigasi utama, sehingga mampu menangani tindakan Back pada perangkat secara otomatis.

fragment_Games_list.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `ConstraintLayout` dari pustaka `androidx.constraintlayout.widget`. Elemen ini digunakan sebagai layout utama yang memungkinkan penempatan elemen-elemen anak dengan aturan constraint.

Pada baris ke [3], terdapat deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android`, yang digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [4], terdapat deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto`, yang memungkinkan penggunaan atribut kustom dari pustaka `AndroidX`.

Pada baris ke [5], atribut `android:background="#000000"` digunakan untuk mengatur latar belakang layout dengan warna hitam.

Pada baris ke [6], atribut `android:layout_width="match_parent"` digunakan untuk mengatur lebar layout agar sesuai dengan lebar layar perangkat.

Pada baris ke [7], atribut `android:layout_height="match_parent"` digunakan untuk mengatur tinggi layout agar sesuai dengan tinggi layar perangkat.

Pada baris ke [9], terdapat elemen `RecyclerView` dari pustaka `androidx.recyclerview.widget`. Elemen ini digunakan untuk menampilkan daftar data dengan performa tinggi.

Pada baris ke [10], atribut `android:id="@+id/recyclerView"` menetapkan ID unik untuk elemen ini sebagai `recyclerView`.

Pada baris ke [11], atribut `android:contentDescription="@string/Games_list_desc"` memberikan deskripsi konten yang bersifat aksesibilitas untuk `RecyclerView`, menggunakan nilai dari sumber string bernama `Games_list_desc`.

Pada baris ke [12] sampai [13], terdapat atribut `android:layout_width="0dp"` dan `android:layout_height="0dp"` mengatur lebar dan tinggi elemen ini agar dapat diatur sepenuhnya oleh constraint.

Pada baris ke [14], atribut `android:padding="8dp"` menambahkan jarak dalam sebesar 8dp di seluruh sisi elemen ini.

Pada baris ke [15], atribut `app:layoutManager="LinearLayoutManager"` menetapkan tata letak RecyclerView menggunakan `LinearLayoutManager`, yang menyusun elemen secara vertikal atau horizontal.

Pada baris ke [15] hingga [18], terdapat atribut constraint yang menghubungkan sisi atas, bawah, kiri, dan kanan RecyclerView dengan sisi parent, sehingga elemen ini berada di tengah layout secara proporsional.

fragment_detail.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `ScrollView` yang berasal dari pustaka bawaan Android. Elemen ini digunakan sebagai wadah untuk memungkinkan konten di dalamnya dapat digulir secara vertikal. deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android` digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [3], deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto` memungkinkan penggunaan atribut kustom dari pustaka `AndroidX` atau `Material`.

Pada baris ke [4], atribut `android:background="#131418"` mengatur latar belakang elemen dengan warna abu-abu gelap (#131418).

Pada baris ke [5] dan [6], atribut `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan untuk mengatur elemen agar mengisi seluruh lebar dan tinggi layar perangkat.

Pada baris ke [8] sampai [9], terdapat elemen `LinearLayout` dengan orientasi vertikal yang berfungsi sebagai kontainer utama untuk menampung elemen-elemen lainnya.

Pada baris ke [10] sampai [12], atribut `android:padding="16dp"` menambahkan jarak sebesar 16dp di semua sisi elemen ini untuk memberikan ruang antara konten dan tepi layar.

Pada baris ke [14] hingga [19], terdapat elemen `ShapeableImageView` dari pustaka `Material Components`. Elemen ini menampilkan gambar dengan ID `detailImageView`, lebar penuh, tinggi 400dp, skala `centerCrop`, dan sudut membulat dengan gaya yang diatur oleh `@style/ShapeAppearance.App.Rounded6dp`.

Pada baris ke [21], terdapat `LinearLayout` horizontal yang digunakan untuk menempatkan teks secara berdampingan.

Pada baris ke [25], atribut `android:gravity="center_vertical"` digunakan untuk menyelaraskan konten di dalam elemen secara vertikal di tengah.

Pada baris ke [26], atribut `android:layout_marginTop="12dp"` menambahkan jarak sebesar 12dp antara elemen ini dan elemen di atasnya.

Pada baris ke [28] sampai [35], terdapat elemen `TextView` dengan ID `detailTitleText`, yang digunakan untuk menampilkan judul. Elemen ini menggunakan warna teks putih, ukuran teks 26sp, dan gaya teks tebal (bold).

Pada baris ke [37] hingga [42], terdapat elemen `TextView` dengan ID `detailYearText`, yang digunakan untuk menampilkan tahun. Elemen ini menggunakan warna teks abu-abu terang, ukuran teks 14sp, dan tinggi sesuai konten.

Pada baris ke [45] hingga [52], terdapat elemen `TextView` tanpa ID yang menampilkan teks “tentang game ini:” sebagai label. Elemen ini menggunakan warna teks putih, ukuran teks 14sp, dan gaya teks tebal.

Pada baris ke [54] hingga [60], terdapat elemen `TextView` dengan ID `detailDescText`, yang digunakan untuk menampilkan deskripsi detail. Elemen ini menggunakan warna teks putih, ukuran teks 14sp, dan margin atas sebesar 4dp untuk memberi jarak dari elemen di atasnya.

item_Games.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `CardView` dari pustaka `androidx.cardview.widget`. Elemen ini digunakan untuk menampilkan konten dalam bentuk kartu dengan sudut membulat dan latar belakang yang dapat disesuaikan.

Pada baris ke [3], deklarasi namespace `xmlns:android` dengan nilai `http://schemas.android.com/apk/res/android` digunakan untuk mendefinisikan atribut-atribut khusus Android pada elemen-elemen XML.

Pada baris ke [4], deklarasi namespace tambahan `xmlns:app` dengan nilai `http://schemas.android.com/apk/res-auto` memungkinkan penggunaan atribut kustom dari pustaka AndroidX.

Pada baris ke [5] hingga [6], atribut `android:layout_width="match_parent"` dan `android:layout_height="wrap_content"` digunakan untuk mengatur elemen agar memenuhi lebar layar perangkat dengan tinggi yang menyesuaikan kontennya.

Pada baris ke [7], atribut `android:layout_margin="4dp"` menambahkan jarak luar sebesar 4dp di seluruh sisi elemen ini.

Pada baris ke [8], atribut `app:cardCornerRadius="6dp"` digunakan untuk mengatur radius sudut kartu menjadi 6dp agar tampak membulat.

Pada baris ke [9], atribut `app:cardBackgroundColor="#343539"` mengatur warna latar belakang kartu dengan warna abu-abu gelap (#343539).

Pada baris ke [11], terdapat elemen `LinearLayout` dengan orientasi horizontal yang digunakan untuk menata elemen-elemen anak secara berdampingan.

Pada baris ke [15], atribut `android:padding="8dp"` menambahkan jarak dalam sebesar 8dp di seluruh sisi elemen ini.

Pada baris ke [18] hingga [25], terdapat elemen `ShapeableImageView` dari pustaka Material Components, yang menampilkan gambar dengan ID `imageView`. Elemen ini memiliki lebar 100dp, tinggi 140dp, margin kanan 12dp, skala gambar `centerCrop`, dan sudut membulat sesuai dengan gaya `@style/ShapeAppearance.App.Rounded6dp`.

Pada baris ke [27] sampai [37], terdapat `LinearLayout` vertikal yang digunakan untuk menata elemen-elemen teks dan tombol secara bertumpuk di dalamnya.

Pada baris ke [29], atribut `android:layout_weight="1"` digunakan untuk membuat elemen ini mengisi ruang yang tersisa dalam `LinearLayout` horizontal induknya.

Pada baris ke [39] hingga [53], terdapat elemen `TextView` untuk menampilkan judul dan tahun. Teks judul menggunakan ID `titleText`, warna putih, ukuran teks 20sp, dan gaya tebal, sedangkan teks tahun menggunakan ID `yearText`, warna abu-abu terang, dan ukuran teks 14sp.

Pada baris ke [56] hingga [63], terdapat elemen `TextView` tanpa ID yang menampilkan teks "tentang game ini:" sebagai label dengan warna putih, ukuran teks 14sp, dan gaya tebal.

Pada baris ke [65] hingga [71], terdapat elemen TextView dengan ID detailText yang digunakan untuk menampilkan deskripsi tambahan dengan warna putih, ukuran teks 14sp, dan margin atas 4dp.

Pada baris ke [73] hingga [78], terdapat LinearLayout horizontal yang digunakan untuk menempatkan dua tombol di sisi kanan bawah kartu.

Pada baris ke [80] hingga [86], terdapat tombol pertama dengan ID viewButton yang memiliki teks "View", lebar 100dp, tinggi 44dp, warna latar biru terang (#b0c4ff), dan warna teks hitam.

Pada baris ke [88], terdapat elemen Space dengan lebar 8dp yang digunakan untuk memberi jarak antara kedua tombol.

Pada baris ke [90] hingga [96], terdapat tombol kedua dengan ID detailButton yang memiliki teks "Detail", lebar dan tinggi yang sama seperti tombol pertama, warna latar biru terang, dan warna teks hitam.

strings.xml

Pada baris ke [1], terdapat elemen root <resources> yang digunakan untuk menampung semua sumber daya string dalam aplikasi Android.

Pada baris ke [2], terdapat elemen <string> dengan atribut name="app_name" yang menyimpan nama aplikasi, yaitu "My favorite games".

Pada baris ke [4], terdapat elemen <string> dengan atribut name="Games_list_desc" yang berfungsi sebagai deskripsi aksesibilitas untuk daftar permainan favorit, yaitu "List of favorite games".

Pada baris ke [6] sampai [15], terdapat sepuluh elemen <string> dengan atribut name="titleX" yang masing-masing menyimpan judul permainan favorit berdasarkan nomor urut, seperti "Terraria", "Mobile Legends", dan lainnya.

Pada baris ke [17] sampai [26], terdapat sepuluh elemen <string> dengan atribut name="yearX" yang masing-masing menyimpan tahun rilis permainan berdasarkan nomor urut, seperti "2011" untuk year1 dan "2020" untuk year6.

Pada baris ke [28] sampai [37], terdapat sepuluh elemen <string> dengan atribut name="detailX" yang masing-masing menyimpan deskripsi detail permainan berdasarkan nomor urut. Contohnya, detail1 berisi deskripsi "Terraria", dan detail6 berisi deskripsi "Genshin Impact".

Pada baris ke [39] sampai [48], terdapat sepuluh elemen <string> dengan atribut name="linkX" yang masing-masing menyimpan URL terkait permainan berdasarkan

nomor urut. Contohnya, link1 berisi URL ke halaman Steam "Terraria", dan link6 berisi URL ke halaman Google Play "Genshin Impact".

styles.xml

Pada baris ke [1], terdapat deklarasi `<?xml version="1.0" encoding="utf-8"?>` yang berfungsi untuk menyatakan versi XML yang digunakan serta karakter encoding yang dipakai, yaitu UTF-8.

Pada baris ke [2], terdapat elemen root `<resources>` yang digunakan untuk menampung semua sumber daya (style) dalam aplikasi Android.

Pada baris ke [3], terdapat elemen `<style>` dengan atribut `name="ShapeAppearance.App.Rounded6dp"` yang mendefinisikan gaya bernama ShapeAppearance.App.Rounded6dp. Gaya ini digunakan untuk mengatur tampilan sudut elemen tertentu menjadi membulat dengan radius 6dp. atribut `parent=""` tidak memiliki nilai tertentu, menunjukkan bahwa gaya ini tidak mewarisi properti dari gaya lain.

Pada baris ke [4], terdapat elemen `<item>` dengan atribut `name="cornerFamily"` dan nilai `rounded`, yang menentukan bahwa elemen-elemen yang menggunakan gaya ini akan memiliki sudut membulat.

Pada baris ke [5], terdapat elemen `<item>` dengan atribut `name="cornerSize"` dan nilai `6dp`, yang menentukan ukuran radius sudut membulat sebesar 6dp.

Pada baris ke [6], elemen `</style>` menutup definisi gaya ShapeAppearance.App.Rounded6dp.

Pada baris ke [7], elemen `</resources>` menutup elemen root `<resources>`.

COMPOSE : Datasource.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.data` yang berfungsi sebagai deklarasi nama paket tempat file ini disimpan dalam struktur proyek aplikasi Android.

Pada baris ke [2] (kosong) hanya berfungsi sebagai pemisah visual agar kode lebih mudah dibaca.

Pada baris ke [3] terdapat `import com.pemrogamanmobile.myfavoritegames.R` yang berfungsi untuk mengimpor file R, yaitu file auto-generated yang berisi referensi ke semua sumber daya (seperti string, drawable, dan layout) dalam aplikasi.

Pada baris ke [4] terdapat import `com.pemrogamanmobile.myfavoritegames.model.Games` yang berfungsi untuk mengimpor kelas Games dari package model, yaitu model data yang digunakan untuk merepresentasikan setiap item game favorit.

Pada baris ke [5] (kosong) kembali sebagai pemisah visual agar struktur kode lebih rapi.

Pada baris ke [6] terdapat deklarasi class `Datasource` { yang berfungsi sebagai sumber data untuk aplikasi ini.

Pada baris ke [7] terdapat fungsi `fun loadGames(): List<Games>` { yang akan mengembalikan sebuah `List<Games>`, yaitu daftar data yang berisi objek-objek Games.

Pada baris ke [8] terdapat `return listOf(` yang berfungsi mengembalikan hasil berupa daftar (`List`) objek Games.

Pada baris ke [9] hingga [18] setiap item Games diisi menggunakan referensi dari resource `R.string`, `R.drawable`, dan `R.string` lainnya, serta satu argumen literal "All".

Games.kt:

Pada baris ke [1] terdapat package `com.pemrogamanmobile.myfavoritegames.model` yang berfungsi sebagai deklarasi nama paket tempat file ini disimpan dalam struktur proyek aplikasi Android.

Pada baris ke [3] terdapat import `androidx.annotation.DrawableRes` yang berfungsi untuk mengimpor anotasi `@DrawableRes`, yaitu anotasi yang menandai bahwa suatu properti atau parameter harus merujuk pada ID resource drawable.

Pada baris ke [4] terdapat import `androidx.annotation.StringRes` yang berfungsi untuk mengimpor anotasi `@StringRes`, yaitu anotasi yang menandai bahwa suatu properti atau parameter harus merujuk pada ID resource string.

Pada baris ke [6] terdapat deklarasi data class `Games` (yang berfungsi untuk mendefinisikan kelas data Games, yaitu model data yang mewakili setiap entri game favorit beserta atribut-atributnya.

Pada baris ke [7] terdapat `@StringRes val detailResourceId: Int`, yang berfungsi sebagai properti `detailResourceId` bertipe `Int` dan dianotasi `@StringRes`, menunjukkan bahwa nilai integer ini mengacu pada ID resource string untuk deskripsi detail game.

Pada baris ke [8] terdapat `@DrawableRes val imageResourceId: Int`, yang berfungsi sebagai properti `imageResourceId` bertipe `Int` dan dianotasi `@DrawableRes`, menunjukkan bahwa nilai integer ini mengacu pada ID resource drawable untuk gambar cover game.

Pada baris ke [9] terdapat `@StringRes val linkResourceId: Int`, yang berfungsi sebagai properti `linkResourceId` bertipe `Int` dan dianotasi `@StringRes`, menunjukkan bahwa nilai integer ini mengacu pada ID resource string untuk menyimpan tautan terkait game.

Pada baris ke [10] terdapat `@StringRes val yearResourceId: Int`, yang berfungsi sebagai properti `yearResourceId` bertipe `Int` dan dianotasi `@StringRes`, menunjukkan bahwa nilai integer ini mengacu pada ID resource string untuk menyimpan informasi tahun rilis game.

Pada baris ke [11] terdapat `@StringRes val titleResourceId: Int`, yang berfungsi sebagai properti `titleResourceId` bertipe `Int` dan dianotasi `@StringRes`, menunjukkan bahwa nilai integer ini mengacu pada ID resource string untuk judul game.

Pada baris ke [12] terdapat `val category: String` yang berfungsi sebagai properti `category` bertipe `String`, yaitu kategori atau tag yang menggambarkan jenis atau pengelompokan game (misalnya "All", "Action", dsb.).

GamesCard.kt

Pada baris ke [1] terdapat `package com.pemrogramanmobile.myfavoritegames` yang berfungsi sebagai deklarasi nama paket tempat file ini disimpan dalam struktur proyek aplikasi Android.

Pada baris ke [3] sampai [23] terdapat rangkaian `import` yang berfungsi mengambil berbagai kelas dan fungsi dari Android SDK, Jetpack Compose, serta library tambahan (seperti Timber dan LocalContext) agar komponen UI, pengelolaan resource, penanganan intent, dan pencatatan log dapat digunakan di dalam fungsi `GameCard`.

Pada baris ke [25] terdapat anotasi `@Composable` yang menandai bahwa fungsi berikutnya merupakan blok pembentuk UI di Jetpack Compose.

Pada baris ke [26] sampai [30] didefinisikan fungsi `GameCard` yang menerima tiga parameter: objek `games` (tipe `Games`) sebagai data game yang akan ditampilkan, sebuah modifier (dengan nilai default `Modifier`) untuk penyesuaian tampilan dari luar, dan `onDetailClick` (sebuah lambda) yang akan dipanggil saat tombol "Detail" ditekan.

Pada baris ke [31] terdapat inisialisasi `val context = LocalContext.current` yang mengambil konteks Android saat ini, digunakan untuk memanggil resource (seperti string dan drawable) serta memulai intent.

Pada baris ke [33] sampai [36] terdapat deklarasi `Card` yang menjadi wadah utama kartu; di sini diterapkan `padding 12 dp` dan warna latar abu-abu gelap (kode `0xFF343539`).

Pada baris ke [37] sampai [49] terdapat Row pertama yang membagi area horizontal menjadi dua bagian: bagian gambar (Image) di sebelah kiri, diatur agar memiliki padding kiri 12 dp, lebar 100 dp, tinggi 140 dp, dan sudut membulat 6 dp; selain itu, gambar diambil dari games.imageResourceId dan di-crop agar memenuhi kotak tanpa merusak rasio aslinya; bagian konten di sebelah kanan, berisi teks dan tombol, diatur agar mengisi lebar yang tersisa secara vertikal.

Pada baris ke [51] sampai [56] terdapat Column yang menumpuk elemen teks vertikal: pada baris ke [57] sampai [77] terdapat Row khusus judul dan tahun rilis, judul diambil dari games.titleResourceId dengan ukuran font 26 sp, tebal, dan warna putih, diberi bobot agar menempati ruang maksimal; tahun rilis diambil dari games.yearResourceId dengan ukuran font 16 sp dan warna abu-abu terang, diselaraskan ke kanan serta vertikal di tengah baris, dan pada baris ke [81] sampai [91] terdapat Column yang menampilkan label “tentang game ini:” dengan gaya font 14 sp tebal dan warna putih, diikuti deskripsi detail game yang diambil dari games.detailResourceId dengan gaya font 14 sp dan warna putih, diberi jarak vertikal 4 dp di atasnya.

Pada baris ke [95] sampai [125] terdapat Row yang menata dua tombol di ujung kanan: tombol “View” (baris ke [100] sampai [111]) akan mengambil URL dari games.linkResourceId, membuat intent ACTION_VIEW untuk membuka browser, mencatat log menggunakan Timber, dan memulai aktivitas; tombol ini memiliki lebar 100 dp, tinggi 44 dp, latar biru muda (0xFFb0c4ff), dan teks “View” berukuran 16 sp berwarna hitam; tombol “Detail” (baris ke [115] sampai [124]) mencatat log dengan Timber berisi judul game, kemudian memanggil callback onDetailClick dengan objek games; ukuran dan warna tombol sama seperti tombol “View”, dengan teks “Detail” berukuran 16 sp dan warna hitam.

Pada baris ke [125] sampai [129] seluruh blok tampilan (Row, Card, dan fungsi GameCard) ditutup, menandakan akhir definisi UI untuk satu kartu game.

DetailScreen.kt

Pada baris ke [1] terdapat package com.pemrogramanmobile.myfavoritegames yang berfungsi sebagai penamaan paket dari aplikasi dan sebagai tempat penyimpanan file DetailScreen.kt.

Pada baris ke [3] sampai [20], berfungsi untuk mengimpor berbagai komponen penting dari Jetpack Compose seperti Image, Column, Row, Text, Surface, Modifier, Color, dan lainnya yang digunakan untuk membuat tampilan layar detail game.

Pada baris ke [22] terdapat anotasi @Composable yang menunjukkan bahwa fungsi DetailScreen merupakan fungsi composable, yaitu fungsi yang digunakan untuk membangun UI di Jetpack Compose.

Pada baris ke [23] terdapat deklarasi fungsi `DetailScreen` dengan parameter `titleResId`, `imageResId`, `yearResId`, dan `detailResId` bertipe `Int`, yang digunakan untuk menampilkan detail game berdasarkan resource ID yang diberikan.

Pada baris ke [24] sampai [27] digunakan `Surface` sebagai wadah utama tampilan yang memenuhi ukuran layar penuh dan memiliki warna latar belakang `Color(0xFF131418)`.

Pada baris ke [31] terdapat `Column` yang digunakan untuk menyusun komponen secara vertikal, dengan jarak padding sebesar 16dp di sekeliling konten.

Pada baris ke [37] sampai [45] digunakan `Image` untuk menampilkan gambar game berdasarkan `imageResId`. Gambar memenuhi lebar penuh (`fillMaxWidth`), tinggi 400dp, memiliki sudut membulat, dan menggunakan `ContentScale.Crop` agar gambar menyesuaikan ukuran.

Pada baris ke [47] sampai [76] digunakan `Column` dan `Row` untuk menampilkan judul dan tahun rilis game. Judul menggunakan ukuran font 26sp dan tebal dengan warna putih, sedangkan tahun menggunakan ukuran font 16sp dengan warna abu-abu keunguan.

Pada baris ke [64] berfungsi untuk menjadi `Spacer` untuk memberi jarak vertikal 8dp antara bagian judul dan deskripsi.

Pada baris ke [66] sampai [75] digunakan `Text` untuk menampilkan label "tentang game ini:" dan isi detail deskripsi dari game. Label ditampilkan dalam font 14sp tebal berwarna putih, sedangkan isi detail ditampilkan dalam font 14sp biasa juga berwarna putih, dengan padding atas sebesar 4dp.

HomeScreen.kt

Pada baris ke [1] terdapat kata kunci `package` `com.pemrogramanmobile.myfavoritegames` yang menyatakan nama paket tempat file ini berada dalam struktur proyek aplikasi Android.

Pada baris ke [3] hingga baris ke [5] terdapat baris `import androidx.compose.foundation.layout.*` `import androidx.compose.foundation.lazy.LazyColumn` dan `import androidx.compose.foundation.lazy.items` yang berfungsi mengambil komponen tata letak dan daftar malas dari Jetpack Compose agar dapat digunakan untuk membuat tampilan daftar vertikal dengan elemen-elemen yang dapat dipetakan dari koleksi data.

Pada baris ke [6] hingga baris ke [8] terdapat baris `import androidx.compose.runtime.Composable` `import androidx.compose.runtime.collectAsState` dan `import androidx.compose.runtime.getValue` yang berfungsi mengambil anotasi penanda

fungsi UI serta utilitas untuk mengubah aliran data state flow menjadi objek yang dapat diamati di dalam komposable.

Pada baris ke [9] hingga baris ke [11] terdapat baris import `androidx.compose.ui.Modifier` import `androidx.compose.ui.unit.dp` dan import `com.pemrogamanmobile.myfavoritegames.model.Games` yang berfungsi mengambil objek modifikasi tata letak, satuan ukuran density-independent pixels, dan kelas data Games sebagai model setiap entri game.

Pada baris ke [13] terdapat anotasi `Composable` yang menandai bahwa fungsi berikutnya merupakan blok pembentuk antarmuka pengguna di Jetpack Compose.

Pada baris ke [14] hingga baris ke [17] terdapat deklarasi fungsi `HomeScreen` yang menerima dua parameter yaitu `gamesViewModel` sebagai sumber data dari `ViewModel` dan `onDetailClick` sebagai fungsi panggilan balik yang akan dijalankan ketika pengguna memilih detail salah satu game.

Pada baris ke [18] terdapat inisialisasi `val games` by `gamesViewModel.gamesState.collectAsState` yang berfungsi mengambil data daftar game dari objek `gamesState` pada `ViewModel` lalu mengonversinya menjadi nilai yang dapat dimonitor oleh Compose.

Pada baris ke [20] hingga baris ke [23] terdapat pemanggilan `LazyColumn` dengan pengaturan `verticalArrangement` `spacedBy 8 dp` untuk memberi jarak antar item dan modifier `fillMaxSize` untuk membuat daftar tersebut mengisi seluruh ruang layar yang tersedia.

Pada baris ke [24] hingga baris ke [29] terdapat blok `items` yang memetakan setiap elemen dalam variabel `games` menjadi baris daftar, kemudian memanggil `GameCard` untuk setiap objek game dengan meneruskan data game dan fungsi `onDetailClick` sehingga setiap kartu game ditampilkan secara berurutan di dalam daftar.

MainActivity.kt:

Pada baris ke [1] terdapat kata kunci `package` `com.pemrogamanmobile.myfavoritegames` yang menyatakan nama paket tempat file ini disimpan dalam struktur proyek aplikasi Android.

Pada baris ke [3] hingga [17] terdapat rangkaian impor yang mengambil berbagai kelas dan fungsi dari Android SDK, Jetpack Compose, `ViewModel Compose`, `Navigation Compose`, sumber data `Datasource`, tema `MyFavoriteGamesTheme`, `LaunchedEffect`, serta library `Timber` untuk logging, sehingga seluruh utilitas tersebut dapat digunakan di dalam `MainActivity`.

Pada baris ke [19] terdapat deklarasi class MainActivity : ComponentActivity yang menunjukkan bahwa MainActivity adalah kelas utama aplikasi yang mewarisi kemampuan dari ComponentActivity untuk mendukung Jetpack Compose.

Pada baris ke [20] terdapat override fun onCreate(savedInstanceState Bundle?) yang menandai metode onCreate di mana aktivitas diinisialisasi ketika pertama kali dibuat.

Pada baris ke [21] terdapat panggilan super.onCreate(savedInstanceState) yang memastikan siklus hidup aktivitas dijalankan dengan benar sebelum inisialisasi lebih lanjut.

Pada baris ke [23] hingga [24] terdapat komentar dan Timber.d("MainActivity created") yang berfungsi mencatat log debug bahwa MainActivity telah berhasil dibuat.

Pada baris ke [26] hingga [28] terdapat panggilan setContent diikuti oleh MyFavoriteGamesTheme dan Surface color Color 0xFF131418 yang menetapkan konten UI menggunakan Jetpack Compose, membungkus seluruh UI dengan tema khusus, dan memberikan latar abu-abu gelap di permukaan utama.

Pada baris ke [29] terdapat inisialisasi val navigationController sama dengan rememberNavController yang membuat dan menyimpan objek NavController untuk mengelola navigasi antar layar Compose.

Pada baris ke [31] hingga [37] terdapat komentar dan inisialisasi val gamesViewModel sama dengan viewModel factory GamesViewModelFactory gameCategory sama dengan "All" datasource sama dengan Datasource() yang berarti GamesViewModel dibuat menggunakan GamesViewModelFactory dengan parameter kategori game "All" dan sumber data Datasource.

Pada baris ke [39] terdapat panggilan NavHost navigationController sama dengan navigationController startDestination "home" yang membuat host navigasi dengan navigationController dan menentukan layar awal bernama "home".

Pada baris ke [40] hingga [54] terdapat blok destinasi composable "home" yang mencatat log debug "Navigated to Home Screen", kemudian memanggil HomeScreen gamesViewModel sama dengan gamesViewModel onDetailClick sama dengan lambda game panah yang di dalamnya memanggil gamesViewModel.onGameClicked game untuk menyimpan game yang dipilih, mencatat log debug "Detail button clicked untuk game game", dan memerintahkan navigationController.navigate "detail" untuk berpindah ke layar detail ketika tombol Detail ditekan.

Pada baris ke [56] hingga [70] terdapat blok destinasi composable "detail" yang mencatat log debug "Navigated to Detail Screen", kemudian menginisialisasi val selectedGame by gamesViewModel.selectedGame.collectAsState untuk mengambil game yang telah dipilih dari ViewModel, lalu jika selectedGame tidak null mencatat

log debug "Displaying details for game game" dan memanggil DetailScreen titleResId sama dengan game.titleResourceId imageResId sama dengan game.imageResourceId yearResId sama dengan game.yearResourceId detailResId sama dengan game.detailResourceId untuk menampilkan detail game.

Pada baris ke [71] hingga [77] terdapat blok ?: run yang berarti jika selectedGame null maka mencatat log debug "No selected game found navigating back to Home Screen" dan menjalankan LaunchedEffect Unit yang memanggil navController.popBackStack untuk kembali ke layar Home.

GamesViewModel.kt

Pada baris ke [1] terdapat package com.pemrogamanmobile.myfavoritegames yang berfungsi sebagai deklarasi nama paket tempat file ini disimpan dalam struktur proyek aplikasi Android.

Pada baris ke [3] sampai [8] terdapat rangkaian import yang mengimpor berbagai kelas dan fungsi: androidx.lifecycle.ViewModel untuk mendefinisikan kelas ViewModel, kotlinx.coroutines.flow.MutableStateFlow dan kotlinx.coroutines.flow.StateFlow untuk mengelola aliran data dalam arsitektur reaktif, com.pemrogamanmobile.myfavoritegames.data.Datasource sebagai sumber data utama, com.pemrogamanmobile.myfavoritegames.model.Games sebagai model data objek game, dan timber.log.Timber untuk mencatat log debug.

Pada baris ke [10] terdapat deklarasi class GamesViewModel(yang menandai dimulainya kelas GamesViewModel, sebuah ViewModel khusus yang mengelola data game sesuai kategori dan berkomunikasi dengan layer UI.

Pada baris ke [11] dan [12] terdapat parameter private val gameCategory: String dan private val datasource: Datasource yang berfungsi menyimpan nilai kategori game yang diinginkan dan instance Datasource untuk mendapatkan daftar game dari sumber data.

Pada baris ke [13] terdapat) : ViewModel() yang menunjukkan bahwa GamesViewModel mewarisi kelas ViewModel agar dapat bertahan selama siklus hidup UI tanpa terpengaruh perubahan konfigurasi.

Pada baris ke [15] terdapat deklarasi private val _gamesState = MutableStateFlow(yang memulai pembuatan objek MutableStateFlow untuk menyimpan dan memancarkan daftar game yang telah difilter sesuai kategori.

Pada baris ke [16] terdapat datasource.loadGames().filter { it.category == gameCategory } yang menjalankan pemanggilan loadGames() pada datasource lalu memfilter hasilnya berdasarkan properti category sama dengan gameCategory, sehingga hanya game yang sesuai kategori yang disimpan dalam _gamesState.

Pada baris ke [18] terdapat `val gameState: StateFlow<List<Games>> get() = _gameState` yang mendefinisikan properti publik `gameState` bertipe `StateFlow` agar UI dapat mengamati daftar game tanpa dapat memodifikasi langsung, sementara `_gameState` tetap privat.

Pada baris ke [20] terdapat deklarasi `private val _selectedGame = MutableStateFlow<Games?>(null)` yang membuat objek `MutableStateFlow` untuk menyimpan game yang dipilih oleh pengguna, dengan nilai awal `null` menandakan belum ada game terpilih.

Pada baris ke [21] terdapat `val selectedGame: StateFlow<Games?> get() = _selectedGame` yang mendefinisikan properti publik `selectedGame` agar UI dapat mengamati game yang dipilih tetapi tidak dapat mengubahnya langsung.

Pada baris ke [24] terdapat komentar `// Log saat data masuk ke list yang menjelaskan bahwa baris berikutnya berfungsi mencatat log saat data game berhasil dimuat.`

Pada baris ke [25] terdapat `Timber.d("Games data loaded: ${_gameState.value}")` yang berfungsi mencatat log debug berisi daftar game yang telah dimuat dan difilter sesuai kategori.

Pada baris ke [26] terdapat `}` yang menutup blok inisialisasi `init`.

Pada baris ke [28] terdapat deklarasi fungsi `fun onGameClicked(game: Games) {` yang mendefinisikan metode untuk menangani aksi ketika pengguna memilih salah satu game di UI.

Pada baris ke [29] terdapat `_selectedGame.value = game` yang berfungsi menetapkan nilai game yang dipilih ke dalam `_selectedGame`, sehingga `selectedGame` akan memancarkan nilai baru kepada observer.

Pada baris ke [30] terdapat komentar `// Log data game yang dipilih yang menjelaskan bahwa baris berikutnya mencatat log terkait game yang baru saja dipilih.`

Pada baris ke [31] terdapat `Timber.d("Game selected: $game")` yang berfungsi mencatat log debug dengan detail objek game yang dipilih..

strings.xml

Pada baris ke [1], terdapat elemen `root <resources>` yang digunakan untuk menampung semua sumber daya string dalam aplikasi Android.

Pada baris ke [2], terdapat elemen `<string>` dengan atribut `name="app_name"` yang menyimpan nama aplikasi, yaitu `"My favorite games"`.

Pada baris ke [4] sampai [13], terdapat sepuluh elemen `<string>` dengan atribut `name="titleX"` yang masing-masing menyimpan judul permainan favorit berdasarkan nomor urut, seperti "Terraria", "Mobile Legends", dan lainnya.

Pada baris ke [15] sampai [24], terdapat sepuluh elemen `<string>` dengan atribut `name="yearX"` yang masing-masing menyimpan tahun rilis permainan berdasarkan nomor urut, seperti "2011" untuk `year1` dan "2020" untuk `year6`.

Pada baris ke [26] sampai [35], terdapat sepuluh elemen `<string>` dengan atribut `name="detailX"` yang masing-masing menyimpan deskripsi detail permainan berdasarkan nomor urut. Contohnya, `detail1` berisi deskripsi "Terraria", dan `detail6` berisi deskripsi "Genshin Impact".

Pada baris ke [37] sampai [46], terdapat sepuluh elemen `<string>` dengan atribut `name="linkX"` yang masing-masing menyimpan URL terkait permainan berdasarkan nomor urut. Contohnya, `link1` berisi URL ke halaman Steam "Terraria", dan `link6` berisi URL ke halaman Google Play "Genshin Impact".

GamesViewModelFactory.kt

Pada baris ke [1] terdapat deklarasi `package com.pemrogamanmobile.myfavoritegames` yang berfungsi menyatakan nama paket tempat file ini berada dalam struktur proyek aplikasi Android.

Pada baris ke [3] terdapat `import androidx.lifecycle.ViewModel` yang berfungsi mengambil kelas `ViewModel` dari `AndroidX Lifecycle`, digunakan sebagai basis untuk `ViewModel` kustom.

Pada baris ke [4] terdapat `import androidx.lifecycle.ViewModelProvider` yang berfungsi mengambil antarmuka `ViewModelProvider.Factory`, digunakan untuk membuat instance `ViewModel` dengan parameter khusus.

Pada baris ke [5] terdapat `import com.pemrogamanmobile.myfavoritegames.data.Datasource` yang berfungsi mengambil kelas `Datasource` dari package `data`, yaitu sumber data untuk daftar game.

Pada baris ke [7] terdapat deklarasi `class GamesViewModelFactory` yang berfungsi mendefinisikan kelas pabrik untuk `ViewModel`, menerima parameter kategori game dan sumber data.

Pada baris ke [8] terdapat `private val gameCategory: String` yang berfungsi menyimpan nilai kategori game yang ingin dimuat ke dalam `ViewModel`.

Pada baris ke [9] terdapat `private val datasource: Datasource = Datasource()` yang berfungsi menyimpan instance `Datasource` dengan nilai default objek baru `Datasource`.

Pada baris ke [10] terdapat pewarisan dari `ViewModelProvider.Factory` yang menandakan bahwa kelas ini menyediakan mekanisme pembuatan `ViewModel` kustom.

Pada baris ke [11] terdapat deklarasi override fun `<T : ViewModel> create(modelClass: Class<T>): T` yang berfungsi menggantikan metode pembuatan `ViewModel`, menerima parameter kelas `ViewModel` yang diminta.

Pada baris ke [12] terdapat kondisi if `modelClass.isAssignableFrom(GameViewModel::class.java)` yang berfungsi memeriksa apakah kelas yang diminta dapat dipetakan ke `GameViewModel`.

Pada baris ke [13] terdapat anotasi `@Suppress("UNCHECKED_CAST")` yang berfungsi menonaktifkan peringatan konversi tipe yang tidak aman saat melakukan casting.

Pada baris ke [14] terdapat return `GameViewModel(gameCategory, datasource) as T` yang berfungsi membuat instance `GameViewModel` dengan parameter kategori dan sumber data, kemudian mengonversinya ke tipe umum `T`.

Pada baris ke [16] terdapat throw `IllegalArgumentException("Unknown ViewModel class")` yang berfungsi melempar pengecualian jika kelas `ViewModel` yang diminta tidak dikenali oleh pabrik.

MyFavoriteGamesApplication.kt

Pada baris ke [1] terdapat deklarasi package `com.pemrogamanmobile.mymoviefavoritesgames` yang menetapkan namespace tempat file ini berada dalam struktur proyek aplikasi Android.

Pada baris ke [3] terdapat perintah import `android.app.Application` yang berfungsi mengambil kelas `Application` dari framework Android sebagai kelas induk untuk inisialisasi aplikasi.

Pada baris ke [4] terdapat perintah import `timber.log.Timber` yang berfungsi mengimpor pustaka Timber untuk logging yang lebih fleksibel dan terstruktur.

Pada baris ke [6] terdapat deklarasi kelas `MyFavoriteGamesApplication` turunan dari `Application` yang memungkinkan kita mengoverride perilaku inisialisasi global aplikasi.

Pada baris ke [7] terdapat deklarasi metode override `onCreate` yang akan dipanggil pertama kali saat instance aplikasi dibuat.

Pada baris ke [8] terdapat pemanggilan super `onCreate` yang memastikan logika inisialisasi bawaan framework Android tetap dijalankan.

Pada baris ke [9] terdapat blok pengkondisian if BuildConfig.DEBUG yang memeriksa apakah aplikasi dalam mode debug agar hanya menjalankan logging lanjutan saat pengembangan.

Pada baris ke [10] terdapat perintah Timber plant DebugTree yang memasang tree bawaan Timber untuk mengeluarkan log ke Logcat selama mode debug.

Penjelasan Debugger :

1. Fungsi Debugger

- Memeriksa Alur Program

Debugger memiliki kemampuan untuk menjalankan program secara terkontrol baris per baris, agar dapat melihat secara detail bagaimana alur eksekusi dapat berjalan. Dengan tujuan supaya mudah memahami bagian mana yang dieksekusi terlebih dahulu, bagaimana nilai variabel berubah, dan kondisi cabang (if/else) mana yang terpenuhi.

- Menemukan dan Memperbaiki Bug (Errors)

Alih-alih membaca seluruh kode secara manual untuk mencari kesalahannya, debugger dapat membantu untuk menemukan titik (line) di mana program tidak berjalan sesuai harapan. Kita dapat menghentikan eksekusi tepat sebelum munculnya error (misalnya NullPointerException, IndexOutOfBoundsException, dan sejenisnya) lalu memeriksa tumpukan panggilan (call stack) dan nilai variabel.

- Memantau Nilai Variabel Secara Real-time

Ketika program berhenti di breakpoint, maka isi variabel, baik variabel sederhana (integer, string, dll.) maupun objek kompleks, dapat terlihat, dan memiliki tujuan untuk memastikan apakah nilainya sesuai ekspektasi.

2. Cara Menggunakan Debugger

Untuk menggunakan Debugger maka dapat dilihat sebagai berikut:

1. Memasang Breakpoint

2. Buka file kode sumber yang ingin di periksa.

3. Klik di margin kiri (atau tekan shortcut) pada baris kode di mana ingin di eksekusi program berhenti (contoh: line 42). Breakpoint biasanya ditandai dengan titik merah.

4. Menjalankan Program dalam Mode Debug

5. Selanjutnya, pilih “Debug” dan jalankan.

6. Program akan berjalan hingga menemui breakpoint pertama, lalu menghentikan eksekusi secara otomatis.

7. Periksa Jendela Debugger, setelah eksekusi terhenti, maka terlihat beberapa panel penting yang bisa dibaca seperti variabel.

3. Fitur Step Into, Step Over, dan Step Out

Berikut adalah beberapa dari fitur step:

1. Step Into

Step into dijalankan saat eksekusi berhenti di suatu baris yang memanggil fungsi/metode. Jika pada baris tersebut ada pemanggilan fungsi, maka “Step Into” akan masuk ke baris pertama fungsi tersebut, atau bisa dibilang menelusuri ke dalam implementasi fungsi.

2. Step Over

Step over dijalankan saat eksekusi berhenti di baris yang memanggil fungsi, “Step Over” akan mengeksekusi keseluruhan fungsi tersebut secara instan, lalu berhenti di baris kode berikutnya setelah fungsi selesai dikembalikan.

3. Step Out

Step out digunakan Ketika berada di dalam suatu fungsi (setelah beberapa kali Step Into), “Step Out” akan menjalankan sisa fungsi tersebut hingga kembali ke pemanggilnya, lalu menghentikan eksekusi pada baris di pemanggil.

SOAL 2

Soal Praktikum:

2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class dalam arsitektur Android adalah objek global yang diinisialisasi sekali saat proses aplikasi dijalankan dan bertindak sebagai “entry point” bagi semua komponen seperti Activity, Service, Receiver, dsb. Fungsi utamanya sendiri, meliputi inisialisasi library dan konfigurasi global (misalnya dependency injection, analytics, atau database), menyediakan ApplicationContext yang dapat diakses di seluruh aplikasi, menyimpan variabel atau singleton yang dibutuhkan lintas-komponen, serta menangani event lifecycle tingkat aplikasi seperti onLowMemory() atau onTrimMemory() untuk manajemen cache dan sumber daya.

MODUL 5 : CONNECT TO THE INTERNET

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

COMPOSE :

MovieDao.kt

Table 51. Source code MovieDao.kt Modul 5

1	package com.pemrogramanmobile.movielist.data.local.dao
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import
	com.pemrogramanmobile.movielist.data.local.entity.MovieEnt
	ity
8	
9	@Dao

```

10 interface MovieDao {
11     @Insert(onConflict = OnConflictStrategy.REPLACE)
12     suspend fun insertMovies(movies: List<MovieEntity>)
13
14     @Query("SELECT * FROM movies")
15     suspend fun getAllMovies(): List<MovieEntity>
16
17     @Query("DELETE FROM movies")
18     suspend fun clearMovies()
19 }

```

MovieDatabase.kt

Table 52. Source code MovieDatabase.kt Modul 5

```

1 package com.pemrogamanmobile.movielist.data.local.db
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import
8 com.pemrogamanmobile.movielist.data.local.dao.MovieDao
9 import com.pemrogamanmobile.movielist.data.local.entity.
10 MovieEntity
11
12 @Database(entities = [MovieEntity::class],
13 version = 1, exportSchema = false)
14 abstract class MovieDatabase : RoomDatabase() {
15     abstract fun movieDao(): MovieDao
16
17     companion object {
18         @Volatile
19         private var INSTANCE: MovieDatabase? = null
20
21         fun getDatabase(context: Context): MovieDatabase =
22             INSTANCE ?: synchronized(this) {
23                 INSTANCE ?: Room.databaseBuilder(
24                     context.applicationContext,
25                     MovieDatabase::class.java,
26                     "movie_db"
27                 ).fallbackToDestructiveMigration()
28             }.build().also { INSTANCE = it }
29     }
30 }

```

MovieEntity.kt

Table 53. Source code MovieEntity.kt Modul 5

1	package
2	com.pemrogamanmobile.movielist.data.local.entity
3	
4	import androidx.room.Entity
5	import androidx.room.PrimaryKey
6	
7	@Entity(tableName = "movies")
8	data class MovieEntity(
9	@PrimaryKey val id: Int,
10	val title: String,
11	val overview: String,
12	val posterPath: String,
13	val releaseDate: String
14	)

PreferencesManager.kt

Table 54. Source code PreferencesManager.kt Modul 5

1	package
	com.pemrogamanmobile.movielist.data.preferences
2	
3	import android.content.Context
4	import android.content.SharedPreferences
5	
6	class PreferencesManager(context: Context) {
7	private val prefs: SharedPreferences =
	context.getSharedPreferences("movie_prefs",
	Context.MODE_PRIVATE)
8	
9	fun saveThemeSetting(isDarkMode: Boolean) {
10	prefs.edit().putBoolean("dark_mode",
	isDarkMode).apply()
11	}
12	
13	fun isDarkMode(): Boolean =
	prefs.getBoolean("dark_mode", false)
14	}

ApiService.kt

Table 55. Source code ApiService.kt Modul 5

1	package
	com.pemrogamanmobile.movielist.data.remote.api
2	
3	import
	com.pemrogamanmobile.movielist.data.remote.dto.
4	MovieResponseDto
5	import retrofit2.http.GET
6	import retrofit2.http.Query
7	
8	interface ApiService {
9	@GET("movie/popular")
10	suspend fun getPopularMovies(
11	@Query("api_key") apiKey: String
12	): MovieResponseDto
13	}

MovieDto.kt

Table 56. Source code MovieDao.kt Modul 5

1	package
2	com.pemrogamanmobile.movielist.data.remote.dto
3	
4	import kotlinx.serialization.SerialName
5	import kotlinx.serialization.Serializable
6	
7	@Serializable
8	data class MovieDto(
9	val id: Int,
10	val title: String,
11	val overview: String,
12	@SerialName("poster_path") val posterPath:
13	String,
14	@SerialName("release_date") val releaseDate:
15	String
16	)
17	
18	@Serializable
	data class MovieResponseDto(
	val results: List<MovieDto>
	)

RetrofitInstance.kt

Table 57. Source code RetrofitInstance.kt Modul 5

1	package com.pemrogamanmobile.movielist.data.remote
2	
3	import com.jakewharton.retrofit2.converter.kotlinx. serialization.asConverterFactory
4	import
5	com.pemrogamanmobile.movielist.data.remote.api. ApiService
6	import kotlinx.serialization.json.Json
7	import okhttp3.MediaType.Companion.toMediaType
8	import retrofit2.Retrofit
9	
10	object RetrofitInstance {
	private const val BASE_URL =
11	"https://api.themoviedb.org/3/"
12	
13	private val retrofit by lazy {
14	Retrofit.Builder()
15	.baseUrl(BASE_URL)
	.addConverterFactory
	(Json { ignoreUnknownKeys = true }.asConverterFactory
16	("application/json".toMediaType()))
17	.build()
18	}
19	
20	val api: ApiService by lazy {
21	retrofit.create(ApiService::class.java)
22	}
23	}

MovieRepositoryImpl.kt

Table 58. Source code MovieRepositoryImpl.kt Modul 5

1	package com.pemrogamanmobile.movielist.data.repository
2	import
3	com.pemrogamanmobile.movielist.data.local.dao.MovieDao
	import com.pemrogamanmobile.movielist.data.remote.api.
4	ApiService
	import com.pemrogamanmobile.movielist.domain.model.
5	Movie
	import com.pemrogamanmobile.movielist.domain.

```

6 repository.MovieRepository
import
7 com.pemrogamanmobile.movielist.utils.NetworkResult
import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.flow.flow
9
10 class MovieRepositoryImpl(
11     private val api: ApiService,
12     private val dao: MovieDao
13 ) : MovieRepository {
14     override fun
15     getPopularMovies(): Flow<NetworkResult
16     <List<Movie>>> = flow {
17         emit(NetworkResult.Loading)
18         try {
19             val response = api.
20             getPopularMovies("a9936cbd33c74fbfa33d892ad498af3c")
21             val movies = response.results.map {
22                 Movie(it.id, it.title,
23                 it.overview, it.posterPath, it.releaseDate)
24             }
25             dao.clearMovies()
26             dao.insertMovies(movies.map { entity ->
27                 com.pemrogamanmobile.movielist.data.local.
28                 entity.
29                 MovieEntity(
30                     entity.id,
31                     entity.title,
32                     entity.overview,
33                     entity.posterPath,
34                     entity.releaseDate
35                 )
36             })
37             emit(NetworkResult.Success(movies))
38         } catch (e: Exception) {
39             val cached = dao.getAllMovies().map {
40                 Movie(it.id, it.title,
41                 it.overview, it.posterPath, it.releaseDate)
42             }
43             if (cached.isNotEmpty()) {
44                 emit(NetworkResult.Success(cached))
45             } else {
46                 emit(NetworkResult.Error
47                 (e.message ?: "Unknown Error"))

```


41	}
42	}
43	}
44	

Movie.kt

Table 59. Source code Movie.kt Modul 5

1	package com.pemrogamanmobile.movielist.domain.model
2	
3	data class Movie(
4	val id: Int,
5	val title: String,
6	val overview: String,
7	val posterPath: String,
8	val releaseDate: String
9	)

MovieRepository.kt

Table 60. Source code MovieRepository.kt Modul 5

1	package
2	com.pemrogamanmobile.movielist.domain.repository
3	import
4	com.pemrogamanmobile.movielist.domain.model.Movie
5	import
6	com.pemrogamanmobile.movielist.utils.NetworkResult
7	import kotlinx.coroutines.flow.Flow
8	interface MovieRepository {
9	fun
	getPopularMovies():
	Flow<NetworkResult<List<Movie>>>
	}

MovieUseCase.kt

Table 61. Source code MovieUseCase.kt Modul 5

1	package
2	com.pemrogamanmobile.movielist.domain.usecase

3	import
	com.pemrogamanmobile.movielist.domain.model.Movie
4	import
	com.pemrogamanmobile.movielist.domain.repository.
	MovieRepository
5	import
	com.pemrogamanmobile.movielist.utils.NetworkResult
6	import kotlinx.coroutines.flow.Flow
7	
8	class MovieUseCase(private val repository:
	MovieRepository) {
9	fun getPopularMovies() :
	Flow<NetworkResult<List<Movie>>> {
10	return repository.getPopularMovies()
11	}
12	}

DetailScreen.kt

Table 62. Source code DetailScreen.kt Modul 5

1	package
	com.pemrogamanmobile.movielist.presentation.ui.screen
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import
	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.foundation.verticalScroll
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.*
10	import androidx.compose.ui.Alignment
11	import androidx.compose.ui.Modifier
12	import androidx.compose.ui.draw.clip
13	import androidx.compose.ui.layout.ContentScale
14	import androidx.compose.ui.text.font.FontWeight
15	import androidx.compose.ui.unit.dp
16	import androidx.compose.ui.unit.sp
17	import androidx.navigation.NavController
18	import coil.compose.rememberAsyncImagePainter
19	import
	com.pemrogamanmobile.movielist.presentation.viewmodel.
	MovieViewModel

```

20 import
   com.pemrogamanmobile.movielist.utils.NetworkResult
21
22 @Composable
23 fun DetailScreen(
24     viewModel: MovieViewModel,
25     movieId: Int,
26     isDarkMode: Boolean,
27     onToggleTheme: () -> Unit
28 ) {
29     val moviesResult by
viewModel.movies.collectAsState()
30     val scrollState = rememberScrollState()
31     val colorScheme = MaterialTheme.colorScheme
32
33     Scaffold(
34         topBar = {
35             TopAppBar(
36                 title = "Detail Movie",
37                 isDarkMode = isDarkMode,
38                 onToggleTheme = onToggleTheme
39             )
40         }
41     ) { paddingValues ->
42         when (moviesResult) {
43             is NetworkResult.Loading -> {
44                 Box(modifier = Modifier
45                     .fillMaxSize()
46                     .padding(paddingValues),
47                     contentAlignment = Alignment.Center
48                 ) {
49                     CircularProgressIndicator(color =
colorScheme.primary)
50                 }
51             }
52
53             is NetworkResult.Success -> {
54                 val movie = (moviesResult as
NetworkResult.Success).data.find { it.id == movieId }
55                 if (movie != null) {
56                     Surface(
57                         modifier = Modifier
58                             .fillMaxSize()
59                             .padding(paddingValues),
60                         color = colorScheme.background

```

61	) {		
62	Column (		
63	modifier = Modifier		
64	.padding (16.dp)		
65	.verticalScroll (scrollState)		
66	) {		
67	Image (		
68	painter	=	
	rememberAsyncImagePainter		
	("https://image.tmdb.org/t/p/w500\${movie.posterPath}"),		
69	contentDescription	=	
	movie.title,		
70	modifier = Modifier		
71	.fillMaxWidth()		
72	.height (400.dp)		
73	.clip (RoundedCornerShape (6.dp)),		
74	contentScale	=	
	ContentScale.Crop		
75	)		
76			
77	Column(modifier	=	
	Modifier.fillMaxWidth()) {		
78	Row (		
79	modifier	=	
	Modifier.fillMaxWidth(),		
80	horizontalArrangement = Arrangement.SpaceBetween		
81	) {		
82	Text (		
83	text	=	
	movie.title,		
84	style	=	
	MaterialTheme.typography.headlineSmall.copy (		
85	color	=	
	colorScheme.onBackground		
86	),		
87	modifier	=	
	Modifier.weight (1f)		
88	)		
89	Text (		
90	text	=	
	movie.releaseDate,		
91			

92	MaterialTheme.typography.bodyMedium.copy(	style	=
93	colorScheme.onBackground.copy(alpha = 0.6f)	color	=
94	)	modifier	=
95	Modifier.align(Alignment.CenterVertically)		
96	)		
97	}		
98		Spacer(modifier	=
99	Modifier.height(8.dp))		
100			
101	Text(		
102	film ini:",	text = "Tentang	
103	MaterialTheme.typography.bodyLarge.copy(	style	=
104	14.sp,	fontSize	=
105	colorScheme.onBackground,	color	=
106	FontWeight.Bold	fontWeight	=
107	)		
108	)		
109			
110	Text(		
111	movie.overview,	text	=
112	MaterialTheme.typography.bodyMedium.copy(	style	=
113	14.sp,	fontSize	=
114	colorScheme.onBackground	color	=
115		),	
116	Modifier.padding(top = 4.dp)	modifier	=
117	)		
118	}		
119	}		
120	} else {		
121	Text(		

122	text = "Movie not found",
123	color = colorScheme.error,
124	modifier = Modifier
125	padding(paddingValues)
126	padding(16.dp)
127	)
128	}
129	}
130	
131	is NetworkResult.Error -> {
132	Text(
133	text = "Error loading movie",
134	color = colorScheme.error,
135	modifier = Modifier
136	padding(paddingValues)
137	padding(16.dp)
138	)
139	}
140	}
141	}
142	}

MainScreen.kt

Table 63. Source code MainScreen.kt Modul 5

1	package
	com.pemrogamanmobile.movielist.presentation.ui.screen
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.lazy.LazyColumn
5	import androidx.compose.foundation.lazy.items
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.*
8	import androidx.compose.ui.Alignment
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.unit.dp
11	import androidx.navigation.NavController
12	import
	com.pemrogamanmobile.movielist.presentation.viewmodel.
	MovieViewModel
13	import
	com.pemrogamanmobile.movielist.utils.NetworkResult
14	
15	@Composable

```

16 fun mainScreen(
17     viewModel: MovieViewModel,
18     navController: NavController,
19     isDarkMode: Boolean,
20     onToggleTheme: () -> Unit
21 ) {
22     val moviesResult by
viewModel.movies.collectAsState()
23     val colorScheme = MaterialTheme.colorScheme
24
25     Scaffold(
26         topBar = {
27             TopBarApp(
28                 title = "Popular Movie",
29                 isDarkMode = isDarkMode,
30                 onToggleTheme = onToggleTheme
31             )
32         }
33     ) { paddingValues ->
34         Surface(
35             modifier = Modifier
36                 .fillMaxSize()
37                 .padding(paddingValues),
38             color = colorScheme.background
39         ) {
40             when (moviesResult) {
41                 is NetworkResult.Loading -> {
42                     Box(modifier =
Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
43                         CircularProgressIndicator(color = colorScheme.primary)
44                     }
45                 }
46
47                 is NetworkResult.Success -> {
48                     val movies = (moviesResult as
NetworkResult.Success).data
49                     LazyColumn(
verticalArrangement =
Arrangement.spacedBy(8.dp),
50                     modifier =
Modifier.padding(8.dp)
51                 ) {
52                     items(movies) { movie ->

```

53	MovieCard(movie) {
54	navController.navigate("detail/\${movie.id}")
55	}
56	}
57	}
58	}
59	
60	is NetworkResult.Error -> {
61	Text(
62	text = "Error: \${(moviesResult
63	as NetworkResult.Error).message}",
64	color = colorScheme.error,
65	modifier
66	Modifier.padding(16.dp)
67	)
68	}
69	}
70	}

MovieCard.kt

Table 64. Source code MovieCard.kt Modul 5

1	package
2	com.pemrogamanmobile.movielist.presentation.ui.screen
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
6	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.material3.Button
8	import androidx.compose.material3.ButtonDefaults
9	import androidx.compose.material3.Card
10	import androidx.compose.material3.CardDefaults
11	import androidx.compose.material3.MaterialTheme
12	import androidx.compose.material3.Text
13	import androidx.compose.runtime.Composable
14	import androidx.compose.ui.Alignment
15	import androidx.compose.ui.Modifier
16	import androidx.compose.ui.draw.clip
17	import androidx.compose.ui.layout.ContentScale
18	import androidx.compose.ui.text.TextStyle


```

18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.unit.dp
20 import androidx.compose.ui.unit.sp
21 import coil.compose.rememberAsyncImagePainter
22 import
23     com.pemrogamanmobile.movielist.domain.model.Movie
24
25 @Composable
26 fun MovieCard(movie: Movie, onClick: () -> Unit) {
27     val colorScheme = MaterialTheme.colorScheme
28
29     Card(
30         modifier = Modifier
31             .fillMaxWidth()
32             .padding(12.dp),
33         colors = CardDefaults.cardColors(
34             containerColor = colorScheme.primary
35         )
36     ) {
37         Row(modifier = Modifier.fillMaxWidth()) {
38
39             Image(
40                 painter = rememberAsyncImagePainter
41                 ("https://image.tmdb.org/t/p/w500${movie.posterPath}"),
42                 contentDescription = movie.title,
43                 modifier = Modifier
44                     .padding(start = 12.dp)
45                     .width(100.dp)
46                     .height(140.dp)
47                     .clip(RoundedCornerShape(6.dp))
48                     .align(Alignment.CenterVertically),
49                 contentScale = ContentScale.Crop
50             )
51
52             Column(
53                 modifier = Modifier
54                     .padding(16.dp)
55                     .fillMaxWidth(),
56                 verticalArrangement =
57                     Arrangement.SpaceBetween
58             ) {
59                 Row(
60                     modifier = Modifier.fillMaxWidth(),
61                     horizontalArrangement =
62                         Arrangement.SpaceBetween,

```

59	verticalAlignment = Alignment.Top	
60	) {	
61	Text(	
62	text = movie.title,	
63	style = TextStyle(	
64	fontSize = 26.sp,	
65	fontWeight	=
	FontWeight.Bold,	
66	color	=
	colorScheme.onSurface	
67	),	
68	modifier = Modifier.weight(1f)	
69	)	
70		
71	Spacer(modifier	=
	Modifier.width(8.dp))	
72		
73	Text(	
74	text = movie.releaseDate,	
75	style = TextStyle(	
76	fontSize = 16.sp,	
77	color	=
	colorScheme.onSurfaceVariant	
78	),	
79	modifier = Modifier	
80	.wrapContentWidth(Alignment.End)	
81	.align(Alignment.CenterVertically)	
82	)	
83	}	
84		
85	Spacer(modifier	=
	Modifier.height(8.dp))	
86		
87	Column(modifier	=
	Modifier.fillMaxWidth()) {	
88	Text(	
89	text = "tentang film ini:",	
90	style = TextStyle(	
91	fontSize = 14.sp,	
92	fontWeight	=
	FontWeight.Bold,	
93	color	=
	colorScheme.onSurface	

```

94         )
95     )
96     Text(
97         text = movie.overview,
98         style = TextStyle(
99             fontSize = 14.sp,
100             color
101             =
102             colorScheme.onSurface
103         ),
104         modifier = Modifier.padding(top
105         = 4.dp)
106     )
107 }
108
109     Spacer(modifier
110     =
111     Modifier.height(12.dp))
112
113     Row(
114         modifier = Modifier.fillMaxWidth(),
115         horizontalArrangement
116         =
117         Arrangement.End,
118         verticalAlignment
119         =
120         Alignment.CenterVertically
121     ) {
122         Button(
123             onClick = onClick,
124             modifier = Modifier
125             .width(100.dp)
126             .height(44.dp),
127             colors
128             =
129             ButtonDefaults.buttonColors(
130                 containerColor
131                 =
132                 colorScheme.onSurface,
133                 contentColor
134                 =
135                 colorScheme.onPrimary
136             )
137         ) {
138             Text("Detail", style
139             =
140             TextStyle(fontSize = 16.sp))
141         }
142     }
143 }
144 }
145 }
146 }
147 }
148 }
149 }

```

TopBarApp.kt

Table 65. Source code TopBarApp.kt Modul 5

1	package	
	com.pemrogamanmobile.movielist.presentation.ui.screen	
2		
3	import androidx.compose.material.icons.Icons	
4	import	
	androidx.compose.material.icons.filled.DarkMode	
5	import	
	androidx.compose.material.icons.filled.LightMode	
6	import androidx.compose.material3.*	
7	import androidx.compose.runtime.Composable	
8		
9	@OptIn(ExperimentalMaterial3Api::class)	
10	@Composable	
11	fun TopBarApp(	
12	title: String,	
13	isDarkMode: Boolean,	
14	onToggleTheme: () -> Unit	
15	) {	
16	TopAppBar(	
17	title = {	
18	Text(text = title)	
19	},	
20	actions = {	
21	IconButton(onClick = onToggleTheme) {	
22	Icon(	
23	imageVector = if (isDarkMode)	
	Icons.Default.LightMode else Icons.Default.DarkMode,	
24	contentDescription = "Toggle	
25	Theme"	
26	)	
27	}	
28	},	
29	colors = TopAppBarDefaults.topAppBarColors(	
	containerColor	=
30	MaterialTheme.colorScheme.primary,	
	titleContentColor	=
31	MaterialTheme.colorScheme.onPrimary,	
	actionIconContentColor	=
32	MaterialTheme.colorScheme.onPrimary	
33	)	

34	)
35	}

MainActivity.kt

Table 66. Source code MainActivity.kt Modul 5

1	package com.pemrogamanmobile.movielist.presentation.ui
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.*
8	import androidx.compose.ui.graphics.Color
9	import androidx.lifecycle.ViewModelProvider
10	import androidx.navigation.NavType
11	import androidx.navigation.compose.NavHost
12	import androidx.navigation.compose.composable
13	import
	androidx.navigation.compose.rememberNavController
14	import androidx.navigation.navArgument
15	import com.pemrogamanmobile.movielist.data.local.db.
	MovieDatabase
16	import com.pemrogamanmobile.movielist.data.remote.
	RetrofitInstance
17	import com.pemrogamanmobile.movielist.data.repository.
	MovieRepositoryImpl
18	import com.pemrogamanmobile.movielist.domain.usecase.
	MovieUseCase
19	import
	com.pemrogamanmobile.movielist.presentation.ui.screen.
	DetailScreen
20	import
	com.pemrogamanmobile.movielist.presentation.ui.screen.
	MainScreen
21	import
	com.pemrogamanmobile.movielist.presentation.viewmodel.
	MovieViewModel
22	import
	com.pemrogamanmobile.movielist.presentation.viewmodel.
	MovieViewModelFactory
23	import
	com.pemrogamanmobile.movielist.presentation.viewmodel.
	ThemeViewModel

```

24 import
   com.pemrogamanmobile.movielist.presentation.viewmodel.
   ThemeViewModelFactory
25 import
   com.pemrogamanmobile.movielist.presentation.ui.theme.
   MovieListTheme
26
27 class MainActivity : ComponentActivity() {
28     override fun onCreate(savedInstanceState: Bundle?)
29     {
30         super.onCreate(savedInstanceState)
31
32         val db =
   MovieDatabase.getDatabase(applicationContext)
33         val repository =
   MovieRepositoryImpl(RetrofitInstance.api,
34 db.movieDao())
35         val useCase = MovieUseCase(repository)
36         val movieViewModel = ViewModelProvider(this,
   MovieViewModelFactory(useCase))
37         [MovieViewModel::class.java]
38
39         val themeViewModel = ViewModelProvider(this,
   ThemeViewModelFactory(applicationContext))
40         [ThemeViewModel::class.java]
41
42         setContent {
43
44             val isDarkMode by
   themeViewModel.isDarkMode.collectAsState()
45
46             MovieListTheme(darkTheme = isDarkMode) {
47                 Surface(color = Color(0xFF131418)) {
48                     val navController =
   rememberNavController()
49                     NavHost(navController =
   navController, startDestination = "main") {
50                         composable("main") {
51                             MainScreen(
52                                 viewModel =
   movieViewModel,
53                                 navController =
   navController,
54                                 isDarkMode =
   isDarkMode,

```

52	onToggleTheme	=	{
53	themeViewModel.toggleTheme()	}	
54	)		
55	}		
56	composable(		
	"detail/{id}",		
57	arguments	=	
58	listOf(navArgument("id") { type = NavType.IntType })		
	) { backStackEntry ->		
59	val id	=	
60	backStackEntry.arguments?.getInt("id") ?: -1		
	DetailScreen(		
61	viewModel	=	
62	movieViewModel,		
63	movieId = id,		
	isDarkMode	=	
64	isDarkMode,		
65	onToggleTheme	=	{
66	themeViewModel.toggleTheme()	}	
67	)		
68	}		
69	}		
70	}		
71	}		
72	}		
73	}		
74	}		

MovieViewModel.kt

Table 67. Source code MovieViewModel.kt Modul 5

1	package
	com.pemrogamanmobile.movielist.presentation.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import
	com.pemrogamanmobile.movielist.domain.model.Movie
6	import com.pemrogamanmobile.movielist.domain.usecase.
	MovieUseCase
7	import
	com.pemrogamanmobile.movielist.utils.NetworkResult
8	import kotlinx.coroutines.flow.MutableStateFlow
9	import kotlinx.coroutines.flow.StateFlow

```

10 import kotlinx.coroutines.launch
11
12 class MovieViewModel(private val
movieUseCase: MovieUseCase) : ViewModel() {
13     private val _movies =
MutableStateFlow<NetworkResult<List<Movie>>>
(NetworkResult.Loading)
14     val movies: StateFlow<NetworkResult<List<Movie>>>
get() = _movies
15
16     init {
17         fetchMovies()
18     }
19
20     fun fetchMovies() {
21         viewModelScope.launch {
22             movieUseCase.getPopularMovies().collect {
23                 _movies.value = it
24             }
25         }
26     }
27 }

```

MovieViewModelFactory.kt

Table 68. Source code MovieViewModelFactory.kt Modul 5

```

1 package
com.pemrogamanmobile.movielist.presentation.viewmodel
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import
com.pemrogamanmobile.movielist.domain.usecase.MovieUseC
ase
6
7 class MovieViewModelFactory(private val useCase:
MovieUseCase) : ViewModelProvider.Factory {
8     override fun <T : ViewModel> create(modelClass:
Class<T>): T {
9         if
(modelClass.isAssignableFrom(MovieViewModel::class.java
)) {
10             return MovieViewModel(useCase) as T
11         }

```


12	throw	IllegalArgumentException("Unknown
13	ViewModel class")	
14	}	
	}	

ThemeViewModel.kt

Table 69. Source code ThemeViewModel.kt Modul 5

1	package	com.pemrogamanmobile.movielist.presentation.viewmodel
2		
3	import androidx.lifecycle.ViewModel	
4	import androidx.lifecycle.viewModelScope	
5	import	com.pemrogamanmobile.movielist.data.preferences.
6	PreferencesManager	
7	import kotlinx.coroutines.flow.MutableStateFlow	
8	import kotlinx.coroutines.flow.StateFlow	
9	import kotlinx.coroutines.launch	
10	class ThemeViewModel(private val preferencesManager:	PreferencesManager) : ViewModel() {
11		
12	private val _isDarkMode	=
	MutableStateFlow(preferencesManager.isDarkMode())	
13	val isDarkMode: StateFlow<Boolean> = _isDarkMode	
14		
15	fun toggleTheme() {	
16	val newMode = !_isDarkMode.value	
17	_isDarkMode.value = newMode	
18	viewModelScope.launch {	
19	preferencesManager.saveThemeSetting(newMode)	
20	}	
21	}	
22	}	

ThemeViewModelFactory.kt

Table 70. Source code ThemeViewModelFactory.kt Modul 5

1	package	com.pemrogamanmobile.movielist.presentation.viewmodel
2		

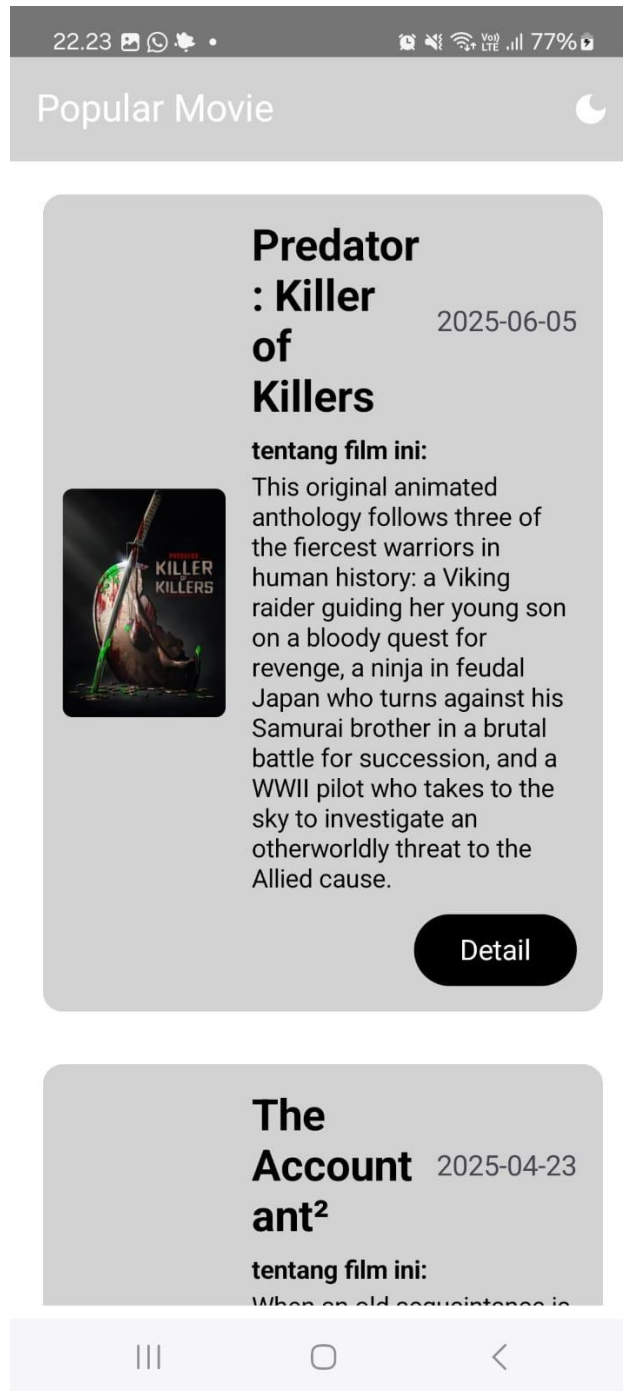
3	import android.content.Context
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider
6	import
	com.pemrogamanmobile.movielist.data.preferences.
	PreferencesManager
7	
8	class ThemeViewModelFactory(private val context:
	Context) : ViewModelProvider.Factory {
9	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
10	val prefs = PreferencesManager(context)
11	return ThemeViewModel(prefs) as T
12	}
13	}

NetworkResult.kt

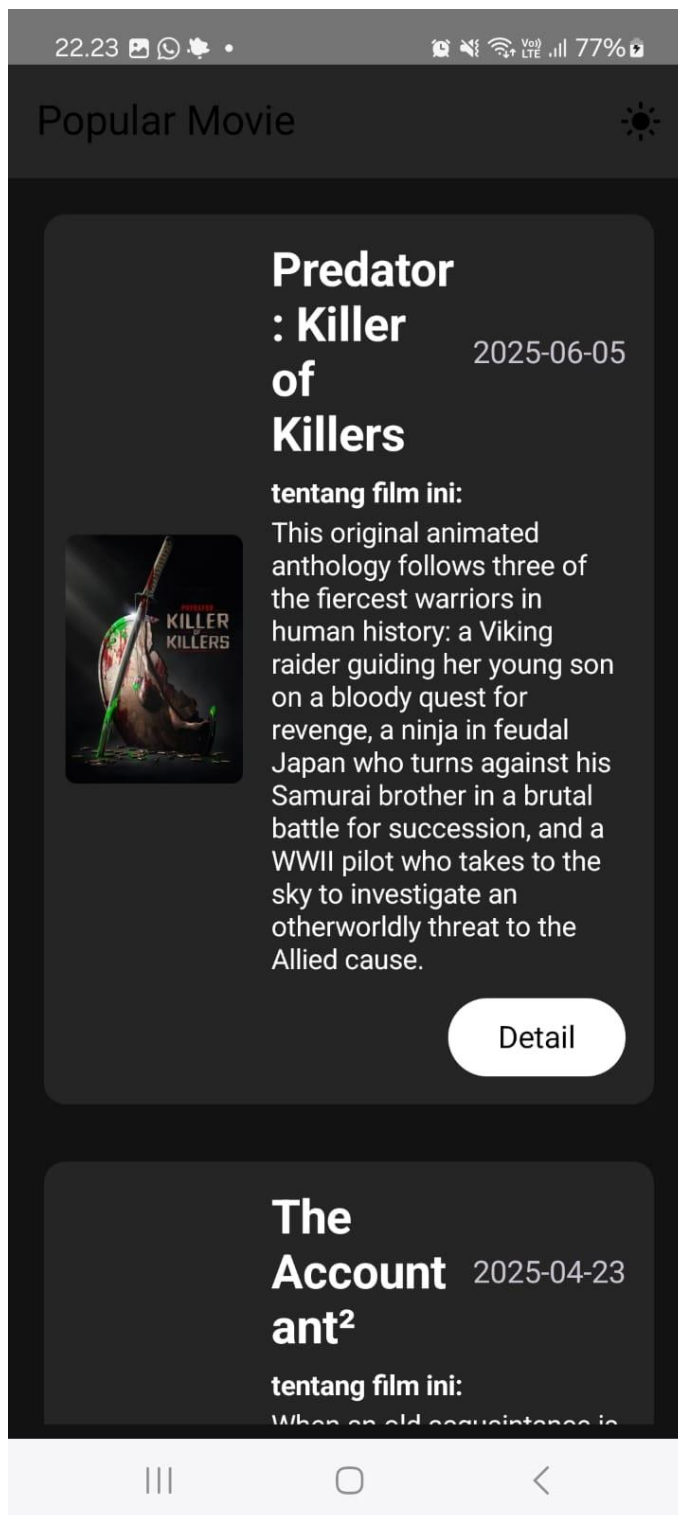
Table 71. Source code NetworkResult.kt Modul 5

1	package com.pemrogamanmobile.movielist.utils
2	
3	sealed class NetworkResult<out T> {
4	data class Success<out T>(val data: T):
	NetworkResult<T>()
5	data class Error(val message: String):
	NetworkResult<Nothing>()
6	object Loading: NetworkResult<Nothing>()
7	}

B. Output Program



Gambar 44. Screenshot output light mode Modul 5



Gambar 45. Screenshot output dark mode Modul 5

C. Pembahasan

COMPOSE :

MovieDao.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.local.dao` yang berfungsi sebagai nama paket dari modul DAO dan tempat menyimpan file ini dalam struktur aplikasi.

Pada baris ke [3] sampai [7], dilakukan impor berbagai komponen penting dari Android Room, yaitu Dao, Insert, OnConflictStrategy, dan Query, serta entitas MovieEntity yang merepresentasikan tabel movies di database lokal.

Pada baris ke [9] terdapat anotasi `@Dao` yang menandai interface ini sebagai Data Access Object untuk Room, yakni komponen yang bertanggung jawab atas operasi baca-tulis ke database.

Pada baris ke [10] hingga [19] dideklarasikan interface MovieDao yang berisi tiga method suspend untuk melakukan operasi database secara asynchronous.

Pada baris ke [11] anotasi `@Insert(onConflict = OnConflictStrategy.REPLACE)` menyatakan bahwa saat menyisipkan daftar entitas, jika terjadi konflik (misalnya primary key duplikat), data lama akan digantikan dengan data baru. Baris ke [12] suspend fun `insertMovies(movies: List<MovieEntity>)` adalah method untuk menyimpan kumpulan MovieEntity.

Pada baris ke [14] anotasi `@Query("SELECT * FROM movies")` mendefinisikan perintah SQL untuk mengambil semua baris dari tabel movies. Baris ke [15] suspend fun `getAllMovies(): List<MovieEntity>` mengembalikan daftar seluruh film yang tersimpan.

Pada baris ke [17] anotasi `@Query("DELETE FROM movies")` mendefinisikan perintah SQL untuk menghapus semua data dalam tabel movies. Baris ke [18] suspend fun `clearMovies()` mengeksekusi operasi pembersihan tersebut.

MovieDatabase.kt:

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.local.db` yang berfungsi sebagai nama paket dari modul database lokal, tempat menyimpan kelas konfigurasi Room Database.

Pada baris ke [3] sampai [6], dilakukan impor Context dari Android serta komponen Room yaitu Database, Room, dan RoomDatabase, yang diperlukan untuk mendefinisikan dan membangun database. Pada baris ke [7] dan [8], diimpor MovieDao dan MovieEntity—DAO dan entitas yang akan dikelola oleh database ini.

Pada baris ke [10] terdapat anotasi `@Database(entities = [MovieEntity::class], version = 1, exportSchema = false)`, yang menyatakan bahwa kelas berikutnya adalah Room Database dengan tabel berdasarkan MovieEntity, menggunakan skema versi 1 dan tanpa mengeksport skema ke file.

Pada baris ke [11] dideklarasikan abstract class `MovieDatabase : RoomDatabase()` sebagai subclass RoomDatabase yang akan menjadi akses utama ke database.

Pada baris ke [12] terdapat abstract fun `movieDao(): MovieDao` yang mendefinisikan fungsi abstrak untuk mendapatkan instance MovieDao, memungkinkan aplikasi melakukan operasi CRUD pada tabel movies.

Pada baris ke [14] hingga [26] terdapat companion object yang menyiapkan pola singleton untuk database agar hanya ada satu instance selama siklus hidup aplikasi.

Pada baris ke [15] anotasi `@Volatile` pada `private var INSTANCE: MovieDatabase? = null` memastikan bahwa setiap thread akan melihat pembaruan terakhir pada variabel ini.

Pada baris ke [18] fungsi `fun getDatabase(context: Context): MovieDatabase` menyediakan akses publik ke instance database.

Pada baris ke [19] digunakan pengecekan `INSTANCE ?`: untuk mengembalikan instance jika sudah ada, atau masuk ke blok `synchronized(this)` agar pembuatan instance hanya terjadi sekali secara thread-safe.

Pada baris ke [19] sampai akhir, `Room.databaseBuilder` membangun database dengan `context.applicationContext`, kelas `MovieDatabase::class.java`, dan nama file "movie_db". Metode `fallbackToDestructiveMigration()` memastikan bahwa saat versi skema berubah tanpa migrasi yang ditentukan, database lama akan dihapus dan dibuat ulang. Hasil builder kemudian dibangun dengan `build()` dan langsung di-assign ke `INSTANCE` lewat `also { INSTANCE = it }`.

MovieEntity.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.local.entity` yang berfungsi sebagai nama paket untuk entitas database lokal dan tempat menyimpan file ini dalam struktur aplikasi.

Pada baris ke [3] sampai [4], dilakukan impor Entity dan PrimaryKey dari Android Room, yang digunakan untuk menandai kelas ini sebagai tabel dan menentukan kolom kunci utama.

Pada baris ke [6] terdapat anotasi `@Entity(tableName = "movies")` yang menyatakan bahwa MovieEntity akan dipetakan ke tabel SQLite bernama movies.

Pada baris ke [7] sampai [13] dideklarasikan data class MovieEntity(...) yang merepresentasikan struktur baris pada tabel movies dengan lima properti:

Pada baris ke [8], anotasi `@PrimaryKey` pada properti val id: Int menandai id sebagai kunci utama tabel, unik untuk setiap film.

Pada baris ke [9], val title: String menyimpan judul film.

Pada baris ke [10], val overview: String menyimpan ringkasan atau sinopsis film.

Pada baris ke [11], val posterPath: String menyimpan path atau URL gambar poster film.

Pada baris ke [12], val releaseDate: String menyimpan tanggal perilisan film dalam format string.

PreferencesManager.kt

Pada baris ke [1] terdapat package com.pemrogamanmobile.movielist.data.preferences yang berfungsi sebagai nama paket untuk modul pengelolaan preferensi aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [4], dilakukan impor Context dan SharedPreferences dari Android, yang dibutuhkan untuk mengakses dan menyimpan data preferensi pengguna.

Pada baris ke [6] dideklarasikan class PreferencesManager(context: Context), sebuah kelas yang menerima Context saat instansiasi untuk menginisialisasi akses ke SharedPreferences.

Pada baris ke [7] terdapat `private val prefs: SharedPreferences = context.getSharedPreferences("movie_prefs", Context.MODE_PRIVATE)` yang membuat atau membuka file preferensi bernama "movie_prefs" dengan mode private, sehingga hanya aplikasi ini yang dapat membacanya.

Pada baris ke [9] sampai [11] didefinisikan fun saveThemeSetting(isDarkMode: Boolean), sebuah fungsi yang menyimpan pilihan tema (gelap atau terang) dengan mengedit prefs, menempatkan boolean di bawah kunci "dark_mode", lalu memanggil apply() agar perubahan langsung ditulis.

Pada baris ke [13] terdapat fun `isDarkMode()`: `Boolean = prefs.getBoolean("dark_mode", false)` yang mengembalikan nilai boolean dari preferensi "dark_mode", dengan default false jika belum pernah disimpan sebelumnya.

ApiService.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.remote.api` yang berfungsi sebagai nama paket untuk modul akses API remote dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [5], dilakukan impor `MovieResponseDto` dari paket DTO, serta anotasi `@GET` dan `@Query` dari Retrofit, yang dibutuhkan untuk mendefinisikan endpoint HTTP dan parameter query.

Pada baris ke [7] dideklarasikan interface `ApiService` sebagai kontrak yang menjelaskan bagaimana aplikasi akan berkomunikasi dengan API remote menggunakan Retrofit.

Pada baris ke [7] sampai [12] didefinisikan fungsi `getPopularMovies`:

Pada baris ke [8] anotasi `@GET("movie/popular")` menyatakan bahwa pemanggilan ini akan melakukan HTTP GET ke path `movie/popular`.

Pada baris ke [9] suspend fun `getPopularMovies()` menandai method ini sebagai suspend, sehingga dapat dipanggil dari coroutine untuk operasi non-blocking.

Pada baris ke [10] anotasi `@Query("api_key")` `apiKey: String` menetapkan bahwa parameter `apiKey` akan diteruskan sebagai query string `?api_key=...` pada request.

Pada baris ke [11] `): MovieResponseDto` menunjukkan bahwa respons JSON dari endpoint akan dipetakan menjadi objek `MovieResponseDto`.

MovieDto.kt:

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.remote.dto` yang berfungsi sebagai nama paket untuk modul DTO (Data Transfer Object) yang memetakan data JSON dari API ke objek Kotlin.

Pada baris ke [3] sampai [4], dilakukan impor `Serializable` dan `SerializedName` dari `kotlinx.serialization`, yang memungkinkan kelas ini diserialisasi dan mendeserialisasi JSON secara otomatis.

Pada baris ke [6] terdapat anotasi `@Serializable` yang menandai data class `MovieDto` sebagai objek yang dapat di-serialize dan di-deserialize oleh pustaka `kotlinx.serialization`.

Pada baris ke [7] sampai [13] dideklarasikan data class `MovieDto(...)` dengan lima properti yang memetakan satu film dari respons API:

`val id: Int` (baris ke [8]) menyimpan ID unik film.

`val title: String` (baris ke [9]) menyimpan judul film.

`val overview: String` (baris ke [10]) menyimpan sinopsis film.

`@SerializedName("poster_path") val posterPath: String` (baris ke [11]) memetakan kunci JSON "poster_path" ke properti `posterPath`.

`@SerializedName("release_date") val releaseDate: String` (baris ke [12]) memetakan kunci JSON "release_date" ke properti `releaseDate`.

Pada baris ke [15] kembali terdapat anotasi `@Serializable` yang menandai data class `MovieResponseDto` dapat di-serialize.

Pada baris ke [16] sampai [18] dideklarasikan data class `MovieResponseDto(val results: List<MovieDto>)` yang merepresentasikan struktur JSON respons top-level, dengan `results` sebagai daftar objek `MovieDto`.

RetrofitInstance.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.remote` yang berfungsi sebagai nama paket untuk modul komunikasi jaringan aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [7], dilakukan impor berbagai komponen penting untuk membangun klien Retrofit dengan dukungan serialisasi JSON: `asConverterFactory` dari Jake Wharton untuk menghubungkan `kotlinx.serialization` ke Retrofit, `ApiService` sebagai interface endpoint, `Json` dari `kotlinx.serialization` untuk konfigurasi parser, ekstensi `toMediaType` dari `OkHttp` untuk mendefinisikan media type, dan Retrofit sebagai inti HTTP client.

Pada baris ke [9] dideklarasikan object `RetrofitInstance`, sebuah singleton yang menyiapkan dan menyediakan klien Retrofit serta service API-nya.

Pada baris ke [10] terdapat `private const val BASE_URL = "https://api.themoviedb.org/3/"` yang menyimpan URL dasar untuk semua permintaan ke The Movie Database API.

Pada baris ke [12] sampai [17] didefinisikan `private val retrofit by lazy { ... }` sehingga instance Retrofit hanya dibuat sekali secara thread-safe ketika pertama kali diakses. Di dalam blok `lazy`:Dipanggil `Retrofit.Builder()` untuk memulai konstruksi klien, `baseUrl(BASE_URL)` menetapkan alamat dasar API, `addConverterFactory(...)` menambahkan converter JSON yang dikonfigurasi dengan `Json` {

`ignoreUnknownKeys = true` } agar parser mengabaikan field tak dikenal, lalu diubah menjadi `ConverterFactory` dengan media type `"application/json"`, `build()` memfinalisasi dan menghasilkan instance `Retrofit`.

Pada baris ke [19] sampai [21] terdapat `val api: ApiService by lazy { retrofit.create(ApiService::class.java) }` yang menyediakan satu-satunya instance `ApiService`, siap untuk dipanggil dari `coroutine` di seluruh aplikasi.

MovieRepositoryImpl.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.data.repository` yang berfungsi sebagai nama paket untuk modul repositori aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [9], dilakukan impor `MovieDao` dan `ApiService` sebagai komponen utama untuk akses data lokal dan remote, `Movie` sebagai model domain, `MovieRepository` sebagai kontrak repositori, `NetworkResult` untuk membungkus status jaringan, serta `Flow` dan `flow` dari `Coroutines` untuk aliran data asinkron.

Pada baris ke [11] dideklarasikan class `MovieRepositoryImpl(private val api: ApiService, private val dao: MovieDao) : MovieRepository` yang mengimplementasikan kontrak `MovieRepository`, menerima instance `ApiService` dan `MovieDao` lewat konstruktor untuk melakukan operasi data.

Pada baris ke [15] hingga [43] diimplementasikan fungsi override `fun getPopularMovies(): Flow<NetworkResult<List<Movie>>> = flow { ... }` yang mengembalikan sebuah `Flow` berisi `NetworkResult` dari daftar `Movie`. Di dalam blok `flow`:

Pada baris ke [16] `emit(NetworkResult.Loading)` mengirim status loading sebelum operasi dimulai.

Pada baris ke [17] sampai [33] dibungkus dalam `try { ... }`, di mana pertama dipanggil `api.getPopularMovies(...)` untuk mengambil data film populer, lalu hasil `response.results` dipetakan menjadi model domain `Movie`. Selanjutnya, `dao.clearMovies()` menghapus cache lama, diikuti `dao.insertMovies(...)` yang menyimpan daftar film baru sebagai `MovieEntity`. Setelah itu, `emit(NetworkResult.Success(movies))` mengirim status sukses beserta data.

Pada baris ke [33] sampai [42] terdapat blok `catch (e: Exception) { ... }` yang menangani kegagalan jaringan. Di sini, data cache diambil lewat `dao.getAllMovies()`, dipetakan menjadi model `Movie`, dan jika tidak kosong maka `emit(NetworkResult.Success(cached))` mengirim data cache; jika kosong, dikembalikan `emit(NetworkResult.Error(e.message ?: "Unknown Error"))` sebagai status error.

Penjelasan caching strategy:

Caching strategy yang digunakan adalah NetworkBoundResource versi sederhananya, Dimana aplikasi pertama kali mencoba untuk mengambil data lewat api, kemudian menyimpannya di database local menggunakan room. Jika data berhasil diambil, maka akan langsung ditampilkan. Sedangkan jika terjadi error maka mengambil data dari database local dan ditampilkan. Strategi ini dipilih karena paling sederhana dan cukup efektif untuk membuat aplikasi dapat berjalan secara offline.

Movie.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.domain.model` yang berfungsi sebagai nama paket untuk modul model domain aplikasi, tempat menyimpan definisi kelas yang merepresentasikan data bisnis.

Pada baris ke [3] terdapat deklarasi data class `Movie` (yang mendefinisikan kelas data `Movie`—sebuah model domain immutable dengan otomatisasi fungsi `equals()`, `hashCode()`, `toString()`, dan `copy()`).

Pada baris ke [4] `val id: Int` menyatakan properti `id` bertipe `Int` yang berfungsi sebagai pengenal unik (identifier) untuk setiap objek `Movie`.

Pada baris ke [5] `val title: String` mendeklarasikan properti `title` bertipe `String` untuk menyimpan judul film.

Pada baris ke [6] `val overview: String` mendefinisikan properti `overview` bertipe `String` yang berisi sinopsis atau ringkasan film.

Pada baris ke [7] `val posterPath: String` menyatakan properti `posterPath` bertipe `String` yang menyimpan path atau URL ke gambar poster film.

Pada baris ke [8] `val releaseDate: String` mendeklarasikan properti `releaseDate` bertipe `String` yang menyimpan tanggal rilis film dalam format teks.

MovieRepository.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.domain.repository` yang berfungsi sebagai nama paket untuk modul repositori domain aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [5], dilakukan impor `Movie` dari paket `domain.model` yang merepresentasikan data film di lapisan domain, `NetworkResult` dari utilitas yang membungkus status jaringan (`Loading`, `Success`, `Error`), serta `Flow` dari `Coroutines` untuk menyediakan aliran data yang bersifat asynchronous.

Pada baris ke [7] terdapat interface `MovieRepository` yang mendefinisikan kontrak repositori untuk operasi terkait entitas `Movie`, memisahkan logika pengambilan data dari implementasi spesifik.

Pada baris ke [8] dideklarasikan fun `getPopularMovies(): Flow<NetworkResult<List<Movie>>>` sebagai satu-satunya method dalam interface ini, yang mengembalikan sebuah `Flow` berisi `NetworkResult` dari daftar model `Movie`, memungkinkan konsumen menerima status loading, hasil sukses, atau error secara reaktif.

Pada baris ke [9] terdapat penutup kurung kurawal `}` yang mengakhiri blok deklarasi interface `MovieRepository`.

MovieUseCase.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.domain.usecase` yang berfungsi sebagai nama paket untuk modul use case di lapisan domain aplikasi dan tempat menyimpan file ini dalam struktur proyek.

Pada baris ke [3] sampai [6], dilakukan impor `Movie` dari paket `domain.model` untuk representasi data film di domain, `MovieRepository` dari paket `domain.repository` sebagai kontrak penyedia data, `NetworkResult` dari utilitas untuk membungkus status jaringan, serta `Flow` dari `Coroutines` untuk menyediakan aliran data yang bersifat asynchronous.

Pada baris ke [8] dideklarasikan class `MovieUseCase(private val repository: MovieRepository)` yang merupakan kelas use case dengan satu dependensi `MovieRepository`, bertugas mengeksekusi logika bisnis yang terkait dengan pengambilan film populer.

Pada baris ke [9] sampai [12] terdapat fungsi fun `getPopularMovies(): Flow<NetworkResult<List<Movie>>>` `{ return repository.getPopularMovies() }` yang memanggil langsung method `getPopularMovies` dari repository dan mengembalikan `Flow` berisi `NetworkResult` daftar `Movie`, menghubungkan lapisan domain dengan repositori tanpa menambah logika tambahan.

DetailScreen.kt

Pada baris ke [1] terdapat deklarasi package `com.pemrogamanmobile.movielist.presentation.ui.screen` yang menunjukkan bahwa file ini berada dalam struktur folder tersebut dan merupakan bagian dari modul `presentation.ui.screen` dalam aplikasi.

Pada baris ke [3] sampai [20], dilakukan impor berbagai komponen dari `Jetpack Compose` dan pustaka lain seperti `Image`, `Column`, `Row`, `Scaffold`, `Surface`, `Text`, serta fungsi `rememberScrollState`, `clip`, dan `ContentScale`. Selain itu, komponen tambahan dari `Coil` yaitu `rememberAsyncImagePainter` digunakan untuk menampilkan gambar

dari URL secara asinkron, serta ViewModel dan utilitas NetworkResult untuk mengelola data film.

Pada baris ke [22] hingga [28], didefinisikan fungsi composable DetailScreen yang menerima empat parameter: viewModel dari tipe MovieViewModel, movieId sebagai ID film yang akan ditampilkan, isDarkMode sebagai indikator mode gelap atau terang, dan onToggleTheme sebagai lambda untuk mengubah tema.

Pada baris ke [29], dibuat variabel moviesResult yang akan memantau perubahan data dari viewModel.movies secara reaktif menggunakan collectAsState.

Pada baris ke [30], scrollState dideklarasikan menggunakan rememberScrollState() agar tampilan dapat discroll secara vertikal jika kontennya melebihi ukuran layar.

Pada baris ke [31], colorScheme didefinisikan untuk mengambil skema warna aktif dari MaterialTheme, baik pada mode terang maupun gelap.

Pada baris ke [33] hingga [41], digunakan Scaffold untuk membuat struktur tata letak utama aplikasi yang mencakup topBar dan isi konten. Bagian topBar menampilkan composable TopBarApp dengan judul "Detail Movie" serta parameter untuk menangani tema gelap.

Pada baris ke [48] hingga [50], ketika moviesResult berada pada status Loading, maka ditampilkan CircularProgressIndicator di tengah layar sebagai indikator proses pemuatan data film.

Pada baris ke [53] hingga [129], jika moviesResult merupakan NetworkResult.Success, maka data film dicari berdasarkan ID menggunakan method find. Jika film ditemukan, maka akan ditampilkan informasi lengkapnya di dalam Surface yang membungkus tampilan dengan warna latar dari tema aktif.

Pada baris ke [56] hingga [61], digunakan Column yang dibungkus verticalScroll(scrollState) agar seluruh konten dapat discroll. Padding 16.dp ditambahkan untuk memberi jarak dari tepi layar.

Pada baris ke [67] hingga [75], ditampilkan gambar poster film menggunakan Image dengan painter dari URL TMDb dan contentDescription dari judul film. Gambar diberi tinggi 400.dp, lebar penuh, dan bentuk sudut membulat menggunakan clip(RoundedCornerShape(6.dp)).

Pada baris ke [77] hingga [117], Column digunakan kembali untuk menampung informasi detail film seperti judul, tanggal rilis, label deskripsi, dan sinopsis.

Pada baris ke [78] hingga [96], Row digunakan untuk menyusun judul film dan tanggal rilis secara horizontal dengan pengaturan Arrangement.SpaceBetween. Judul film

menggunakan gaya `headlineSmall` dan tanggal rilis menggunakan `bodyMedium` dengan warna teks transparan 60%.

Pada baris ke [98], `Spacer` ditambahkan setinggi 8.dp sebagai pemisah antar elemen.

Pada baris ke [100] hingga [107], ditampilkan teks "Tentang film ini:" sebagai label menggunakan gaya `bodyLarge`, ukuran font 14.sp, warna `onBackground`, dan `FontWeight.Bold`.

Pada baris ke [109] hingga [116], ditampilkan teks deskripsi film yang diambil dari `movie.overview` menggunakan gaya `bodyMedium` dengan ukuran font 14.sp, warna `onBackground`, dan jarak atas 4.dp dari label sebelumnya.

Pada baris ke [120] hingga [127], jika film dengan ID tersebut tidak ditemukan dalam data sukses, maka akan ditampilkan teks "Movie not found" dengan warna error dari tema dan padding 16.dp.

Pada baris ke [131] hingga [139], jika `moviesResult` merupakan `NetworkResult.Error`, maka akan ditampilkan teks "Error loading movie" dengan warna error dan padding yang sama seperti kondisi film tidak ditemukan.

MainScreen.kt

Pada baris ke [1] terdapat deklarasi `package com.pemrogramanmobile.movielist.presentation.ui.screen` yang menunjukkan bahwa file ini berada dalam struktur paket bernama `presentation.ui.screen` di dalam modul `movielist` dari aplikasi. Package ini digunakan untuk mengelompokkan dan mengorganisasi kode agar lebih rapi.

Pada baris ke [3] sampai [13], dilakukan impor berbagai komponen dari Android Jetpack Compose seperti `layout`, `lazy column`, `material3`, serta elemen-elemen penting seperti `Modifier`, `Alignment`, dan `dp` (`density-independent pixels`). Selain itu, juga diimpor `NavController` dari `androidx.navigation` untuk navigasi antar layar, serta `MovieViewModel` sebagai `ViewModel` yang menyimpan dan mengelola data film, dan `NetworkResult` yang digunakan untuk merepresentasikan status permintaan data (`loading`, `sukses`, atau `error`).

Pada baris ke [15] sampai [21], terdapat fungsi `composable` bernama `MainScreen` yang menerima empat parameter: `viewModel` dari tipe `MovieViewModel` untuk mengakses data film, `navController` dari tipe `NavController` untuk navigasi antar layar, `isDarkMode` bertipe `Boolean` untuk menentukan apakah tema gelap sedang aktif, dan `onToggleTheme`, yaitu `lambda function` yang dipanggil saat pengguna ingin mengganti tema aplikasi.

Pada baris ke [22] sampai [23], dideklarasikan sebuah variabel `moviesResult` menggunakan delegasi `by` dari `collectAsState()` untuk memantau perubahan aliran data

film dari `viewModel.movies`. Nilai ini akan otomatis diperbarui saat data dalam flow berubah. Selain itu, `colorScheme` diambil dari `MaterialTheme.colorScheme` untuk mendapatkan skema warna saat ini (termasuk tema terang atau gelap).

Pada baris ke [25] sampai [33], digunakan komponen `Scaffold` untuk membuat struktur layar standar dengan top bar dan konten utama. Di dalam `topBar`, fungsi `TopAppBar` dipanggil dengan parameter `title`, `isDarkMode`, dan `onToggleTheme` untuk menampilkan judul halaman dan menyediakan tombol pengganti tema.

Pada baris ke [34] hingga [39], elemen `Surface` digunakan untuk membungkus isi tampilan utama layar. Modifier digunakan untuk mengisi seluruh ukuran layar dan memberi padding sesuai dengan `paddingValues` dari `Scaffold`. Warna latar belakang diatur menggunakan `colorScheme.background` untuk menyesuaikan dengan tema saat ini.

Pada baris ke [40] hingga [45], terdapat pengecekan kondisi status `moviesResult` menggunakan `when`. Jika statusnya adalah `Loading`, maka akan ditampilkan `CircularProgressIndicator` yang ditempatkan di tengah layar untuk menandakan bahwa data sedang dimuat.

Pada baris ke [47] hingga [59], jika statusnya adalah `Success`, maka data film akan diambil dari `moviesResult` yang dikonversi menjadi tipe `NetworkResult.Success`, kemudian hasilnya ditampilkan dalam `LazyColumn`. Setiap item dalam daftar akan digambarkan menggunakan `MovieCard`, dan ketika sebuah kartu ditekan, maka akan menavigasi ke halaman detail film berdasarkan `movie.id`.

Pada baris ke [61] hingga [67], jika statusnya adalah `Error`, maka akan ditampilkan teks dengan pesan error yang diambil dari `moviesResult` sebagai `NetworkResult.Error`. Teks ini diberi warna `colorScheme.error` dan padding sebesar 16.dp agar tidak menempel di tepi layar.

MovieCard.kt

Pada baris ke [1] terdapat deklarasi `package com.pemrogamanmobile.movieslist.presentation.ui.screen` yang berfungsi sebagai penanda lokasi file ini dalam struktur paket proyek Android, serta membantu dalam pengorganisasian dan pemanggilan kelas atau fungsi dari bagian lain aplikasi.

Pada baris ke [3] sampai [22], dilakukan impor berbagai komponen dari Jetpack Compose seperti `Image`, `Row`, `Column`, `Spacer`, `Card`, `Text`, dan lain-lain, yang digunakan untuk membangun antarmuka pengguna. Selain itu, digunakan juga komponen tema dari `MaterialTheme`, `Button`, `RoundedCornerShape`, serta `ContentScale` dan `FontWeight`. Library `Coil` juga diimpor untuk memuat gambar secara asynchronous melalui `rememberAsyncImagePainter`. Impor lainnya mencakup `Movie`, yaitu model data domain untuk informasi film.

Pada baris ke [24] sampai [129], terdapat fungsi `composable` bernama `MovieCard` yang menerima dua parameter, yaitu `movie` bertipe `Movie` untuk menampilkan data film, dan `onClick` sebagai fungsi `lambda` yang dijalankan saat tombol “Detail” ditekan.

Pada baris ke [26], dideklarasikan `val colorScheme = MaterialTheme.colorScheme` untuk mengambil skema warna saat ini dari tema `Material3`, agar elemen UI dapat mengikuti warna yang konsisten dalam aplikasi.

Pada baris ke [28] hingga [35], digunakan komponen `Card` dari `Material3` sebagai wadah utama tampilan kartu film. Kartu ini dibuat memenuhi lebar penuh dengan `padding 12.dp`, serta diberi warna latar belakang sesuai warna utama dari tema menggunakan `CardDefaults.cardColors`.

Pada baris ke [36], digunakan `Row` agar isi dari kartu tersusun secara horizontal, sehingga gambar poster film dan informasi lainnya dapat ditampilkan sejajar dari kiri ke kanan.

Pada baris ke [38] sampai [48], digunakan `Image` untuk menampilkan poster film yang diambil dari URL menggunakan `path movie.posterPath` dan dimuat menggunakan `rememberAsyncImagePainter`. Gambar diberi `padding` kiri `12.dp`, lebar `100.dp`, tinggi `140.dp`, serta dipotong menggunakan `clip(RoundedCornerShape(6.dp))` agar sudutnya membulat. Posisi gambar disejajarkan ke tengah secara vertikal dan menggunakan `ContentScale.Crop` agar konten gambar menyesuaikan kotak.

Pada baris ke [50] hingga [126], digunakan `Column` di sebelah kanan gambar sebagai kolom utama yang memuat informasi tentang film. Kolom ini diberi `padding 16.dp` dan mengisi lebar penuh agar semua elemen teks dan tombol dapat ditata secara vertikal.

Pada baris ke [56] hingga [83], digunakan `Row` untuk menampilkan judul film dan tanggal rilis secara berdampingan di bagian atas kolom. Judul film ditampilkan menggunakan `Text` dengan ukuran font `26.sp`, tebal (`FontWeight.Bold`), dan warna teks menyesuaikan `colorScheme.onSurface`. Teks judul juga diberikan `Modifier.weight(1f)` agar bisa mengisi ruang tersisa. Di sampingnya, terdapat teks `releaseDate` yang ditampilkan dengan ukuran font `16.sp` dan warna teks `onSurfaceVariant`.

Pada baris ke [85], diberikan `Spacer` dengan tinggi `8.dp` untuk memberi jarak antara bagian judul dan deskripsi.

Pada baris ke [87] hingga [104], digunakan `Column` untuk menampilkan teks label “tentang film ini:” dan overview dari film. Label tersebut menggunakan font tebal dan ukuran kecil (`14.sp`), dan deskripsi film ditampilkan di bawahnya dengan warna teks yang sama tapi tidak ditebalkan, serta diberi `padding` atas `4.dp` untuk memisahkannya dari label.

Pada baris ke [106], diberikan lagi Spacer dengan tinggi 12.dp untuk memberi jarak antara bagian deskripsi dan tombol.

Pada baris ke [108] hingga [125], digunakan Row untuk menempatkan tombol “Detail” di bagian kanan bawah kartu. Row ini mengatur agar tombol disejajarkan ke kanan menggunakan Arrangement.End dan berada di tengah secara vertikal.

Pada baris ke [113] hingga [124], dibuat Button dengan lebar 100.dp dan tinggi 44.dp yang akan memanggil fungsi onClick saat ditekan. Tombol ini menggunakan warna latar onSurface dan teks berwarna onPrimary, serta label tombol ditulis “Detail” dengan ukuran font 16.sp menggunakan Text.

TopBarApp.kt

Pada baris ke [1] terdapat deklarasi package com.pemrogramanmobile.movielist.presentation.ui.screen yang menunjukkan bahwa file ini berada dalam paket ui.screen dari modul presentation dalam aplikasi movielist, yang merupakan bagian dari struktur organisasi kode dalam proyek Android.

Pada baris ke [3] sampai [7], dilakukan impor berbagai komponen dari Jetpack Compose seperti ikon DarkMode dan LightMode dari material.icons.filled, serta berbagai elemen dari material3 seperti TopAppBar, Text, Icon, IconButton, dan MaterialTheme, yang semuanya diperlukan untuk membangun tampilan bar atas (Top App Bar) dengan ikon pengubah mode gelap dan terang.

Pada baris ke [9] digunakan anotasi @OptIn(ExperimentalMaterial3Api::class) yang berarti bahwa fungsi ini menggunakan fitur eksperimental dari Material3 API yang belum sepenuhnya stabil, sehingga diperlukan opt-in secara eksplisit untuk menghindari peringatan saat kompilasi.

Pada baris ke [10] sampai [15], dideklarasikan sebuah fungsi composable bernama TopBarApp yang menerima tiga parameter: title bertipe String sebagai judul yang akan ditampilkan di bar atas, isDarkMode bertipe Boolean untuk menentukan apakah saat ini sedang dalam mode gelap, dan onToggleTheme, sebuah lambda tanpa parameter yang akan dipanggil ketika ikon tema ditekan.

Pada baris ke [16] hingga [19], digunakan komponen TopAppBar dari Material3 untuk membangun tampilan bar atas. Pada parameter title, terdapat Text(text = title) yang akan menampilkan teks judul sesuai nilai parameter title.

Pada baris ke [20] sampai [27], didefinisikan bagian actions dari TopAppBar yang berisi sebuah IconButton dengan aksi onClick = onToggleTheme. Ketika tombol ditekan, lambda onToggleTheme akan dipanggil untuk mengubah tema. Di dalam IconButton, terdapat Icon yang akan menampilkan ikon LightMode jika sedang dalam mode gelap (isDarkMode == true), dan ikon DarkMode jika sedang dalam mode terang.

Pada baris ke [28] sampai [32], terdapat parameter colors yang menggunakan TopAppBarDefaults.topAppBarColors untuk menetapkan skema warna dari TopAppBar, yaitu warna latar belakang diambil dari MaterialTheme.colorScheme.primary, warna teks judul dari onPrimary, dan warna ikon aksi juga dari onPrimary agar kontras dengan warna latar.

MainActivity.kt

Pada baris ke [1] terdapat deklarasi package com.pemrogramanmobile.movielist.presentation.ui yang menunjukkan bahwa file ini berada di dalam paket bernama presentation.ui dari aplikasi movielist. Paket ini berfungsi sebagai wadah untuk menyusun file berdasarkan fitur tampilan (UI) aplikasi.

Pada baris ke [3] sampai [25], dilakukan impor berbagai komponen penting dari Android Jetpack dan library Compose seperti ComponentActivity, setContent, Material3, dan NavHost. Impor lainnya mencakup rememberNavController untuk navigasi antar layar, ViewModelProvider untuk menginisialisasi ViewModel, serta kelas-kelas dari folder data, domain, dan presentation untuk menghubungkan database, API, ViewModel, dan UI aplikasi.

Pada baris ke [27], terdapat deklarasi kelas MainActivity yang merupakan turunan dari ComponentActivity. Kelas ini berperan sebagai entry point utama dari aplikasi, di mana seluruh logika UI pertama kali dijalankan melalui fungsi onCreate.

Pada baris ke [29] terdapat override dari fungsi onCreate, yang akan dieksekusi saat MainActivity pertama kali dijalankan. Parameter savedInstanceState digunakan untuk menyimpan dan memulihkan status aplikasi jika dibutuhkan.

Pada baris ke [31] dibuat instance dari database lokal menggunakan MovieDatabase.getDatabase(applicationContext), yang akan mengakses atau membuat database SQLite melalui Room.

Pada baris ke [32] dibuat instance repository dari MovieRepositoryImpl yang menerima API Retrofit dan DAO dari database untuk melakukan operasi data baik secara lokal maupun jaringan.

Pada baris ke [33] dibuat instance useCase dari MovieUseCase, yang berfungsi sebagai perantara antara ViewModel dan Repository untuk mengatur logika bisnis aplikasi.

Pada baris ke [34] dideklarasikan movieViewModel dengan menggunakan ViewModelProvider dan MovieViewModelFactory. ViewModel ini mengatur data dan status tampilan utama dari daftar film di aplikasi.

Pada baris ke [36] dibuat instance `themeViewModel` menggunakan `ThemeViewModelFactory`, yang digunakan untuk mengatur dan menyimpan preferensi tema gelap atau terang berdasarkan status yang tersimpan di `DataStore`.

Pada baris ke [38] dipanggil fungsi `setContent` yang akan menggantikan isi tampilan aktivitas menggunakan `Compose`. Seluruh UI ditulis dalam blok `lambda` ini.

Pada baris ke [40] digunakan delegasi `by` untuk mengamati perubahan data `isDarkMode` dari `themeViewModel` secara otomatis menggunakan `collectAsState()` yang merupakan ekstensi `Compose` untuk `Flow`.

Pada baris ke [42] dipanggil fungsi `MovieListTheme` dengan parameter `darkTheme = isDarkMode`, yang akan mengatur apakah tampilan menggunakan tema gelap atau terang berdasarkan nilai boolean `isDarkMode`.

Pada baris ke [43] digunakan `Surface` sebagai wadah utama untuk seluruh konten UI, dan diberi warna latar `Color(0xFF131418)`, yaitu warna gelap yang akan digunakan sebagai background utama aplikasi.

Pada baris ke [44] dibuat objek `navController` menggunakan `rememberNavController()`, yang digunakan untuk mengatur navigasi antar layar dalam aplikasi.

Pada baris ke [45] hingga [66] digunakan `NavHost` dengan `navController` dan `startDestination` bernilai `"main"`, yang menunjukkan bahwa layar utama aplikasi adalah halaman utama. Di dalam `NavHost` terdapat dua rute: rute pertama `"main"` akan memanggil composable `MainScreen`, dan rute kedua `"detail/{id}"` akan memanggil composable `DetailScreen` dengan parameter `id` bertipe `integer`.

Pada baris ke [46] sampai [53], dipanggil `MainScreen` saat rute `"main"` diakses. Di dalamnya dikirim parameter `viewModel` untuk data film, `navController` untuk navigasi, `isDarkMode` untuk status tema saat ini, dan `onToggleTheme` sebagai `lambda` untuk mengganti tema saat tombol ditekan.

Pada baris ke [54] sampai [65] didefinisikan rute `"detail/{id}"` yang menerima argumen berupa `id` bertipe `IntType`. Argumen ini diambil dari `BackStackEntry`, dan jika tidak ada maka diberi nilai default `-1`. Setelah itu, fungsi `DetailScreen` dipanggil dengan parameter `movieId`, `viewModel`, `isDarkMode`, dan `onToggleTheme`.

Pada baris ke [63] dipanggil `lambda onToggleTheme` yang akan mengeksekusi method `toggleTheme()` dari `themeViewModel` untuk mengganti mode tampilan gelap atau terang aplikasi.

MovieViewModel.kt

Pada baris ke [1] terdapat package `com.pemrogramanmobile.movielist.presentation.viewmodel` yang berfungsi sebagai penamaan paket tempat file ini berada dan mengelompokkan file `ViewModel` agar mudah dikelola sesuai struktur arsitektur aplikasi.

Pada baris ke [3] hingga [4], dilakukan impor `ViewModel` dan `viewModelScope` dari `AndroidX Lifecycle` yang diperlukan untuk membuat kelas `ViewModel` dan menjalankan coroutine di dalamnya agar tidak memblokir thread UI.

Pada baris ke [5] diimpor model data `Movie` dari domain model, yang mewakili entitas film yang digunakan dalam logika bisnis.

Pada baris ke [6] dilakukan impor `MovieUseCase`, yaitu class yang menyimpan logika pemanggilan data film dari repository yang digunakan oleh `ViewModel`.

Pada baris ke [7] diimpor `NetworkResult`, sebuah kelas pembungkus hasil jaringan (sukses, loading, error) yang berguna untuk menangani status pengambilan data dari API.

Pada baris ke [8] sampai [9], dilakukan impor `MutableStateFlow` dan `StateFlow` dari `Kotlin Coroutines Flow`, yang digunakan untuk menyimpan dan mengalirkan status data film ke UI secara reaktif.

Pada baris ke [10] diimpor `launch` dari `Kotlin Coroutines` untuk menjalankan fungsi secara asinkron di dalam coroutine scope milik `ViewModel`.

Pada baris ke [12], dibuat sebuah class bernama `MovieViewModel` yang merupakan turunan dari `ViewModel`, dengan parameter konstruktor `movieUseCase` yang bertipe `MovieUseCase`, digunakan sebagai perantara untuk mengambil data film dari repository.

Pada baris ke [13], dideklarasikan variabel privat `_movies` bertipe `MutableStateFlow` yang menyimpan data berupa objek `NetworkResult<List<Movie>>`, dan langsung diinisialisasi dengan nilai `Loading` sebagai status awal.

Pada baris ke [14], dibuat properti publik `movies` bertipe `StateFlow` yang hanya menyediakan getter dari `_movies`, agar data hanya bisa dibaca dari luar `ViewModel` namun tidak bisa diubah secara langsung.

Pada baris ke [16], terdapat blok `init` yang akan dipanggil saat objek `MovieViewModel` pertama kali dibuat, dan di dalamnya langsung memanggil fungsi `fetchMovies()` untuk memulai pengambilan data film.

Pada baris ke [20], dibuat fungsi `fetchMovies()` yang digunakan untuk mengambil data film populer dari `MovieUseCase` secara asynchronous.

Pada baris ke [21], fungsi ini menjalankan coroutine menggunakan `viewModelScope.launch` agar tetap terikat ke siklus hidup `ViewModel` dan tidak menyebabkan memory leak.

Pada baris ke [22], `movieUseCase.getPopularMovies()` dipanggil dan dikumpulkan (collect) sebagai aliran data `Flow`, yang setiap hasilnya akan diteruskan ke `_movies.value`.

Pada baris ke [23], nilai hasil pengambilan film disimpan ke dalam `_movies.value`, yang nantinya akan diamati oleh UI untuk ditampilkan berdasarkan status loading, sukses, atau error.

MovieViewModelFactory.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.presentation.viewmodel` yang menunjukkan bahwa file ini merupakan bagian dari struktur paket aplikasi yang mengatur komponen ViewModel pada layer presentasi.

Pada baris ke [3] dan [4], dilakukan impor class `ViewModel` dan `ViewModelProvider` dari `AndroidX Lifecycle`, yang dibutuhkan untuk membuat dan mengelola `ViewModel` secara dinamis, terutama saat menggunakan `dependency injection` manual tanpa framework seperti `Hilt` atau `Dagger`.

Pada baris ke [5] diimpor `MovieUseCase`, yaitu class yang bertanggung jawab menyediakan logika bisnis untuk pengambilan data film yang akan digunakan oleh `ViewModel`.

Pada baris ke [7], dibuat class `MovieViewModelFactory` yang merupakan turunan dari `ViewModelProvider.Factory`, dan berfungsi sebagai pabrik (factory) untuk membuat instance `MovieViewModel` dengan parameter `useCase` yang diteruskan melalui konstruktor.

Pada baris ke [8], dilakukan override terhadap fungsi `create` dari interface `ViewModelProvider.Factory`, dengan parameter `modelClass` bertipe `Class<T>`, untuk menentukan `ViewModel` apa yang akan dibuat berdasarkan tipe yang diminta.

Pada baris ke [9] sampai [11], dilakukan pengecekan apakah `modelClass` yang diminta merupakan turunan dari `MovieViewModel`. Jika benar, maka akan mengembalikan instance `MovieViewModel` yang dibuat menggunakan `useCase` yang sudah disediakan, dan di-casting menjadi tipe generik `T`.

Pada baris ke [12] dibuat pengecualian menggunakan `throw IllegalArgumentException` dengan pesan “Unknown ViewModel class” apabila `modelClass` yang diminta bukan `MovieViewModel`, sebagai penanganan jika `ViewModel` yang tidak dikenali diminta untuk dibuat.

ThemeViewModel.kt

Pada baris ke [1] terdapat package `com.pemrogamanmobile.movielist.presentation.viewmodel` yang menunjukkan bahwa file ini termasuk dalam struktur modul presentasi aplikasi, khususnya digunakan untuk menyimpan dan mengelola logic `ViewModel` yang berkaitan dengan tampilan aplikasi.

Pada baris ke [3] sampai [5], dilakukan impor class `ViewModel` dari `AndroidX Lifecycle` untuk membuat class berbasis `ViewModel`, `viewModelScope` untuk menjalankan coroutine di dalam lifecycle `ViewModel`, serta `PreferencesManager` yang

merupakan class kustom untuk mengelola penyimpanan preferensi tema gelap atau terang.

Pada baris ke [6] dan [7], diimpor `MutableStateFlow` dan `StateFlow` dari Kotlin Coroutines Flow, yang digunakan untuk menyimpan dan mengamati status mode tema secara reaktif agar dapat terintegrasi dengan UI secara otomatis.

Pada baris ke [10] dideklarasikan class `ThemeViewModel` yang merupakan turunan dari `ViewModel`, dengan konstruktor yang menerima parameter `preferencesManager` bertipe `PreferencesManager` untuk membaca dan menyimpan status tema dari penyimpanan lokal.

Pada baris ke [12] dibuat variabel privat `_isDarkMode` bertipe `MutableStateFlow` dengan nilai awal yang diambil dari fungsi `preferencesManager.isDarkMode()`, yang mengembalikan apakah saat ini aplikasi menggunakan mode gelap atau tidak.

Pada baris ke [13] dideklarasikan variabel publik `isDarkMode` bertipe `StateFlow<Boolean>` yang merepresentasikan status terkini mode tema dan hanya bisa dibaca dari luar class, sedangkan perubahan hanya dapat dilakukan dari dalam `ViewModel`.

Pada baris ke [15] sampai [21] dibuat fungsi `toggleTheme()` yang akan membalikkan status tema saat ini. Pertama, nilai mode saat ini dibalik, lalu disimpan ke `_isDarkMode`. Kemudian, di dalam coroutine menggunakan `viewModelScope.launch`, perubahan tema tersebut juga disimpan secara permanen melalui fungsi `preferencesManager.saveThemeSetting(newMode)`.

ThemeViewModelFactory.kt

Pada baris ke [1] terdapat package `com.pemrogramanmobile.movielist.presentation.viewmodel` yang menunjukkan bahwa file ini berada dalam direktori presentasi pada struktur aplikasi dan berfungsi untuk menyimpan logika `ViewModel` yang berkaitan dengan tampilan antarmuka pengguna.

Pada baris ke [3] sampai [6], dilakukan impor class `Context` dari Android, `ViewModel` dan `ViewModelProvider` dari `AndroidX Lifecycle`, serta class `PreferencesManager` dari package `data.preferences`, yang keseluruhannya diperlukan untuk membuat dan menyediakan `ViewModel` dengan dependensi tambahan berupa konteks aplikasi.

Pada baris ke [8] dideklarasikan class `ThemeViewModelFactory` yang mengimplementasikan interface `ViewModelProvider.Factory`, dengan satu parameter konstruktor berupa `context` yang digunakan untuk menginisialisasi class `PreferencesManager`.

Pada baris ke [9] sampai [12] dibuat override fungsi `create()` yang menerima parameter `modelClass` bertipe `Class<T>` untuk menentukan tipe `ViewModel` yang ingin dibuat.

Di dalam fungsi ini, objek prefs bertipe PreferencesManager diinisialisasi menggunakan context yang diberikan.

Pada baris ke [11] nilai prefs kemudian digunakan untuk membuat instance dari ThemeViewModel, yang kemudian dikembalikan dengan melakukan casting ke tipe T agar sesuai dengan parameter generik fungsi create.

NetworkResult.kt

Pada baris ke [1] terdapat package com.pemrogamanmobile.movielist.utils yang menunjukkan bahwa file ini berada dalam direktori utilitas dari aplikasi, dan berfungsi untuk menyimpan class-class bantu atau kelas utilitas seperti pengelola hasil dari permintaan jaringan.

Pada baris ke [3] dideklarasikan class NetworkResult sebagai sealed class dengan parameter generik T, yang berarti hanya subclass yang didefinisikan di dalam file ini yang dapat menjadi turunan dari NetworkResult. Ini berguna untuk membatasi hasil respons jaringan agar hanya memiliki beberapa kemungkinan keadaan yang spesifik.

Pada baris ke [4] dibuat subclass Success sebagai data class dengan parameter data bertipe generik T, yang mewakili keadaan ketika permintaan jaringan berhasil dan mengembalikan data.

Pada baris ke [5] didefinisikan subclass Error sebagai data class dengan parameter message bertipe String, yang menunjukkan bahwa permintaan jaringan gagal dan mengembalikan pesan kesalahan.

Pada baris ke [6] terdapat deklarasi object Loading yang merupakan turunan dari NetworkResult dan menandakan bahwa proses permintaan jaringan sedang berlangsung tanpa membawa data apapun. menyuntikkan getMoviesUseCase yang telah diterima oleh factory ini.

Pada baris [15], @Suppress("UNCHECKED_CAST") digunakan untuk menekan peringatan kompilasi karena casting dari MovieListViewModel ke T tidak dapat diperiksa secara ketat pada waktu kompilasi. Ini aman karena pemeriksaan isAssignableFrom sudah memastikan jenis yang benar.

Pada baris [17], jika modelClass yang diminta bukan MovieListViewModel, throw IllegalArgumentException("Unknown ViewModel class") akan dilemparkan, mengindikasikan bahwa factory ini tidak tahu cara membuat ViewModel untuk kelas yang diminta.

Tautan Git

Berikut adalah tautan untuk semua source code yang telah dibuat.

https://github.com/Harry154git/Pemrograman_mobile